

第7章: データベース設計 (PostgreSQL + Prisma)

この章では、BON-LOG（盆栽SNS）のデータベース設計を学びます。データベースはアプリケーションの「記憶」にあたる部分です。ユーザー情報、投稿、コメント、いいね、フォロー関係など、あらゆるデータを安全かつ効率的に保存・取得するための仕組みを、基礎から順を追って解説します。

7.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアル](#) の [進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
RDB・Prisma 概要	—	テーブル・リレーション・ORM の概念が分かる
セットアップ	<code>.env</code> , <code>prisma/schema.prisma</code>	<code>npx prisma generate</code> が通る
スキーマ定義	<code>prisma/schema.prisma</code>	モデルが定義され、 <code>db push</code> が成功する
BON-LOG 設計・CRUD	<code>prisma/schema.prisma</code> , <code>lib/db.ts</code>	実際のモデルで CRUD が書ける
マイグレーション	<code>prisma/migrations/*</code>	スキーマ変更が履歴管理できる

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとうなる・このあと変わること・確認方法** を確認しながら進めてください。

本章で登場する専門用語一覧（初心者向け）

この章には多くの専門用語が登場します。先に全体像を把握しておくことで、各セクションの理解がスムーズになります。詳細はそれぞれの登場箇所です改めて解説しますので、ここではざっくりとした意味を掴んでおいてください。

用語クイックリファレンス:

データベースの基本概念

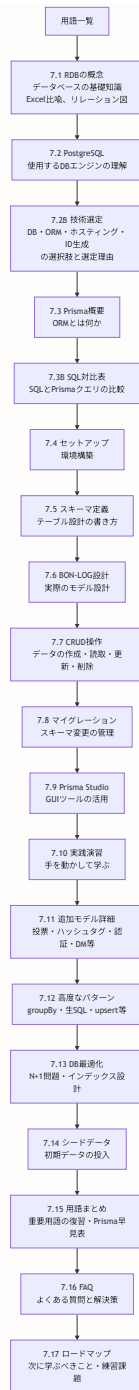
用語	ひとことと言うと
データベース (DB)	データを整理して保存・検索するための仕組み
リレーショナルデータベース (RDB)	データを「表 (テーブル)」の形で管理する方式。テーブル同士を「関連 (リレーション)」で結ぶ
テーブル	データを格納する「表」。Excelのシートに相当
カラム (列)	テーブルの「項目」。Excelの列見出しに相当
レコード (行)	テーブルの「1件分のデータ」。Excelの1行に相当
主キー (Primary Key)	各レコードを一意に識別するID。マイナンバーのようなもの
外部キー (Foreign Key)	別テーブルのレコードを参照するキー。「この投稿は誰のもの？」を示す紐付け
インデックス	検索を高速化する「索引」。本の巻末索引と同じ発想
スキーマ	データベースの「設計図」。テーブル構造の定義書

操作と管理の用語

用語	ひとことと言うと
CRUD	データの4大操作: 作成(Create)・読取(Read)・更新(Update)・削除(Delete) の頭文字
SQL	データベースに命令を出すための専用言語 (SELECT, INSERT, UPDATE, DELETE など)
トランザクション	「全部成功 or 全部取消」を保証する仕組み。銀行振込で「引き落とし+入金」をセットで行うイメージ
マイグレーション	テーブル構造の変更を記録・管理する仕組み。Gitがコードの変更履歴を管理するのと似ている
ORM	プログラムからDBを操作するための「通訳」ライブラリ。SQLを書かずにTypeScriptでDBを扱える
N+1問題	ループ内でDBに何度も問い合わせしてしまう性能問題。1回でまとめて取得すれば解決できる

初心者の方へ: 全部暗記する必要はありません。読み進める中で「あの用語なんだっけ？」と思ったら、このリストに戻って確認してください。各用語は登場する箇所で改めて丁寧に解説しています。

本章の全体像:



7.1 リレーショナルデータベースの概念

このセクションで学ぶこと

- データベースとは何か、なぜ必要なのか
- リレーショナルデータベースの基本用語（テーブル、行、列、キー）
- テーブル間の関係（リレーション）の3つの種類
- 現実世界の例え話を通じた直感的な理解

7.1.1 リレーショナルデータベースとは

リレーショナルデータベース（RDB）は、データを「テーブル」という表形式で管理するデータベースです。SNSのような複雑なアプリケーションでは、ユーザー情報、投稿、コメント、フォロー関係など、様々なデータを効率的に保存・取得する必要があります。

データベースは「Excelの超強力版」

データベースを最も直感的に理解するには、**Excelスプレッドシートの超強力版**だと考えるのが一番です。

Excelとデータベースの比較:

Excel	→	リレーショナルデータベース
ファイル(.xlsx)	→	データベース
シート (Sheet1, ...)	→	テーブル (users, posts, ...)
列 (A列, B列, ...)	→	カラム (id, email, ...)
行 (1行目, 2行目...)	→	レコード (1件目, 2件目...)
セルの値	→	フィールドの値

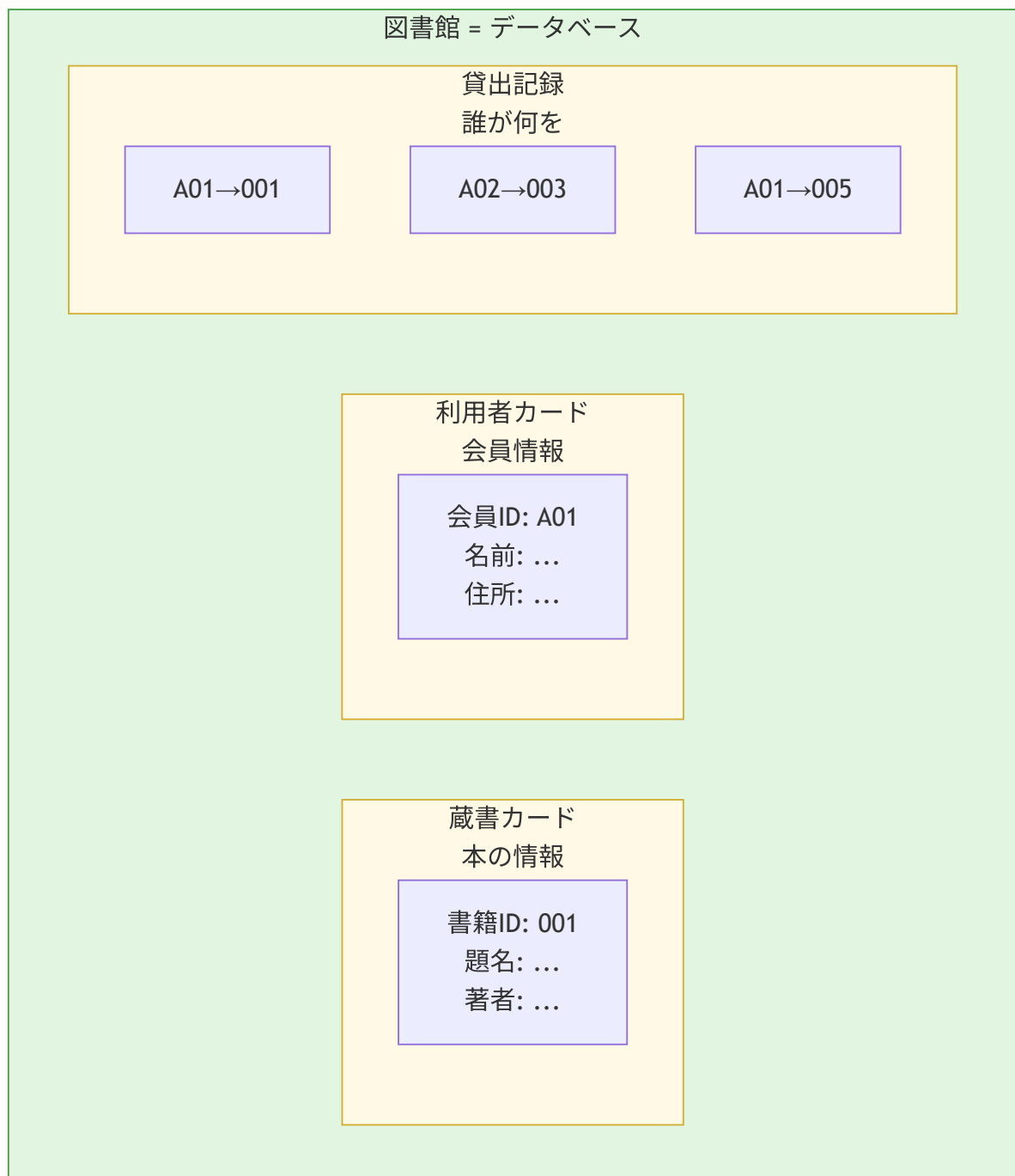
でも、データベースにはExcelにない「超能力」がある:

超能力	Excel	データベース
1. 同時アクセス	複数人が同時編集すると壊れることがある	1,000人が同時にアクセスしても安全にデータを処理
2. データの整合性保証	「会員番号」に重複した値を入力できてしまう	「主キー」や「ユニーク制約」で重複を自動的に拒否
3. テーブル間のリレーション	VLOOKUP関数で別シートを参照（手動で設定、壊れやすい）	「外部キー」で別テーブルとの関係を定義（自動で整合性維持）
4. 高速な検索	10万行あるとCtrl+Fで検索が遅い	「インデックス」で10万件でも一瞬で検索
5. トランザクション	途中で保存に失敗すると中途半端な状態になる	「全部成功」か「全部取消」の二択を保証

つまり: データベースは「数千万件のデータを、大勢が同時に、安全に、高速に扱える仕組み」です。Excelでは10万行を超えると操作が重くなりますが、データベースは億単位のレコードでも扱えます。

現実世界のたとえ

データベースを理解するために、図書館を想像してみましょう。



図書館のたとえ:

- 図書館 = データベース
- 各カード棚 = テーブル
- 各カード = 行 (レコード)
- カードの項目 = 列 (カラム)

BON-LOGでも同様に、ユーザー情報は「usersテーブル」に、投稿は「postsテーブル」に、いいねは「likesテーブル」に、それぞれ整理して保存します。テーブル同士は「外部キー」でつながっており、「この投稿は誰が書いたか」「このいいねはどの投稿に対するものか」といった関係を表現できます。

基本用語

用語	英語名	説明	図書館での例え
テーブル	Table	データを格納する表	カード棚（蔵書カード棚、利用者カード棚）
行（レコード）	Row/Record	テーブル内の1件のデータ	1枚のカード（1人分の利用者情報）
列（カラム）	Column/Field	データの属性・項目	カードの記入欄（名前欄、住所欄）
主キー	Primary Key	各行を一意に識別するID	会員番号（絶対に重複しない）
外部キー	Foreign Key	別テーブルとの関連を示すキー	貸出記録の「会員番号」欄
インデックス	Index	検索を高速化する索引	五十音順の索引カード

初心者向けポイント: 主キー（Primary Key）は「マイナンバー」のようなものです。日本に同じマイナンバーを持つ人が2人いないように、テーブル内で主キーの値が重複することは絶対にありません。これにより、どのレコードも一意に特定できます。

テーブル設計の例

users テーブル

id	email	nickname	createdAt
u1	alice@example.com	Alice	2024-01-01 10:00:00
u2	bob@example.com	Bob	2024-01-02 11:00:00

posts テーブル

id	userId	content	createdAt
p1	u1	初めての投稿です	2024-01-03 12:00:00
p2	u2	よろしく！	2024-01-03 13:00:00
p3	u1	盆栽楽しい	2024-01-04 14:00:00

usersテーブルとpostsテーブルの関係:

users テーブル				posts テーブル			
id	email	nickname		id	userId	content	
u1	a@...	Alice	→	p1	u1	初めての投稿です	
u2	b@...	Bob	→	p3	u1	盆栽楽しい	
			→	p2	u2	よろしく!	

主キー(id)で一意に識別 外部キー(userId)でどのユーザーの投稿か紐付け

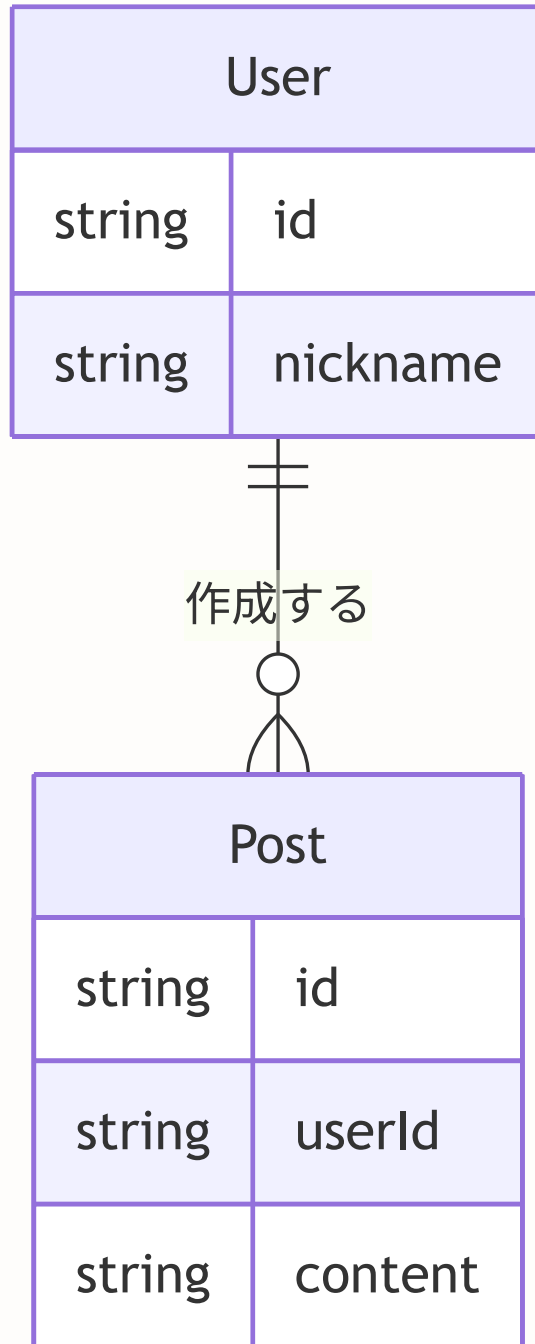
なぜ外部キーが重要なのか?: もし外部キーがなかったら、「この投稿は誰が書いたのか?」がわかりません。外部キーを使うことで、テーブル間の関係を明確に定義し、データの整合性（矛盾がない状態）を保てます。

7.1.2 リレーションの種類

データベースのテーブル間の関係（リレーション）には、大きく3つの種類があります。これを現実世界の例えとともに理解しましょう。

1対多（One-to-Many） -- 最も頻出

1つのユーザーが複数の投稿を持つ関係。最も一般的なリレーションです。

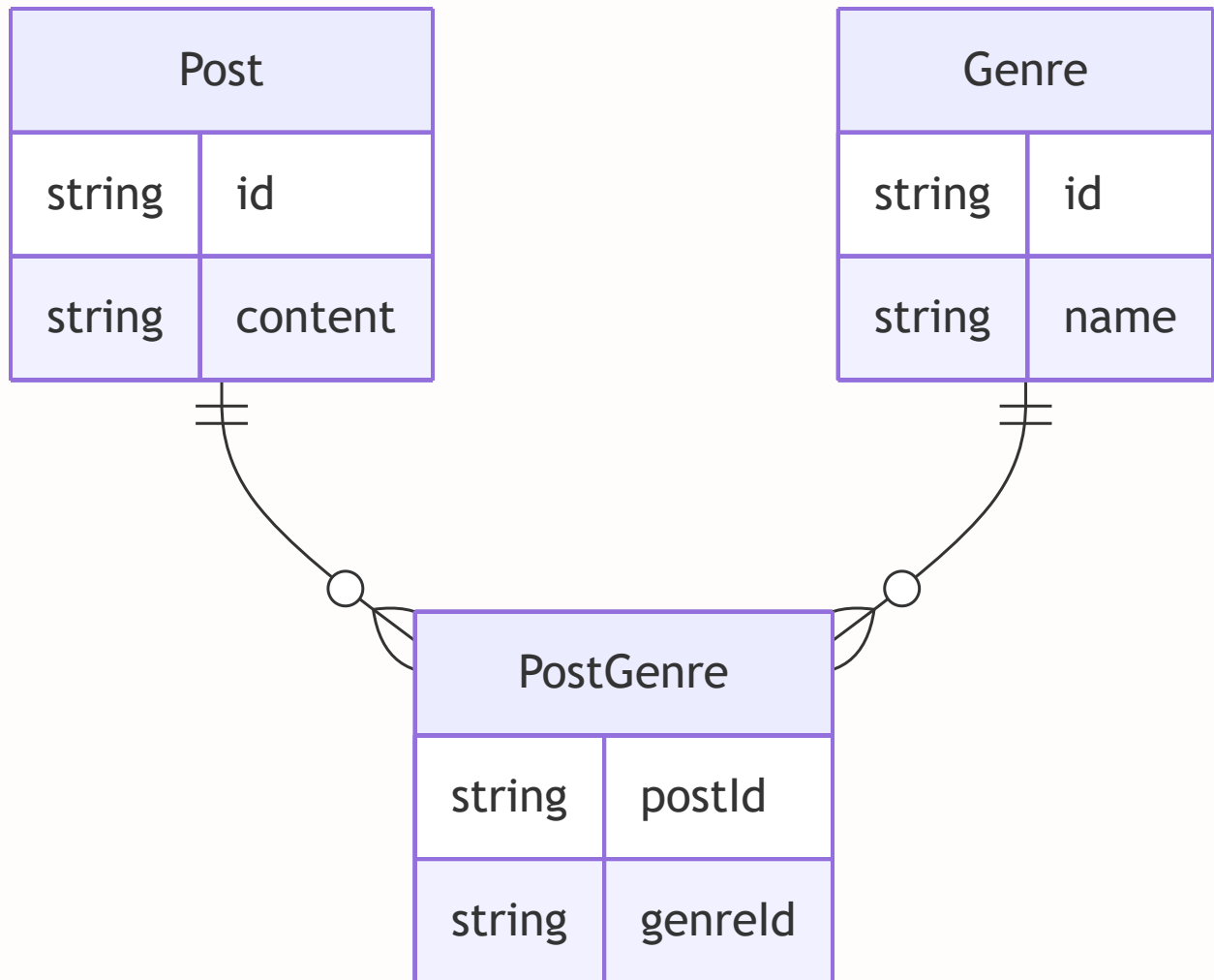


1対多の関係:

- 現実の例え: 1人の先生が複数の生徒を担当する
- BON-LOGの例: 1人のユーザーが複数の投稿を作成する
- 記法: `User (1) —> Post (多)`

多対多 (Many-to-Many)

投稿は複数のジャンルに属することができ、1つのジャンルには複数の投稿が属する関係です。多対多を直接表現することはできないため、**中間テーブル**を使って2つの「1対多」に分解します。

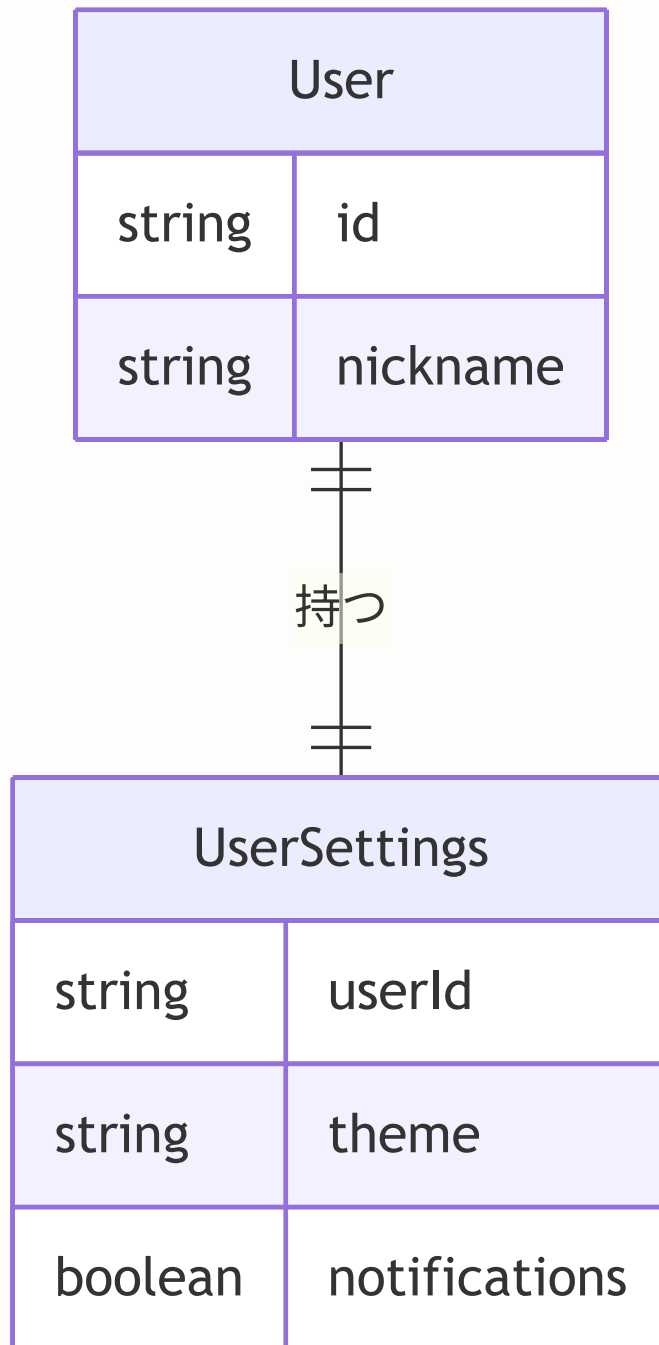


多対多の関係（中間テーブルで分解）：

- 現実の例え: 1人の学生は複数の授業を履修し、1つの授業には複数の学生がいる
- 中間テーブル（PostGenre）が2つの1対多関係に分解する
- 記法: `Post >—< PostGenre >—< Genre`

1対1（One-to-One）

ユーザーと設定情報のように、1対1で対応する関係。比較的レアです。

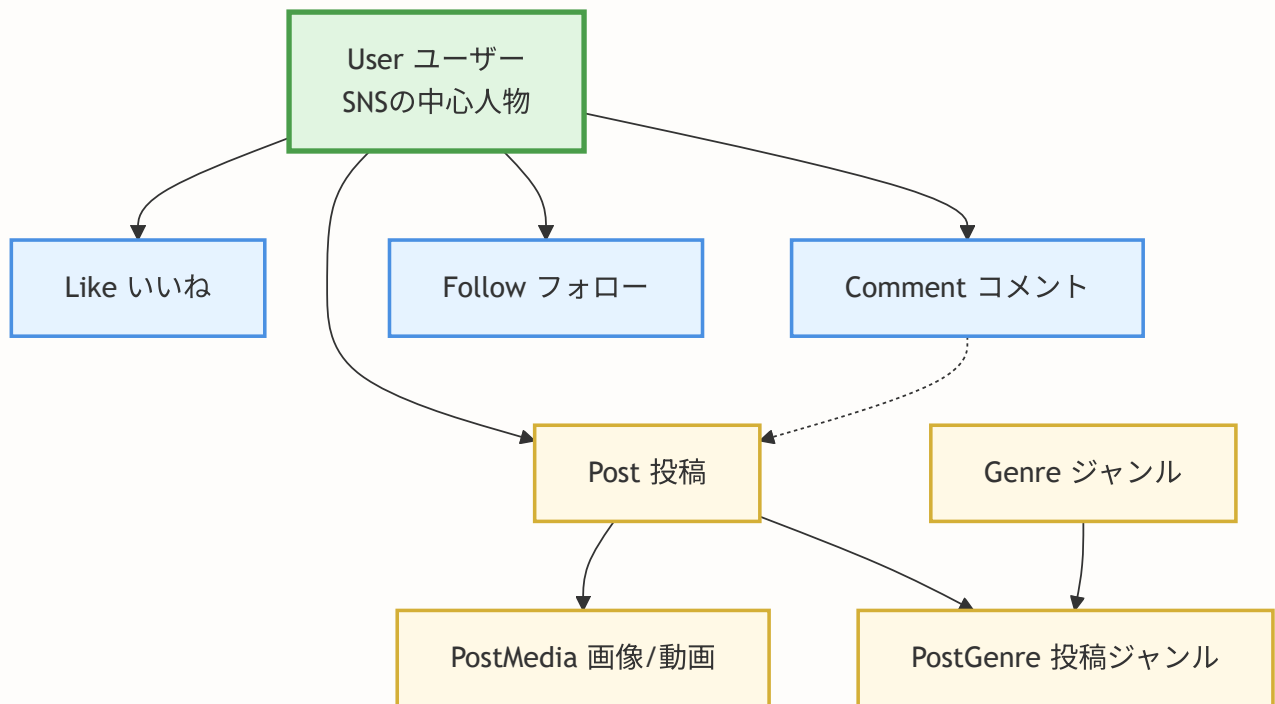
**1対1の関係:**

- 現実の例え: 1人に1つのパスポート
- 記法: `User (1) — (1) UserSettings`

7.1.3 BON-LOGの全テーブル関係図（初心者向け）

ここまでの用語を踏まえて、BON-LOGの主要テーブルがどのように繋がっているかを図で確認しましょう。矢印は「参照する（外部キーで紐付ける）」方向を示しています。

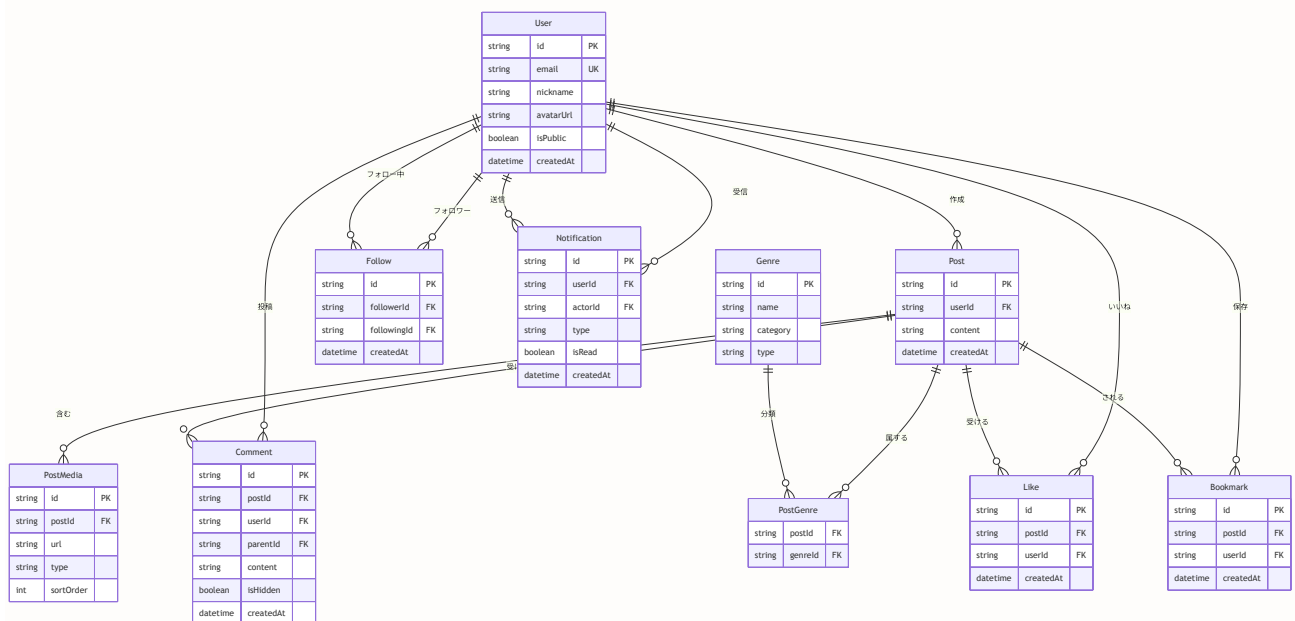
BON-LOG テーブル関係の全体像（初心者向け・簡略版）：



矢印の読み方：

- **User → Post** = 「1人のユーザーは複数の投稿を持つ」（1対多）
- **Post → PostMedia** = 「1つの投稿は複数の画像/動画を持つ」（1対多）
- **Post ↔ Genre** = 「投稿とジャンルは多対多」（PostGenreが橋渡し）
- **User → Follow** = 「フォロー関係はUser同士の紐付け」（自己結合）

####BON-LOG データベースER図（詳細版）



このER図は、BON-LOGの主要なテーブルとそれらの関係を示しています。各テーブルの主キー（PK）、外部キー（FK）、ユニーク制約（UK）が明記されています。

リレーションの種類をBON-LOGの実例で理解する：

- 1対多（最も多い）
 ユーザー 1人 — 投稿 複数 「Aliceが書いた投稿は何件もある」
 投稿 1件 — いいね 複数 「1つの投稿に複数のいいねが付く」
 投稿 1件 — コメント 複数 「1つの投稿に複数のコメントが付く」
- 多対多（中間テーブルが必要）
 投稿 — PostGenre — ジャンル
 「1つの投稿に複数のジャンル、1つのジャンルに複数の投稿」
- 自己結合（同じテーブル内の参照）
 User(Alice) — フォロワー —> User(Bob)
 「Userテーブルの中で、あるユーザーが別のユーザーを参照」

初心者向けポイント: テーブル間の関係を理解することが、データベース設計の最も重要なステップです。「このデータはどのテーブルに入れるか?」「テーブル同士はどう紐付けるか?」を考えることが設計の本質です。

▶ 理解度チェック: リレーショナルデータベースの基本

7.2 PostgreSQLの基本

このセクションで学ぶこと

- PostgreSQLとは何か、なぜBON-LOGで採用するのか
- 他のデータベースとの比較
- Docker Composeでの起動方法と確認手順

7.2.1 PostgreSQLとは

PostgreSQL（ポストグレスキューエル、通称「ポスグレ」）は、オープンソースの高機能なリレーショナルデータベース管理システム（RDBMS）です。1986年にカリフォルニア大学バークレー校で開発が始まり、30年以上の歴史を持つ、信頼性の高いデータベースです。

特徴:

- **高い信頼性とデータ整合性:** 銀行システムにも使われるほどの堅牢性
- **複雑なクエリに対応:** JOIN、サブクエリ、ウィンドウ関数などSQL機能が豊富
- **JSON型など豊富なデータ型:** JSONB型でNoSQL的な柔軟なデータ保存も可能
- **トランザクション対応:** 「全部成功するか、全部失敗するか」を保証
- **大規模データにも対応:** テラバイト級のデータも扱える

なぜPostgreSQLを選ぶのか？**データベース比較:**

項目	PostgreSQL	MySQL	SQLite
信頼性	◎ 非常に高い	○ 高い	△ 普通
機能の豊富さ	◎ 非常に多い	○ 多い	△ 最小限
JSON対応	◎ JSONB型	○ JSON型	× なし
学習コスト	○ 普通	○ 普通	◎ 低い
Supabase対応	◎ ネイティブ	× 非対応	× 非対応
Vercel連携	◎ 良好	○ 普通	△ 制限あり

BON-LOGでは、Supabase（本番DB）との親和性と豊富な機能からPostgreSQLを採用しています。

7.2.2 Docker Composeでの起動（復習）

第1章で設定済みですが、PostgreSQLの起動方法を再確認しましょう。

```
# PostgreSQLコンテナを起動（-d はバックグラウンドで実行する指定）
docker compose up -d postgres

# 起動確認（STATUSが"Up"になっていればOK）
docker compose ps

# ログ確認（エラーが出ていないか確認）
docker compose logs postgres

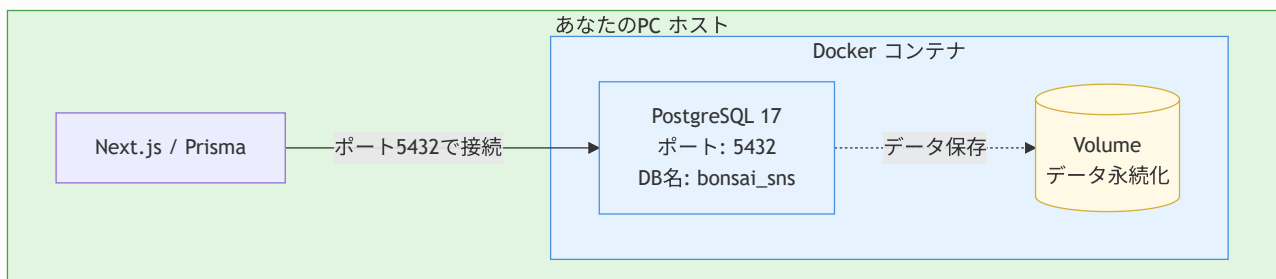
# 停止（データは保持される）
docker compose down

# 停止 + データも完全に削除（やり直したい場合のみ）
docker compose down -v
```

`docker-compose.yml` の該当部分:

```
services:
  postgres:
    image: postgres:17-alpine      # PostgreSQL 17 (軽量版Alpine Linux)
    environment:
      POSTGRES_USER: bonsai        # DBに接続するユーザー名
      POSTGRES_PASSWORD: bonsai_password # DBに接続するパスワード
      POSTGRES_DB: bonsai_sns      # 作成するデータベース名
    ports:
      - "5432:5432"               # ホスト側ポート:コンテナ側ポート (PostgreSQLの標準ポート)
    volumes:
      - postgres_data:/var/lib/postgresql/data # データの永続化 (コンテナを再起動してもデータが残る)
```

Docker Composeの仕組み:



よくあるトラブル	原因	解決法
<code>docker compose up</code> で「port already in use」	ポート5432が既に使われている	<code>docker compose down</code> してから再起動、または他のPostgreSQLプロセスを停止
コンテナが起動してもすぐ停止する	パスワードやDB名の設定ミス	<code>docker compose logs postgres</code> でエラーを確認
データが消えてしまった	<code>docker compose down -v</code> を実行した	<code>-v</code> オプションはデータも削除する。通常は <code>-v</code> なしで停止する
Docker自体が起動しない	Docker Desktopが停止している	Docker Desktopアプリを起動する

7.2B 技術選定の詳細（なぜこの技術を選んだのか）

このセクションで学ぶこと

- データベース、ORM、ホスティング、ID生成の各技術にどんな選択肢があるのか
- それぞれの特徴と比較
- BON-LOGが各技術を選んだ具体的な理由

実際のプロジェクトでは、「何を使うか」を決める段階が非常に重要です。このセクションでは、BON-LOGの技術選定の背景を、他の選択肢との比較を交えて解説します。初心者の方は「世の中にはこんな選択肢があるのか」と知っておくだけで十分です。

7.2B.1 データベースの選択肢

RDB（リレーショナルデータベース）vs NoSQL

データベースには大きく2つの種類があります。

RDB（リレーショナルデータベース）：

データを「表（テーブル）」で管理し、テーブル同士を「関連（リレーション）」で結ぶ。
構造が厳密に定義されており、データの整合性が高い。

NoSQL（Not Only SQL）：

テーブル構造にとらわれず、柔軟な形式でデータを保存する。
ドキュメント型、キーバリュー型、グラフ型など様々な種類がある。

RDB vs NoSQL 比較表:

項目	RDB (PostgreSQL, MySQL等)	NoSQL (MongoDB, Redis等)
データ構造	テーブル（行と列）	ドキュメント/KV/グラフ
スキーマ	厳密（事前に定義必須）	柔軟（後から変更容易）
整合性	非常に高い（ACID準拠）	結果整合性（柔軟）
リレーション	得意（JOINが強力）	苦手（埋め込みで対応）
トランザクション	標準サポート	限定的サポートが多い
適したケース	SNS、EC、銀行、業務系	IoT、チャット、CMS
不向きなケース	非構造化データの大量蓄積	複雑なリレーションが必要

BON-LOGの判断:

- SNSはユーザー・投稿・いいね・フォローなどテーブル間の関係が複雑
- 「AさんがBさんの投稿にいいねした」のような操作で整合性が重要
- **RDBが最適**

初心者向けポイント: SNSのような「ユーザーが投稿し、他のユーザーがそれにいいねやコメントをする」アプリケーションでは、データ間の関係（リレーション）が非常に重要です。これはRDBが最も得意とする分野です。

RDB同士の比較

RDBの中にもさまざまな選択肢があります。

主要RDBの比較:

項目	PostgreSQL	MySQL	SQLite	PlanetScale	CockroachDB
種別	OSS	OSS	組み込み型	サーバーレス	分散DB
信頼性	◎ 非常に高い	○ 高い	△ 普通	○ 高い	◎ 非常に高い
機能の豊富さ	◎ 非常に多い	○ 多い	△ 最小限	○ 多い	○ 多い
JSON対応	◎ JSONB型	○ JSON型	× なし	○ JSON型	◎ JSONB型
全文検索	◎ pg_trgm	○ FULLTEXT	× なし	○ FULLTEXT	△ 限定的
配列型	◎ ネイティブ	× なし	× なし	× なし	◎ あり
拡張機能	◎ 豊富	△ 少ない	× なし	× 不可	△ 少ない
Supabase対応	◎ ネイティブ	× 非対応	× 非対応	× 非対応	× 非対応
学習コスト	○ 普通	○ 普通	◎ 低い	○ 普通	△ やや高い
無料プラン	◎ Supabase等	○ PlanetScale	◎ 完全無料	○ あり	○ あり
適した規模	小～超大規模	小～大規模	小規模	中～大規模	大～超大規模

なぜPostgreSQLを選んだのか

BON-LOGでPostgreSQLを採用した具体的な理由は以下の通りです。

#	理由	詳細
1	全文検索 (pg_trgm拡張)	BON-LOGでは投稿やユーザーの検索機能が必要。pg_trgm拡張を使えば日本語のあいまい検索が可能。別途ElasticsearchやAlgoliaを導入せずに済み、運用コストとシステム複雑度を大幅に削減
2	JSONB型とArrays (配列型)	SystemSettingモデルではJSON型で柔軟な設定値を保存。RDBの厳密さとNoSQLの柔軟さを両立できる。配列型はタグのリストなど、簡単なリストの保存に便利（中間テーブルを作らずに済むケースもある）
3	Supabaseとの完全互換性	本番環境のホスティングにSupabaseを採用。SupabaseはPostgreSQL専用のプラットフォーム。開発環境と本番環境で同じDBエンジンを使えるため、「開発では動くのに本番で動かない」を防止
4	豊富な拡張機能エコシステム	pg_trgm（全文検索）、PostGIS（地理情報）、pgcrypto（暗号化）など、多数の公式拡張が利用可能。盆栽園マップ機能で将来PostGISを使う可能性も
5	コミュニティと実績	30年以上の歴史、世界中の大企業で採用実績あり。問題に直面しても情報が豊富で解決しやすい。Next.js + Prisma + PostgreSQLは王道の組合せ

7.2B.2 ORMの選択肢

Node.js/TypeScriptのエコシステムには複数のORMやクエリビルダーがあります。

主要ORM/クエリビルダーの比較:

項目	Prisma	Drizzle ORM	TypeORM	Sequelize	Knex.js (ビルダー)	生SQL
型安全性	◎ 自動	◎ 自動	○ 手動	△ 部分的	△ 部分的	× なし
スキーマ定義	独自DSL	TS関数	デコレータ	JS/TSクラス	なし	SQL
マイグレーション	◎ 自動	○ 手動	○ 自動	○ 自動	○ 手動	× 手動
学習コスト	◎ 低い	○ やや低い	△ やや高い	○ 普通	△ やや高い	× 高い
パフォーマンス	○ 良好	◎ 高速	○ 良好	△ 普通	◎ 高速	◎ 最速
GUI管理ツール	◎ Studio	× なし	× なし	× なし	× なし	× なし
エコシステム	◎ 充実	○ 成長中	◎ 充実	○ 成熟	○ 成熟	-
Next.js連携	◎ 公式	○ 良好	△ 課題あり	△ 課題あり	○ 良好	○ 可能
ドキュメント	◎ 充実	○ 充実	○ 充実	○ 充実	○ 充実	-

なぜPrismaを選んだのか

#	理由	詳細
1	直感的なスキーマ定義言語（Prisma Schema Language）	TypeORMのデコレータ方式（ <code>@Entity()</code> <code>@Column()</code> <code>@PrimaryGeneratedColumn()</code> ）はデコレータの種類を覚える必要がある。Prismaのスキーマ方式（ <code>model User { id String @id @default(cuid()) }</code> ）は読めば意味がわかり、学習コストが低い
2	TypeScriptの型が自動生成される	スキーマを書いて <code>npx prisma generate</code> するだけで、TypeScriptの型定義が自動的に作られる。エディタの補完が完璧に効き、存在しないカラムを書くとコンパイル時にエラーになり、実行前にバグを発見できる
3	マイグレーション管理が組み込み	スキーマの変更を自動検出してマイグレーションを生成。開発中は <code>db push</code> で素早く反映、本番環境では <code>migrate dev</code> で履歴管理。別途ツール（knex migrate等）を導入する必要がない
4	Prisma Studio（GUI管理ツール）	<code>npx prisma studio</code> でブラウザベースの管理画面が起動。Excelのような画面でデータの閲覧・編集・削除が可能。初心者でもデータの中身を直感的に確認でき、デバッグやテストデータの投入が楽

Drizzle ORMについて: 2024年以降に急速に人気が高まっているORMです。Prismaよりも「SQLに近い」記法で、パフォーマンスが高いのが特徴です。しかしGUI管理ツールがなく、ドキュメントやエコシステムがPrismaほど成熟していないため、初心者がチュートリアルで学ぶにはPrismaの方が適しています。

7.2B.3 DBホスティングの選択肢

データベースをどこで動かすか（ホスティング先）も重要な選択です。

主要DBホスティングの比較:

項目	Supabase	Neon	Railway	AWS RDS	ローカル
DB種別	PostgreSQL	PostgreSQL	複数対応	複数対応	Docker等
無料枠	500MB	512MB	\$5/月～	750時間	完全無料
管理UI	◎ 充実	○ あり	○ あり	○ あり	× なし
接続プーリング	◎ 内蔵	◎ 内蔵	× 別途	× 別途	× 不要
バックアップ	◎ 自動	◎ 自動	○ 手動	◎ 自動	× 手動
スケーリング	○ 自動	◎ 自動	○ 手動	◎ 柔軟	× 不可
学習コスト	◎ 低い	◎ 低い	○ 普通	△ やや高い	○ 普通
Vercel連携	◎ 良好	◎ 良好	○ 可能	○ 可能	× 不可
認証・API	◎ 付属	× なし	× なし	× なし	× なし

なぜSupabaseを選んだのか

Supabase採用の理由：

1. PostgreSQLへの直接接続が可能
 - Prismaから標準的なPostgreSQL接続URLでアクセスできる
 - 独自のSDKに依存しない（PrismaのORMをそのまま使える）
2. 無料枠が個人開発に十分
 - 500MBのストレージ、月5万リクエスト
 - BON-LOGの開発・テスト段階には十分な容量
3. 管理UIが充実
 - テーブルの閲覧・編集がブラウザで可能
 - SQLエディタで直接クエリを実行できる
 - Prisma Studioの代替としても使える
4. 接続プーリングが内蔵
 - Vercelのサーバーレス環境で重要
 - 追加設定なしでコネクション管理が最適化される
5. 将来の拡張性
 - 認証（Supabase Auth）、ストレージ、リアルタイム機能も提供
 - 必要に応じて段階的に採用できる

7.2B.4 ID生成の選択肢

データベースの主キー（各レコードを一意に識別するID）の生成方法にも選択肢があります。

主要なID生成方式の比較：

項目	cuid	UUID (v4)	nanoid	連番(AI)	ULID
例	clx1abc2d...	550e8400-...	V1StGXR8_Z...	1, 2, 3...	01ARZ3NDEK...
長さ	25文字	36文字	21文字	可変	26文字
衝突耐性	◎ 非常に高い	◎ 非常に高い	◎ 非常に高い	◎ なし(連番)	◎ 非常に高い
ソート可能	◎ 時系列順	× 不可	× 不可	◎ 自然順	◎ 時系列順
URL安全	◎ そのまま可	○ ハイフン有	◎ そのまま可	◎ そのまま可	◎ そのまま可
推測困難性	◎ 高い	◎ 高い	◎ 高い	× 容易に推測	△ やや推測可
データベース性能	○ 良好	△ やや劣る	○ 良好	◎ 最速	○ 良好
分散環境対応	◎ 対応	◎ 対応	◎ 対応	× 非対応	◎ 対応
Prisma対応	◎ 組み込み	◎ 組み込み	△ 手動実装	◎ 組み込み	△ 手動実装

なぜcuidを選んだのか

#	理由	詳細
1	衝突耐性が非常に高い	タイムスタンプ + カウンタ + フィンガープリント + ランダム値を組み合わせでIDを生成。複数のサーバーで同時にIDを生成しても衝突しない
2	ソート可能（時系列順）	タイムスタンプが含まれるため、ID順 ≒ 作成順。UUIDはランダムなのでID順にソートする意味がない。インデックスの効率が良い（B-treeに有利）
3	URL安全（そのまま使える）	cuid: <code>/posts/clx1abc2d0000xyz</code> / UUID: <code>/posts/550e8400-e29b-41d4-a716-446655440000</code> 。cuidの方が短く、ハイフンも含まないのでURLに最適
4	Prismaで標準サポート	<code>@default(cuid())</code> と書くだけで自動生成される。nanoidやULIDは手動で実装が必要。設定が簡単で初心者にも扱いやすい

連番(auto-increment)を使わない理由:

問題点	説明
ユーザー数の推測	<code>/users/1</code> , <code>/users/2</code> からユーザー数が推測可能
次のIDの予測	<code>/users/3</code> から「次は <code>/users/4</code> だろう」と予測可能
分散環境の困難	分散環境で連番の一意性を保つのが困難
結論	セキュリティとスケーラビリティの両面で不適

▶ 理解度チェック: 技術選定

7.3 Prismaとは

このセクションで学ぶこと

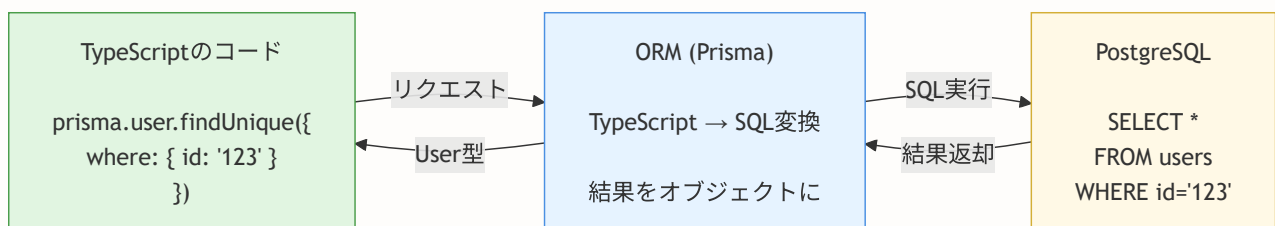
- ORM（Object-Relational Mapping）とは何か
- Prismaを使うメリットと使わない場合の比較
- Prismaの3つの主要コンポーネント（Client、Migrate、Studio）

7.3.1 ORMの概念

ORM（Object-Relational Mapping）は、データベースのテーブルをプログラムのオブジェクトとして扱えるようにする技術です。

ORMの位置づけを理解するたとえ: ORMは「通訳」のようなものです。あなた（TypeScriptのコード）と外国人（PostgreSQL）が会話するとき、直接外国語（SQL）を書くこともできますが、通訳（ORM）を通すことで、母国語（TypeScript）のまま会話できます。

ORMの役割:



ORMなし（生SQL）：

```
// SQLを直接文字列として書く（タイプミスに気づきにくい）
const result = await db.query(
  'SELECT * FROM users WHERE id = $1', // SQL文字列（型チェックなし）
  [userId]                             // パラメータを配列で渡す
)
const user = result.rows[0] // 結果の型が不明（any型）
```

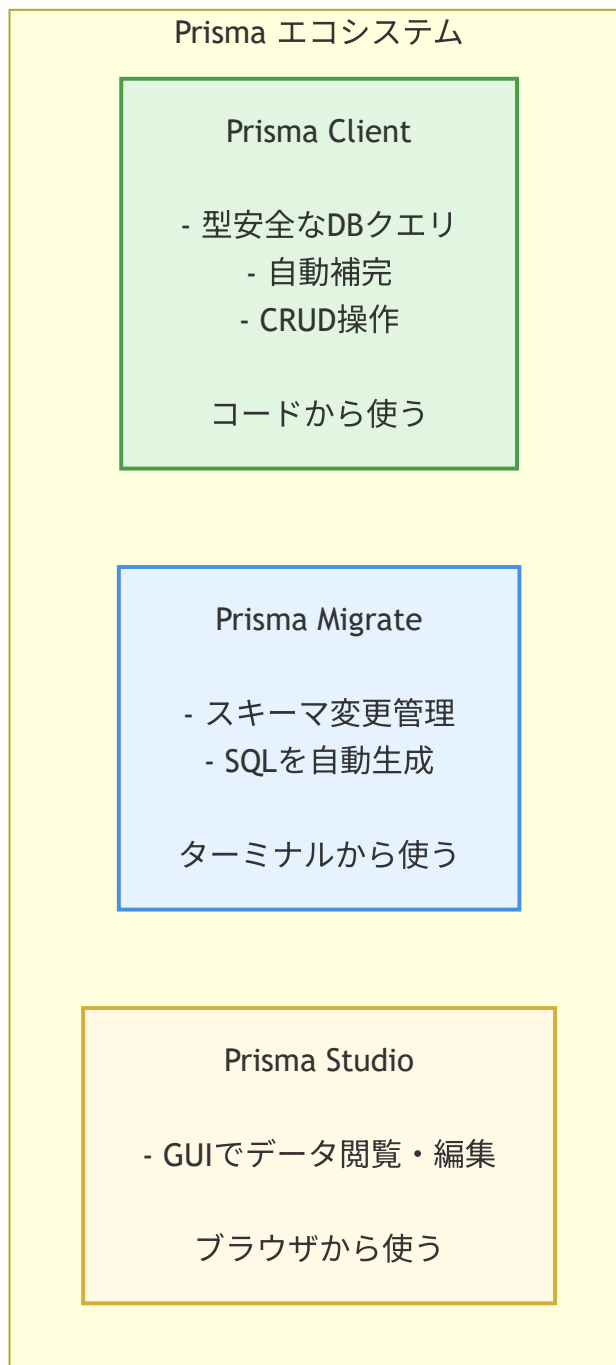
ORMあり（Prisma）：

```
// TypeScriptのメソッドチェーンで書く（補完が効く、型安全）
const user = await prisma.user.findUnique({
  where: { id: userId } // 型チェック付き（存在しないカラムを指定するとエラー）
})
// user の型は User | null（自動的に推論される）
```

7.3.2 Prismaの特徴

Prismaは、Node.js/TypeScript向けの次世代ORMです。従来のORMと比較して、特にTypeScriptとの親和性が高い点が特徴です。

Prismaの3つの主要コンポーネント:



1. **型安全**: TypeScriptと完全統合、コンパイル時にエラー検出。存在しないカラム名を指定すると、コードを実行する前にエディタが赤線で教えてくれます。
2. **直感的なAPI**: `findMany`, `create`, `update` など英語で読めるメソッド名。SQLを知らなくてもデータ操作ができます。
3. **マイグレーション管理**: スキーマ変更を追跡可能。「いつ、どのような変更をしたか」の履歴が残ります。
4. **Prisma Studio**: GUIでデータベースを操作可能。Excelのような画面でデータを確認・編集できます。
5. **パフォーマンス**: 最適化されたクエリを自動生成。不要なカラムの取得を避けるなど、効率的なSQLに変換されます。

▶ 理解度チェック: ORMとPrisma

7.3B SQLとPrismaクエリの対比表

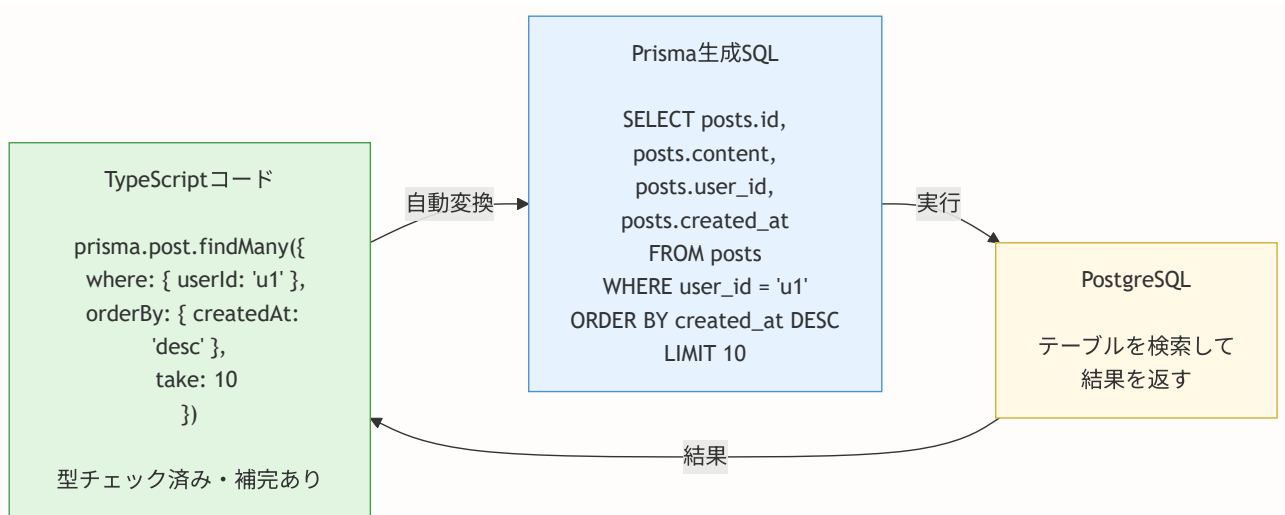
ORMの理解を深めるために、生のSQL文とPrismaのクエリを並べて比較してみましょう。「Prismaが裏側で何をしているか」がわかると、より深い理解が得られます。

初心者の方へ: SQLを知らなくてもPrismaは使えますが、SQLの基本を知っておくとデバッグや最適化の際に非常に役立ちます。ここでは「こんな対応関係なんだ」とざっくり理解しておけば十分です。

SQL vs Prisma 対比表:

操作	SQL (データベースの言語)	Prisma (TypeScriptで書くORM)
全件取得	<code>SELECT * FROM users;</code>	<code>prisma.user.findMany()</code>
条件取得	<code>SELECT * FROM users WHERE id = 'u1';</code>	<code>prisma.user.findUnique({ where: { id: 'u1' } })</code>
部分取得	<code>SELECT id, nickname FROM users;</code>	<code>prisma.user.findMany({ select: { id: true, nickname: true } })</code>
作成	<code>INSERT INTO users (email, nickname) VALUES ('a@b.com', 'Alice');</code>	<code>prisma.user.create({ data: { email: 'a@b.com', nickname: 'Alice' } })</code>
更新	<code>UPDATE users SET nickname = 'Bob' WHERE id = 'u1';</code>	<code>prisma.user.update({ where: { id: 'u1' }, data: { nickname: 'Bob' } })</code>
削除	<code>DELETE FROM users WHERE id = 'u1';</code>	<code>prisma.user.delete({ where: { id: 'u1' } })</code>
件数取得	<code>SELECT COUNT(*) FROM users WHERE is_public = true;</code>	<code>prisma.user.count({ where: { isPublic: true } })</code>
ソート	<code>SELECT * FROM posts ORDER BY created_at DESC LIMIT 20;</code>	<code>prisma.post.findMany({ orderBy: { createdAt: 'desc' }, take: 20 })</code>
結合 (JOIN)	<code>SELECT p.*, u.nickname FROM posts p LEFT JOIN users u ON p.user_id = u.id;</code>	<code>prisma.post.findMany({ include: { user: { select: { nickname: true } } } })</code>
あいまい検索	<code>SELECT * FROM users WHERE nickname LIKE '%Alice%';</code>	<code>prisma.user.findMany({ where: { nickname: { contains: 'Alice' } } })</code>

PrismaがSQLに変換される流れ:



Prismaの裏側: Prismaは内部でSQLクエリを生成してデータベースに送信しています。開発環境でログ設定を `['query']` にすると (lib/db.tsで設定済み)、ターミナルに実際のSQLが表示されるので、どんなSQLが実行されているか確認できます。

7.4 Prismaのセットアップ

このセクションで学ぶこと

- Prismaのインストール手順
- データベース接続URLの設定方法
- Prismaの開発ワークフロー (スキーマ定義 → 生成 → 反映の流れ)

7.4.1 インストール

```
# Prisma CLIと Prismaクライアントをインストール
# prisma: CLIツール（スキーマ管理、マイグレーション用）
# @prisma/client: アプリケーションコードから使うライブラリ
# @prisma/adpater-pg: PrismaPgアダプター（Prisma 6でPostgreSQLに必要）
# pg: Node.js用のPostgreSQLドライバー（接続プール機能を提供）
npm install prisma @prisma/client @prisma/adpater-pg pg

# TypeScript型定義（pgの型情報）
npm install -D @types/pg

# Prismaの初期化（既にプロジェクトにある場合は不要）
# これを実行すると prisma/ フォルダと .env ファイルが作成される
npx prisma init
```

これにより以下が作成されます:

- `prisma/schema.prisma` - スキーマ定義ファイル（テーブル構造を定義する最重要ファイル）
- `.env` - 環境変数ファイル（データベース接続情報など）

初期化後のフォルダ構成:

```
bonsai-sns-project/
├─ prisma/
│   └─ schema.prisma  ← テーブル定義はここに書く
├─ .env               ← DB接続URLはここに設定
├─ .env.local         ← ローカル環境用（.envより優先）
└─ ...
```

7.4.2 環境変数の設定

`.env.local` ファイルに以下を追加します。この接続URLは、Prismaがどのデータベースに接続するかを指定するものです。

```
# PostgreSQL接続URL
# 構造: postgresql://ユーザー名:パスワード@ホスト:ポート/データベース名?schema=スキーマ名
DATABASE_URL="postgresql://bonsai:bonsai_password@localhost:5432/bonsai_sns?schema=public"

# Supabase使用時はDIRECT_URLも設定（接続プーリングを回避するため）
DIRECT_URL="postgresql:// ... "
```

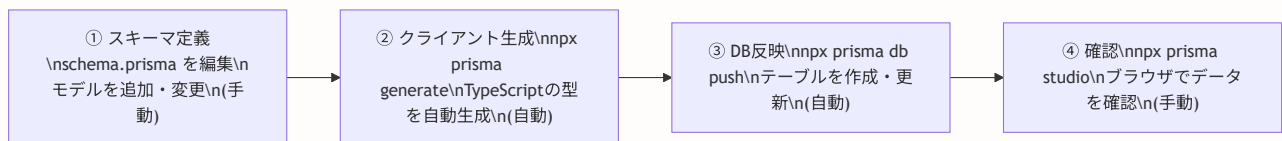
接続URLの各部分の意味: `postgresql://bonsai:bonsai_password@localhost:5432/bonsai_sns?schema=public`

部分	値	説明
プロトコル	<code>postgresql://</code>	PostgreSQLを示す
ユーザー名	<code>bonsai</code>	DBのログインユーザー
パスワード	<code>bonsai_password</code>	DBのログインパスワード
ホスト名	<code>localhost</code>	localhostは自分のPC
ポート番号	<code>5432</code>	PostgreSQLの標準ポート
データベース名	<code>bonsai_sns</code>	接続先のデータベース
スキーマ	<code>public</code>	使用するスキーマ

よくあるトラブル	原因	解決法
<code>Can't reach database server</code>	PostgreSQLが起動していない	<code>docker compose up -d postgres</code> を実行
<code>Authentication failed</code>	ユーザー名またはパスワードが違う	docker-compose.ymlの設定と一致しているか確認
<code>Database does not exist</code>	データベースが作成されていない	docker-compose.ymlの <code>POSTGRES_DB</code> を確認
<code>.env.local</code> が読み込まれない	ファイル名のタイプミス	<code>.env.local</code> (ドットから始まる) を正確に確認

7.4.3 Prismaのワークフロー

Prismaを使った開発では、以下の4ステップを繰り返します。



```

# ===== Step 1: schema.prismaを編集してモデルを定義 =====
# テキストエディタで手動編集します

# ===== Step 2: Prismaクライアントを生成（型定義を作成） =====
#   → node_modules/.prisma/client/ に型定義ファイルが生成される
npx prisma generate
  
```

Step 2 の実行結果:


```
$ npx prisma generate
Prisma schema loaded from prisma/schema.prisma

✓ Generated Prisma Client (v6.2.1) to ./node_modules/@prisma/client in 245ms

You can now start using Prisma Client in your code. Reference:
https://pris.ly/d/client

import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()
```

この時点ではデータベースにはまだ何も変更されていません。TypeScriptの型定義ファイルが生成されただけです。

```
# ===== Step 3: スキーマをデータベースに反映 =====
# 開発環境: 即座に反映 (マイグレーションファイルなし、素早く試行錯誤できる)
npx prisma db push
```

Step 3 の実行結果:

```
$ npx prisma db push
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "bonsai_sns", schema "public"
  at "localhost:5432"

🚀 Your database is now in sync with your Prisma schema. Done in 1.23s

✓ Generated Prisma Client (v6.2.1) to ./node_modules/@prisma/client in 198ms
```

`db push` が成功すると、schema.prismaに定義したモデルに対応するテーブルがデータベースに作成されます。
`db push` は内部的に `generate` も実行するので、Step 2 と Step 3 をまとめて `npx prisma db push` だけで済ませることもできます。

```
# 本番環境: マイグレーションファイルを作成して管理 (変更履歴が残る)
npx prisma migrate dev --name initial_schema

# ===== Step 4: データベースをGUIで確認 =====
# ブラウザで http://localhost:5555 が開く
npx prisma studio
```

Step 4 の実行結果:

```
$ npx prisma studio
Prisma Studio is up on http://localhost:5555
```

実行結果の確認方法 `npx prisma studio` を実行すると、ブラウザでデータベースの中身を確認できます。

<http://localhost:5555> でテーブル一覧が表示され、作成したテーブルの構造やデータを目で確認できます。まだデータを投入していない段階では、テーブルは存在するがレコードは0件の状態です。

初心者向けポイント: 開発中は `npx prisma db push` を使ってスキーマを素早くDBに反映し、本番環境やチーム開発では `npx prisma migrate dev` を使ってマイグレーションファイル（変更履歴）を作成するのがベストプラクティスです。

7.5 schema.prismaの書き方

このセクションで学ぶこと

- schema.prisma ファイルの3つの構成要素（generator, datasource, model）
- Prismaで使えるデータ型と対応するPostgreSQLの型
- 属性（@id, @unique, @default, @map など）の使い方
- オプショナル型（NULL許容）の意味と使い方

7.5.1 基本構造

`schema.prisma` ファイルは、データベースの設計図です。3つの主要パートから構成されます。

スキーマ（Schema）とは？: 「スキーマ」という言葉は「設計図」「構造定義」を意味します。建築でいえば設計図面にあたり、「どの部屋がどこにあるか」「部屋の大きさはどれくらいか」「部屋同士はどう繋がっているか」を定義します。データベースのスキーマは「どのテーブルがあるか」「各テーブルにはどんなカラムがあるか」「テーブル同士はどう関連しているか」を定義するものです。Prismaでは `schema.prisma` ファイル1つでこの設計図を記述します。

「スキーマ」の3つの意味 本章では「スキーマ」が3つの異なる意味で使われます：

- **Prismaスキーマ**: `prisma/schema.prisma` ファイル（テーブル定義を書く設計図）
- **データベーススキーマ**: テーブル構造全体の設計（概念的な意味）
- **PostgreSQLスキーマ**: データベース内の名前空間（通常 `public` ）

文脈で判断できますが、混乱した場合はこの区別を思い出してください。

```
// prisma/schema.prisma

// =====
// パート1: generator (ジェネレーター)
// Prismaクライアントの生成方法を設定
// npx prisma generate で使われる
// =====

generator client {
  provider = "prisma-client-js" // JavaScript/TypeScript用クライアントを生成
}

// =====
// パート2: datasource (データソース)
// どのデータベースに接続するかを設定
// =====

datasource db {
  provider = "postgresql" // 使用するDB種類 (postgresql, mysql, sqlite等)
  url      = env("DATABASE_URL") // 接続URLを環境変数から読み込む
  directUrl = env("DIRECT_URL") // Supabase使用時の直接接続URL
}

// =====
// パート3: model (モデル)
// テーブル定義 - ここがメインの記述部分
// 1つのmodelが1つのテーブルに対応する
// =====

model User {
  // カラム名 型      属性 (制約やデフォルト値)
  id      String  @id @default(cuid()) // 主キー、自動でユニークID生成
  email   String  @unique                // メールアドレス (重複不可)
  nickname String                // ニックネーム (必須)
  createdAt DateTime @default(now())    // 作成日時 (自動で現在時刻)
  updatedAt DateTime @updatedAt        // 更新日時 (更新時に自動変更)

  // テーブル名を明示的に指定 (省略するとモデル名がそのままテーブル名になる)
  @@map("users") // DBのテーブル名は "users" (小文字・複数形)
}
```

schema.prismaの3パート構成:

パート	記述	役割
generator	<code>generator client { ... }</code>	「何を生成するか」の設定
datasource	<code>datasource db { ... }</code>	「どこに接続するか」の設定
model	<code>model User { ... }</code> / <code>model Post { ... }</code> / <code>model Comment { ... }</code> / ...	テーブル定義（必要な数だけ書く）

7.5.2 データ型

Prismaのデータ型は、PostgreSQLのデータ型に自動的に変換されます。よく使うものから順に紹介します。

Prisma型	PostgreSQL型	用途	BON-LOGでの使用例
<code>String</code>	TEXT/VARCHAR	テキストデータ	メールアドレス、ニックネーム、投稿内容
<code>Int</code>	INTEGER	整数	表示順序 (sortOrder)、カウント
<code>Boolean</code>	BOOLEAN	true/false	公開設定 (isPublic)、プレミアム会員フラグ
<code>DateTime</code>	TIMESTAMP	日時	作成日時、更新日時
<code>Float</code>	DOUBLE PRECISION	小数点数	星評価 (レビュー)、経度・緯度
<code>Json</code>	JSONB	JSON形式のデータ	柔軟な設定データ
<code>BigInt</code>	BIGINT	大きな整数	非常に大きな数値（通常はIntで十分）
<code>Bytes</code>	BYTEA	バイナリデータ	ファイルの直接保存（通常は使わない）

初心者向けポイント: 最初は `String`、`Int`、`Boolean`、`DateTime` の4つを覚えておけば、ほとんどのケースに対応できます。

7.5.3 属性 (Attributes)

属性は `@` マーク（フィールドレベル）や `@@` マーク（モデルレベル）で指定する制約やデフォルト値です。

```

model User {
  // — @id: 主キー（このカラムでレコードを一意に識別する） —
  // cuid()は衝突しにくいユニークIDを自動生成する関数
  // 例: "clx1abc2d0000xyz ..." のような文字列が自動的に設定される
  id          String    @id @default(cuid())

  // — @unique: ユニーク制約（同じ値を持つレコードは作れない） —
  // 同じメールアドレスで2回登録しようするとエラーになる
  email       String    @unique

  // — @default: デフォルト値（値を指定しなかった場合に使われる値） —
  // ユーザー作成時にisPublicを指定しなければ、自動的にtrueになる
  isPublic    Boolean   @default(true)
  // now()は現在の日時を自動的に設定する関数
  createdAt   DateTime  @default(now())

  // — @updatedAt: レコード更新時に自動で現在時刻に更新される —
  // updateメソッドが呼ばれるたびに自動的に時刻が変わる
  updatedAt   DateTime  @updatedAt

  // — @map: DBのカラム名を指定（TypeScriptとDBの命名規則の違いを吸収） —
  // TypeScriptではキャメルケース(avatarUrl)、DBではスネークケース(avatar_url)
  avatarUrl   String?   @map("avatar_url")

  // — @@map: テーブル名を指定（モデルレベルの属性は@@を使う） —
  // Prismaのモデル名は "User"（単数形、先頭大文字）
  // DBのテーブル名は "users"（複数形、小文字）
  @@map("users")
}

```

属性	レベル	用途	例
@id	フィールド	主キーの指定	id String @id
@unique	フィールド	ユニーク制約	email String @unique
@default()	フィールド	デフォルト値	@default(true), @default(now()), @default(cuid())
@updatedAt	フィールド	更新時自動変更	updatedAt DateTime @updatedAt
@map()	フィールド	DBカラム名指定	@map("avatar_url")
@relation()	フィールド	リレーション定義	@relation(fields: [...], references: [...])
@@map()	モデル	DBテーブル名指定	@@map("users")
@@unique()	モデル	複合ユニーク制約	@@unique([userId, postId])
@@id()	モデル	複合主キー	@@id([followerId, followingId])

7.5.4 オプショナル型

? を付けると **NULL**（値が未設定であること）を許可します。? がないフィールドは必須項目となり、値を指定しないとエラーになります。

```
model User {
  id      String  @id @default(cuid())
  email   String  @unique // 必須: NULLは許可されない（登録時に必ず必要）
  nickname String // 必須: NULLは許可されない（登録時に必ず必要）
  password String? // 任意: NULLを許可（GoogleログインなどOAuth時はパスワードなし）
  bio     String? // 任意: 自己紹介は入力しなくてもOK
  location String? // 任意: 地域は入力しなくてもOK
}
```

必須フィールドとオプショナルフィールドの違い（新規ユーザー登録フォーム）：

フィールド	入力例	必須/任意	? の有無
メールアドレス	<code>alice@example.com</code>	必須	? なし
ニックネーム	<code>Alice</code>	必須	? なし
パスワード	<code>****</code>	任意	? あり
自己紹介	(空欄可)	任意	? あり
地域	(空欄可)	任意	? あり

※ 必須項目を空にして登録するとDBエラーになる

TypeScriptの型との対応: Prismaのオプショナル型は、TypeScriptでは `string | null` 型として扱われます。? がないフィールドは `string` 型 (NULLにならないことが保証される) になります。

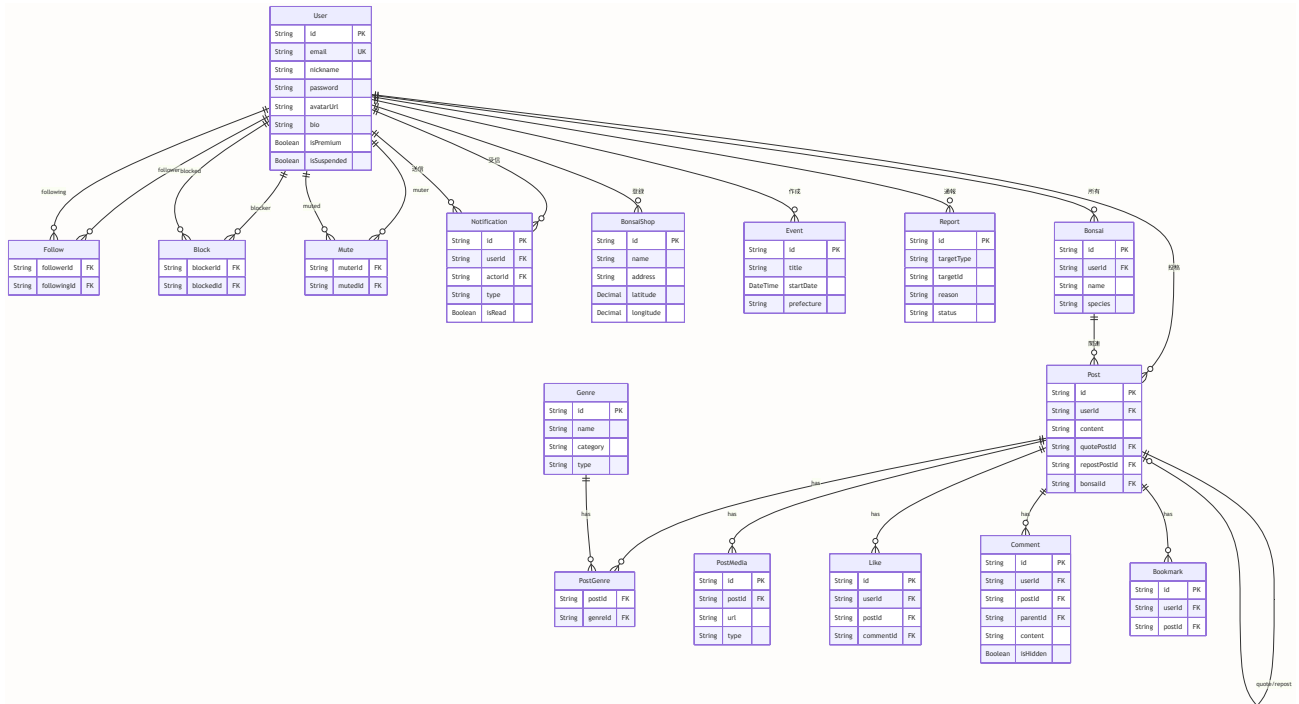
▶ 理解度チェック: `schema.prisma`の書き方

7.6 BON-LOGのスキーマ設計

このセクションで学ぶこと

- BON-LOG全体のER図 (Entity-Relationship Diagram)
- 各モデルの設計意図と実装方法
- リレーション定義の書き方 (1対多、多対多、自己参照)
- `onDelete: Cascade` の意味と使い方

まず、BON-LOGのデータベース全体像をER図で確認しましょう。



7.6.1 Userモデル

Userモデルは、BON-LOGに登録されたユーザーの情報を保存するテーブルです。SNSの中心となるモデルであり、他の多くのテーブルがこのUserを参照します。


```
// prisma/schema.prisma より抜粋

model User {
  // — 基本情報 —
  id          String    @id @default(cuid())      // キー: ユーザーの一意ID
  email       String    @unique @db.VarChar(100) // メールアドレス (ログインに使用、重複不可)
  emailVerified DateTime? @map("email_verified") // メール確認完了日時 (未確認ならNULL)
  password    String?   // パスワードのハッシュ値 (OAuth時はNULL)
  nickname    String    @db.VarChar(50)          // 表示名 (必須、最大50文字)

  // — プロフィール情報 —
  avatarUrl   String?   @map("avatar_url")        // プロフィール画像のURL
  headerUrl   String?   @map("header_url")        // ヘッダー画像のURL
  bio         String?   @db.VarChar(200)          // 自己紹介文 (最大200文字)
  location    String?   @db.VarChar(100)          // 居住地域 (任意)
  bonsaiStartYear Int?   @map("bonsai_start_year") // 盆栽を始めた年 (任意)
  bonsaiStartMonth Int? @map("bonsai_start_month") // 盆栽を始めた月 (任意)
  birthDate   DateTime? @map("birth_date") @db.Date // 生年月日 (任意)

  // — 設定 —
  isPublic    Boolean   @default(true) @map("is_public") // アカウント公開設定
  isSuspended Boolean   @default(false) @map("is_suspended") // アカウント停止フラグ
  suspendedAt DateTime? @map("suspended_at")           // 停止日時

  // — 有料会員 —
  isPremium    Boolean   @default(false) @map("is_premium")
  premiumExpiresAt DateTime? @map("premium_expires_at")
  stripeCustomerId String? @unique @map("stripe_customer_id")
  stripeSubscriptionId String? @unique @map("stripe_subscription_id")

  // — 2段階認証 (2FA) —
  twoFactorEnabled Boolean   @default(false) @map("two_factor_enabled")
  twoFactorSecret   String?   @map("two_factor_secret")
  twoFactorBackupCodes String[] @map("two_factor_backup_codes")

  // — 通知設定 —
  notificationPreferences Json? @default("{}") @map("notification_preferences")

  // — 日時 —
  createdAt    DateTime @default(now()) @map("created_at") // 登録日時
  updatedAt    DateTime @updatedAt @map("updated_at")      // 最終更新日時

  // — リレーション (このユーザーに関連する他テーブルのデータ) —
  accounts      Account[] // OAuthアカウント (NextAuth.js)
  sessions      Session[] // セッション (NextAuth.js)
  posts         Post[]    // 投稿一覧
  comments      Comment[] // コメント一覧
  likes         Like[]    // いいね一覧
  bookmarks     Bookmark[] // ブックマーク一覧
  followers     Follow[]  @relation("FollowingToUser") // フォロワー一覧
  following     Follow[]  @relation("FollowerToUser") // フォロー中一覧
  blockedBy     Block[]   @relation("BlockedUser") // ブロックされた側
  blocking      Block[]   @relation("BlockingUser") // ブロックした側
  mutedBy       Mute[]    @relation("MutedUser") // ミュートされた側
  muting        Mute[]    @relation("MutingUser") // ミュートした側
  followRequestsSent FollowRequest[] @relation("FollowRequestSent")
}
```

```

followRequestsReceived FollowRequest[] @relation("FollowRequestReceived")
notifications           Notification[] @relation("NotificationUser")
actorNotifications     Notification[] @relation("NotificationActor")
shopReviews            ShopReview[]    // 盆栽園レビュー
bonsaiShops            BonsaiShop[]    // 登録した盆栽園
events                 Event[]         // 登録したイベント
reports                Report[]        // 通報
adminUser              AdminUser?     // 管理者情報
conversationParticipants ConversationParticipant[] // DM参加
sentMessages           Message[]       // 送信メッセージ
scheduledPosts          ScheduledPost[] // 予約投稿
payments               Payment[]        // 支払い履歴
draftPosts             DraftPost[]      // 下書き
hiddenPosts            UserHiddenPost[] // 非表示投稿
bonsais                Bonsai[]         // 盆栽コレクション
shopChangeRequests     ShopChangeRequest[] // 盆栽園変更リクエスト
analytics              UserAnalytics[]  // アナリティクス
commentThreadMutes     CommentThreadMute[] // コメントスレッドミュート

@@map("users") // DBテーブル名: users
}

```

使用ファイル: `prisma/schema.prisma`, `lib/actions/user.ts`, `lib/actions/auth.ts`

設計ポイント:

- `cuid()`: 衝突しにくい一意のID生成。UUID (`uuid()`) よりも短く、ソート可能です
- `@map`: JavaScriptはキャメルケース (`avatarUrl`)、DBはスネークケース (`avatar_url`) で統一する慣習に対応
- `@db.VarChar(50)`: PostgreSQLの文字列長制限。アプリ側のバリデーションに加えてDB側でも制限をかけることで二重の安全策を講じています
- `isPremium` + `stripeCustomerId`: Stripe決済と連携した有料会員管理
- `twoFactorEnabled` + `twoFactorSecret`: TOTP方式の2段階認証。シークレットは暗号化して保存します
- `notificationPreferences`: Json型で通知の種類ごとのON/OFF設定を柔軟に保存します
- `isSuspended`: 通報による自動停止や管理者による手動停止に対応
- `password` が `String?` (オプション) な理由: GoogleログインなどのOAuth認証ではパスワードが不要なため

7.6.2 Postモデル（投稿）

Postモデルは投稿データを保存します。BON-LOGのメイン機能であり、テキスト投稿、引用投稿、リポスト（再投稿）を1つのモデルで表現しています。

```

model Post {
  // — 基本情報 —
  id          String    @id @default(cuid())           // 投稿の一意ID
  userId      String    @map("user_id")              // 投稿者のユーザーID（外部キー）
  content     String?   // 投稿テキスト（最大500文字はアプリ側で制御）

  // — 引用・リポスト —
  quotePostId String?   @map("quote_post_id")         // 引用元投稿のID（引用でない場合はNULL）
  repostPostId String? @map("repost_post_id")         // リポスト元投稿のID（リポストでない場合はNULL）

  // — 日時 —
  createdAt   DateTime @default(now()) @map("created_at") // 投稿日時
  updatedAt   DateTime @updatedAt @map("updated_at")      // 更新日時

  // — リレーション —
  // fields: [userId] → このモデルのuserIdカラムが
  // references: [id] → Userモデルのidカラムを参照する
  // onDelete: Cascade → ユーザーが削除されたら、この投稿も自動削除
  //
  // 【onDelete の選択肢】
  // Cascade → 親が消えたら子も消す（ユーザー削除 → 投稿も削除）
  // SetNull → 親が消えたら外部キーをNULLにする（孤立するが残る）
  // Restrict → 子が存在する限り親の削除を拒否する（安全策）
  // NoAction → データベースのデフォルト動作に任せる
  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  // 自己参照リレーション（Postテーブルが自分自身を参照する）
  // "quotes"という名前で引用元と引用先を区別
  quotePost  Post?    @relation("quotes", fields: [quotePostId], references: [id])
  quotedBy   Post[]    @relation("quotes") // この投稿を引用している投稿一覧

  // "reposts"という名前でリポスト元とリポスト先を区別
  repostPost Post?    @relation("reposts", fields: [repostPostId], references: [id])
  repostedBy Post[]    @relation("reposts") // この投稿をリポストしている投稿一覧

  // 関連データ
  media      PostMedia[] // この投稿に添付された画像・動画
  genres     PostGenre[] // この投稿のジャンル（最大3つ）
  comments   Comment[]   // この投稿へのコメント一覧
  likes      Like[]       // この投稿へのいいね一覧
  bookmarks  Bookmark[]  // この投稿のブックマーク一覧

  @@map("posts") // DBテーブル名: posts
}

```

投稿の3つの種類（1つのPostモデルで表現）：

#	種類	content	quotePostId	repostPostId	参照先
1	通常の投稿	"松の盆栽 ... "	NULL	NULL	なし
2	引用投稿（コメント付きで他人の投稿を紹介）	"素晴らしい!"	"p001"	NULL	元の投稿 id: "p001"
3	リポスト（そのまま再投稿）	NULL	NULL	"p002"	元の投稿 id: "p002"

設計ポイント:

- `userId`: 外部キー（誰が投稿したか）。UserモデルのIDを参照しています
- `onDelete: Cascade`: ユーザー削除時に、そのユーザーの投稿も自動的に削除されます。これがないとユーザー削除時に「存在しないユーザーの投稿」が孤立してしまいます
- 自己参照リレーション: `quotePostId` で同じPostテーブルのレコードを参照する高度なパターンです

7.6.3 PostMediaモデル（投稿メディア）

投稿に添付された画像や動画を管理するモデルです。1つの投稿に最大4枚の画像、または1本の動画を添付できます。

```
model PostMedia {
  id      String  @id @default(cuid())           // メディアの一意ID
  postId  String  @map("post_id")               // どの投稿に紐づくか（外部キー）
  url     String
  type    String
  sortOrder Int   @default(0) @map("sort_order") // 種別: 'image' (画像) or 'video' (動画)
                                                    // 表示順序 (0から始まる)

  // この投稿が削除されたら、メディアも自動削除
  post    Post    @relation(fields: [postId], references: [id], onDelete: Cascade)

  @@map("post_media") // DBテーブル名: post_media
}
```

設計ポイント:

- 1つの投稿に複数のメディアを紐付け（1対多: Post 1 → PostMedia 多）
- `sortOrder`: メディアの表示順序を管理（0番目が最初に表示される画像）
- `type` フィールド: `String` 型で `'image'` または `'video'` を格納します。enum型を使わず文字列にすることで、将来新しいメディアタイプ（GIF等）を追加する際にスキーマ変更が不要になります

7.6.4 Genreモデル（ジャンル）

盆栽のジャンル（松柏類、雑木類、用品・道具など）を管理するモデルです。投稿には1~3個のジャンルを選択する必要があります。PostとGenreは多対多の関係なので、中間テーブル（PostGenre）を使います。

```
model Genre {
  id      String   @id @default(cuid())           // ジャンルの一意ID
  name    String
  category String   // カテゴリ（例: "樹種"、"用品"）
  type    String   @default("post")              // 用途: "post"（投稿用）or "shop"（盆栽園用）
  sortOrder Int    @default(0) @map("sort_order") // 表示順序

  postGenres      PostGenre[] // このジャンルに属する投稿一覧
  shopGenres      ShopGenre[] // このジャンルに属する盆栽園一覧
  scheduledPostGenres ScheduledPostGenre[] // 予約投稿のジャンル
  draftPostGenres DraftPostGenre[] // 下書きのジャンル

  @@map("genres") // DBテーブル名: genres
}

// — 投稿とジャンルの中間テーブル（多対多リレーション） —
// なぜ中間テーブルが必要？
// → 1つの投稿は複数のジャンルに属し、1つのジャンルには複数の投稿がある
// → この「多対多」の関係を直接表現するカラムはないため、
// 間に橋渡し役のテーブルを置く
model PostGenre {
  postId String @map("post_id") // どの投稿か
  genreId String @map("genre_id") // どのジャンルか

  post Post @relation(fields: [postId], references: [id], onDelete: Cascade)
  genre Genre @relation(fields: [genreId], references: [id], onDelete: Cascade)

  // 複合主キー: postIdとgenreIdの組み合わせが一意
  // → 同じ投稿に同じジャンルを2回紐付けることを防止
  @@id([postId, genreId])
  @@map("post_genres") // DBテーブル名: post_genres
}
```

多対多リレーションの実際のデータ例:

genres テーブル:

id	name	category	type
g1	松柏類	樹種	post
g2	雑木類	樹種	post

post_genres テーブル:

postId	genreId
p1	g1
p1	g2
p2	g2
p3	g1

posts テーブル:

id	content
p1	黒松の...
p2	紅葉の...
p3	五葉松...

- 投稿p1は松柏類(g1)と雑木類(g2)の2ジャンル
- ジャンル雑木類(g2)には投稿p1とp2が属する

設計ポイント:

- 多対多リレーションは中間テーブル **PostGenre** で実現
- **@@id([postId, genreId])**: 複合主キーで「同じ投稿に同じジャンルを2回付ける」重複を防止
- **category**: ジャンルの分類（「樹種」「用品」など）。UIでジャンルをグループ分けして表示する際に使います
- **type**: **"post"**（投稿用ジャンル）と **"shop"**（盆栽園用ジャンル）を区別。同じGenreテーブルで複数の用途のジャンルを管理します

7.6.5 Followモデル（フォロー関係）

Followモデルは、ユーザー間のフォロー関係を管理します。これは「自己結合」（同じテーブルを2回参照する）という少し特殊なパターンです。

```

model Follow {
  followerId String @map("follower_id") // フォローする人 (AがBをフォロー → AのID)
  followingId String @map("following_id") // フォローされる人 (AがBをフォロー → BのID)
  createdAt DateTime @default(now()) @map("created_at") // フォローした日時

  // 同じUserテーブルを2回参照するため、名前で区別する
  follower User @relation("FollowerToUser", fields: [followerId], references: [id], onDelete: Cascade)
  following User @relation("FollowingToUser", fields: [followingId], references: [id], onDelete: Cascade)

  // 複合主キー: 同じペアの重複を防止
  // AliceがBobを2回フォローすることはできない
  @@id([followerId, followingId])
  @@index([followerId])
  @@index([followingId])
  @@map("follows") // DBテーブル名: follows
}

```

フォロー関係の図解:

```

Alice (u1) —フォロー—> Bob (u2)    follows: { followerId: "u1", followingId: "u2" }
Alice (u1) —フォロー—> Charlie (u3) follows: { followerId: "u1", followingId: "u3" }
Bob (u2)   —フォロー—> Alice (u1)   follows: { followerId: "u2", followingId: "u1" }

```

Aliceの視点:

- フォロー中 (following): Bob, Charlie
- フォロワー (followers): Bob
- AliceとBobは相互フォロー

設計ポイント:

- 自己結合: 同じ `User` テーブルを2回参照する特殊なパターン
- `@relation("FollowerToUser")` と `@relation("FollowingToUser")` で、どちら側のリレーションかを区別。
Userモデル側の `followers` は `@relation("FollowingToUser")` に、`following` は `@relation("FollowerToUser")` に対応します
- 複合主キー `@@id([followerId, followingId])` で同じペアの重複フォローを防止
- `@@index([followerId])` と `@@index([followingId])`: フォロワー一覧・フォローワー一覧の取得を高速化するインデックス

7.6.6 Like・Comment・Bookmarkモデル

SNSの主要なインタラクション機能を支えるモデルです。

```
// — いいねモデル —
// 投稿とコメントの両方に「いいね」できる設計
model Like {
  id      String    @id @default(cuid())           // いいねの一意ID
  userId  String    @map("user_id")                // いいねしたユーザー
  postId  String?   @map("post_id")                // 投稿へのいいね（投稿いいねの場合に値が入る）
  commentId String? @map("comment_id")            // コメントへのいいね（コメントいいねの場合に値が入る）
  createdAt DateTime @default(now()) @map("created_at") // いいねした日時

  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  post      Post?     @relation(fields: [postId], references: [id], onDelete: Cascade)
  comment   Comment?  @relation(fields: [commentId], references: [id], onDelete: Cascade)

  // 複合ユニーク制約: 同じユーザーが同じ投稿に2回いいねできない
  @@unique([userId, postId])
  // 複合ユニーク制約: 同じユーザーが同じコメントに2回いいねできない
  @@unique([userId, commentId])
  @@map("likes") // DBテーブル名: likes
}

// — コメントモデル（スレッド形式対応） —
model Comment {
  id      String    @id @default(cuid())           // コメントの一意ID
  postId  String    @map("post_id")                // どの投稿へのコメントか
  userId  String    @map("user_id")                // コメントしたユーザー
  parentId String?   @map("parent_id")            // 親コメントのID（返信の場合に値が入る）
  content String     @db.Text                      // コメント本文
  isHidden Boolean   @default(false) @map("is_hidden") // 管理者による非表示フラグ
  hiddenAt DateTime? @map("hidden_at")            // 非表示にされた日時
  deletedAt DateTime? @map("deleted_at")          // 論理削除日時
  createdAt DateTime @default(now()) @map("created_at") // コメント日時

  post      Post      @relation(fields: [postId], references: [id], onDelete: Cascade)
  user      User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  parent    Comment?  @relation("CommentReplies", fields: [parentId], references: [id], onDelete: Cascade)
  replies   Comment[] @relation("CommentReplies") // このコメントへの返信一覧
  likes     Like[]    // このコメントへのいいね一覧
  media     CommentMedia[] // コメントに添付されたメディア

  @@index([postId])
  @@index([userId])
  @@map("comments") // DBテーブル名: comments
}

// — ブックマークモデル —
// 投稿を後で読み返すために保存する機能
model Bookmark {
  id      String    @id @default(cuid())           // ブックマークの一意ID
  userId  String    @map("user_id")                // ブックマークしたユーザー
  postId  String    @map("post_id")                // ブックマークした投稿
  createdAt DateTime @default(now()) @map("created_at") // ブックマーク日時

  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  post      Post       @relation(fields: [postId], references: [id], onDelete: Cascade)

  // 複合ユニーク制約: 同じ投稿を2回ブックマークできない
}
```



```
    @@unique([userId, postId])  
    @@map("bookmarks") // DBテーブル名: bookmarks  
}
```

@@unique の重要性: `@@unique([userId, postId])` がないと、同じユーザーが同じ投稿に何回もいいねやブックマークができてしまいます。データベースレベルでこの制約を設けることで、アプリケーションコードにバグがあっても不正なデータが入ることを防げます。

▶ 理解度チェック: BON-LOGのスキーマ設計

7.6.7 Block・Muteモデル（ブロック・ミュート）

ユーザー間の関係制御を担うモデルです。ブロックは双方向で関係を断ち、ミュートは片方向で投稿を非表示にします。

```
// prisma/schema.prisma より抜粋

// — ブロックモデル —
// ブロックすると相互フォロー解除 + 相手の投稿が非表示になる
model Block {
  blockerId String @map("blocker_id") // ブロックした人
  blockedId String @map("blocked_id") // ブロックされた人
  createdAt DateTime @default(now()) @map("created_at")

  blocker User @relation("BlockingUser", fields: [blockerId], references: [id], onDelete: Cascade)
  blocked User @relation("BlockedUser", fields: [blockedId], references: [id], onDelete: Cascade)

  // 複合主キー: 同じ組み合わせは1つだけ
  @@id([blockerId, blockedId])
  @@index([blockedId])
  @@map("blocks")
}

// — ミュートモデル —
// ミュートすると相手の投稿が非表示になるが、フォロー関係は維持
model Mute {
  muterId String @map("muter_id") // ミュートした人
  mutedId String @map("muted_id") // ミュートされた人
  createdAt DateTime @default(now()) @map("created_at")

  muter User @relation("MutingUser", fields: [muterId], references: [id], onDelete: Cascade)
  muted User @relation("MutedUser", fields: [mutedId], references: [id], onDelete: Cascade)

  // 複合主キー: 同じ組み合わせは1つだけ
  @@id([muterId, mutedId])
  @@index([mutedId])
  @@map("mutes")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/block.ts`, `lib/actions/mute.ts`

ブロックの実装コード:

```

// lib/actions/block.ts

// ブロック実行: トランザクションで相互フォロー解除 + ブロック作成
export async function blockUser(targetUserId: string) {
  const { userId, error: authError } = await requireAuth()
  if (!userId) return { error: authError! }

  if (userId === targetUserId) {
    return { error: '自分自身をブロックできません' }
  }

  // $transaction で複数操作をアトミックに実行
  await prisma.$transaction([
    // 相互フォローを一括削除
    prisma.follow.deleteMany({
      where: {
        OR: [
          { followerId: userId, followingId: targetUserId },
          { followerId: targetUserId, followingId: userId },
        ],
      },
    }),
    // ブロックレコード作成
    prisma.block.create({
      data: { blockerId: userId, blockedId: targetUserId },
    }),
  ])

  revalidatePath('/feed')
  return { success: true }
}

// ブロック状態の双方向確認
export async function isBlocked(targetUserId: string) {
  const [block, blockedBy] = await Promise.all([
    prisma.block.findUnique({
      where: { blockerId_blockedId: { blockerId: session.user.id, blockedId: targetUserId } },
    }),
    prisma.block.findUnique({
      where: { blockerId_blockedId: { blockerId: targetUserId, blockedId: session.user.id } },
    }),
  ])

  return { blocked: !!block, blockedBy: !!blockedBy }
}

```

実行結果:

```
// ブロック実行
{ success: true }

// ブロック状態確認
{ blocked: true, blockedBy: false }
// → 自分が相手をブロック中、相手からはブロックされていない
```

ブロックとミュートの違い:

項目	ブロック	ミュート
フォロー関係	双方向解除	維持
相手への影響	あり（相手も自分を見えない）	なし（相手は気づかない）
主なユースケース	嫌がらせ対策	興味のない投稿を非表示

この実装で可能になること: 悪質なユーザーとの関係を完全に断つブロック機能と、穏やかに投稿を非表示にするミュート機能の両方を提供できます。

実装しない場合の影響: 嫌がらせを受けているユーザーが加害者のコンテンツを避ける手段がなくなり、SNSの安全性が大きく損なわれます。

7.6.8 FollowRequestモデル（フォローリクエスト）

非公開アカウントへのフォローリクエストを管理するモデルです。

```
// prisma/schema.prisma より抜粋

// フォローリクエスト（非公開アカウント用）
model FollowRequest {
  id          String          @id @default(cuid())
  requesterId String          @map("requester_id") // リクエスト送信者
  targetId    String          @map("target_id")    // リクエスト受信者（非公開アカウント）
  status      RequestStatus @default(pending)
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

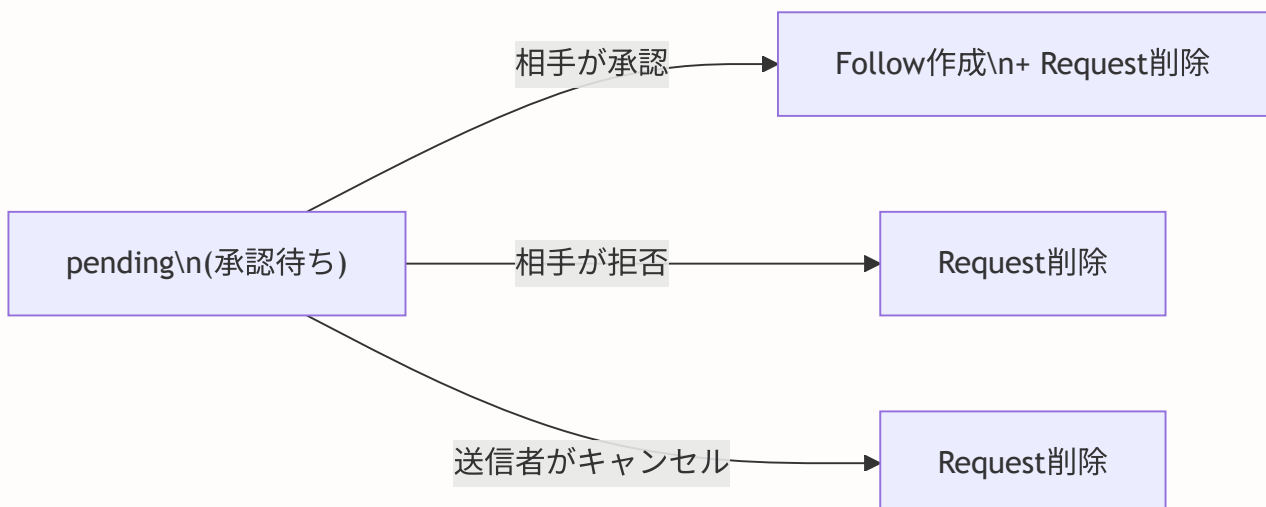
  requester User @relation("FollowRequestSent", fields: [requesterId], references: [id], onDelete: delete)
  target     User @relation("FollowRequestReceived", fields: [targetId], references: [id], onDelete: delete)

  @@unique([requesterId, targetId]) // 同じ相手に複数回リクエスト不可
  @@index([requesterId])
  @@index([targetId])
  @@index([status])
  @@map("follow_requests")
}

// リクエストの状態を列挙型で定義
enum RequestStatus {
  pending // 承認待ち
  approved // 承認済み
  rejected // 拒否済み
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/follow-request.ts`

フォローリクエストのライフサイクル:



実装コード:

```
// lib/actions/follow-request.ts

// フォローリクエスト送信
export async function sendFollowRequest(targetUserId: string) {
  const session = await auth()
  if (!session?.user?.id) return { error: '認証が必要です' }

  // リクエスト作成 + 通知送信
  await prisma.followRequest.create({
    data: {
      requesterId: session.user.id,
      targetId: targetUserId,
    },
  })

  await prisma.notification.create({
    data: {
      userId: targetUserId,
      actorId: session.user.id,
      type: 'follow_request',
    },
  })

  return { success: true, status: 'pending' }
}
```

実行結果:

```
{ success: true, status: 'pending' }
// → 相手にフォローリクエスト通知が送信される
```

この実装で可能になること: 非公開アカウントのユーザーが、誰にフォローを許可するかを自分で制御できます。

実装しない場合の影響: 非公開設定にしても誰でもフォローできてしまい、プライバシー保護が機能しません。

7.6.9 Notificationモデル（通知）

ユーザーへのアプリ内通知を管理するモデルです。いいね、コメント、フォローなど様々なイベントに対応しています。

```
// prisma/schema.prisma より抜粋

// 通知タイプの列挙型
enum NotificationType {
  like           // 投稿へのいいね
  comment        // 投稿へのコメント
  follow         // フォロー
  quote          // 引用投稿
  reply          // コメントへの返信
  comment_like   // コメントへのいいね
  follow_request // フォローリクエスト
  follow_request_approved // フォローリクエスト承認
  system         // システム通知
  mention        // メンション
  message        // ダイレクトメッセージ
  repost         // リポスト
  subscription_expiring // サブスクリプション期限
}

// 通知モデル
model Notification {
  id          String      @id @default(cuid())
  userId      String      @map("user_id")      // 通知を受け取るユーザー
  actorId     String      @map("actor_id")      // アクション実行者
  type        NotificationType // 通知タイプ
  postId      String?      @map("post_id")      // 関連投稿（あれば）
  commentId   String?      @map("comment_id")   // 関連コメント（あれば）
  isRead      Boolean      @default(false) @map("is_read")
  createdAt   DateTime     @default(now()) @map("created_at")

  user      User    @relation("NotificationUser", fields: [userId], references: [id], onDelete: Cascade)
  actor     User    @relation("NotificationActor", fields: [actorId], references: [id], onDelete: Cascade)
  post      Post?   @relation(fields: [postId], references: [id], onDelete: Cascade)
  comment   Comment? @relation(fields: [commentId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([userId, isRead]) // 未読通知の高速取得
  @@map("notifications")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/notification.ts`

実装コード（通知一覧取得）:

```

// lib/actions/notification.ts

// 通知一覧を取得（ミュートユーザーを除外）
export async function getNotifications(cursor?: string, limit = 20) {
  const session = await auth()
  if (!session?.user?.id) return { error: '認証が必要です' }

  // ミュートしているユーザーIDを取得
  const mutedUserIds = await getMutedUserIds(session.user.id)

  const notifications = await prisma.notification.findMany({
    where: {
      userId: session.user.id,
      ... (mutedUserIds.length > 0 && {
        actorId: { notIn: mutedUserIds }, // ミュートユーザーを除外
      }),
    },
    include: {
      actor: { select: { id: true, nickname: true, avatarUrl: true } },
      post: { select: { id: true, content: true } },
    },
    orderBy: { createdAt: 'desc' },
    take: limit,
    ... (cursor && { cursor: { id: cursor }, skip: 1 }),
  })

  return { notifications }
}

// 全通知を既読にする
export async function markAllNotificationsAsRead() {
  const session = await auth()
  if (!session?.user?.id) return { error: '認証が必要です' }

  await prisma.notification.updateMany({
    where: { userId: session.user.id, isRead: false },
    data: { isRead: true },
  })

  return { success: true }
}

```

実行結果:


```
// 通知一覧
{
  notifications: [
    {
      id: "clx ... ",
      type: "like",
      isRead: false,
      actor: { id: "user2", nickname: "盆栽太郎", avatarUrl: "/ ... " },
      post: { id: "post1", content: "五葉松の手入れ ... " }
    },
    // ...
  ]
}
```

設計ポイント: `@@index([userId, isRead])` は「未読通知だけを取得」するクエリを高速化する複合インデックスです。通知ページでは未読フィルタが頻繁に使われるため、この最適化は重要です。

この実装で可能になること: いいね、コメント、フォローなど全てのインタラクションを1つの通知画面で確認できます。

実装しない場合の影響: ユーザーが自分の投稿への反応に気づけず、SNSの双方向コミュニケーションが成立しません。

7.6.10 BonsaiShop関連モデル（盆栽園マップ）

盆栽園の情報、レビュー、ジャンル分類を管理する一連のモデルです。

```
// prisma/schema.prisma より抜粋

// 盆栽園
model BonsaiShop {
  id          String    @id @default(cuid())
  name        String    @db.VarChar(100)
  address     String
  latitude    Decimal?  @db.Decimal(10, 7) // 緯度 (小数点以下7桁)
  longitude   Decimal?  @db.Decimal(10, 7) // 経度 (小数点以下7桁)
  phone       String?   @db.VarChar(20)
  website     String?
  businessHours String? @map("business_hours")
  closedDays  String?   @map("closed_days")
  isHidden    Boolean   @default(false) @map("is_hidden") // 通報による非表示
  hiddenAt    DateTime? @map("hidden_at")
  createdBy   String    @map("created_by") // 登録者
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  creator      User          @relation(fields: [createdBy], references: [id], onDelete: Cascade)
  genres       ShopGenre[]   // 取扱ジャンル
  reviews      ShopReview[]  // レビュー一覧
  changeRequests ShopChangeRequest[] // 変更リクエスト

  @@index([isHidden])
  @@index([createdBy])
  @@map("bonsai_shops")
}

// 盆栽園ジャンル (中間テーブル)
model ShopGenre {
  shopId String @map("shop_id")
  genreId String @map("genre_id")

  shop BonsaiShop @relation(fields: [shopId], references: [id], onDelete: Cascade)
  genre Genre @relation(fields: [genreId], references: [id], onDelete: Cascade)

  @@id([shopId, genreId])
  @@map("shop_genres")
}

// 盆栽園レビュー
model ShopReview {
  id          String    @id @default(cuid())
  shopId      String    @map("shop_id")
  userId      String    @map("user_id")
  rating      Int        // 1-5の評価
  content     String?   @db.Text
  isHidden    Boolean   @default(false) @map("is_hidden")
  hiddenAt    DateTime? @map("hidden_at")
  createdAt   DateTime  @default(now()) @map("created_at")

  shop BonsaiShop @relation(fields: [shopId], references: [id], onDelete: Cascade)
  user  User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  images ShopReviewImage[] // レビュー画像
}
```

```
    @@index([shopId])
    @@index([userId])
    @@index([isHidden])
    @@map("shop_reviews")
  }

  // レビュー画像
  model ShopReviewImage {
    id      String @id @default(cuid())
    reviewId String @map("review_id")
    url     String

    review ShopReview @relation(fields: [reviewId], references: [id], onDelete: Cascade)

    @@index([reviewId])
    @@map("shop_review_images")
  }
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/shop.ts`, `lib/actions/review.ts`

盆栽園の検索・フィルタリング:

```
// lib/actions/shop.ts

// 盆栽園一覧をフィルター条件付きで取得
export async function getShops(options?: {
  search?: string
  genreId?: string
  prefecture?: string
}) {
  const { search, genreId, prefecture } = options || {}

  const shops = await prisma.bonsaiShop.findMany({
    where: {
      isHidden: false,
      ... (search && {
        OR: [
          { name: { contains: search, mode: 'insensitive' } },
          { address: { contains: search, mode: 'insensitive' } },
        ],
      }),
      ... (genreId && {
        genres: { some: { genreId } },
      }),
    },
    include: {
      creator: { select: { id: true, nickname: true, avatarUrl: true } },
      genres: { include: { genre: true } },
      _count: { select: { reviews: { where: { isHidden: false } } } },
    },
  })

  return shops
}
```

実行結果:

```
[
  {
    id: "shop1",
    name: "春花園BONSAI美術館",
    address: "東京都江戸川区 ... ",
    latitude: 35.6762,
    longitude: 139.8672,
    genres: [{ genre: { name: "小品盆栽" } }, { genre: { name: "道具" } }],
    _count: { reviews: 12 }
  },
  // ...
]
```

設計ポイント: `Decimal(10, 7)` は緯度・経度の精度として約1cmの誤差に対応しています。Leaflet地図との連携に十分な精度です。

この実装で可能になること: 全国の盆栽園を地図上で表示し、ジャンル・地域で絞り込み検索ができます。

実装しない場合の影響: 盆栽園の情報が管理できず、SNSの「盆栽園マップ」機能が提供できません。

7.6.11 Eventモデル（イベント）

盆栽関連のイベント情報を管理するモデルです。

```
// prisma/schema.prisma より抜粋

// イベント
model Event {
  id          String      @id @default(cuid())
  title       String      @db.VarChar(100)
  description  String?     @db.Text
  startDate   DateTime     @map("start_date")
  endDate     DateTime?    @map("end_date")
  prefecture  String?      // 都道府県
  city        String?      // 市区町村
  venue       String?      // 会場名
  organizer   String?      // 主催者
  admissionFee String?     @map("admission_fee") // 入場料
  hasSales    Boolean      @default(false) @map("has_sales") // 即売会の有無
  externalUrl  String?     @map("external_url") // 外部リンク
  isHidden    Boolean      @default(false) @map("is_hidden")
  hiddenAt    DateTime?    @map("hidden_at")
  createdBy   String      @map("created_by")
  createdAt   DateTime     @default(now()) @map("created_at")

  creator User @relation(fields: [createdBy], references: [id], onDelete: Cascade)

  @@index([startDate]) // 日付順ソートの高速化
  @@index([prefecture]) // 都道府県フィルタの高速化
  @@index([isHidden])
  @@map("events")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/event.ts`

イベント取得の実装コード:

```
// lib/actions/event.ts

// 今後開催されるイベント一覧を取得
export async function getUpcomingEvents(options?: {
  prefecture?: string
  region?: string
}) {
  const today = getStartOfToday()

  const events = await prisma.event.findMany({
    where: {
      isHidden: false,
      OR: [
        { endDate: { gte: today } }, // 終了日が今日以降
        { endDate: null, startDate: { gte: today } }, // 終了日なし、開始日が今日以降
      ],
      ... (options?.prefecture && { prefecture: options.prefecture }),
    },
    include: {
      creator: { select: { id: true, nickname: true, avatarUrl: true } },
    },
    orderBy: { startDate: 'asc' },
  })

  return events
}
```

実行結果:

```
[
  {
    id: "evt1",
    title: "第50回 日本盆栽大観展",
    startDate: "2026-02-01T00:00:00.000Z",
    endDate: "2026-02-05T00:00:00.000Z",
    prefecture: "東京都",
    venue: "上野グリーンクラブ",
    hasSales: true,
    creator: { nickname: "盆栽花子" }
  },
  // ...
]
```

この実装で可能になること: カレンダー形式で盆栽イベントを閲覧し、地域フィルタで近くのイベントを見つけられます。

実装しない場合の影響: イベント情報の共有手段がなくなり、盆栽コミュニティの活性化に繋がりません。

7.6.12 Reportモデル（通報）

不適切なコンテンツの通報と自動非表示機能を支えるモデルです。

```
// prisma/schema.prisma より抜粋

// 通報対象の種類
enum ReportTargetType {
  post      // 投稿
  comment   // コメント
  event     // イベント
  shop      // 盆栽園
  review    // レビュー
  user      // ユーザー
}

// 通報理由
enum ReportReason {
  spam          // スパム
  inappropriate // 不適切な内容
  harassment    // 嫌がらせ
  copyright     // 著作権侵害
  other         // その他
}

// 通報ステータス
enum ReportStatus {
  pending      // 未対応
  reviewed     // 確認中
  resolved     // 対応済み
  dismissed    // 却下
  auto_hidden  // 自動非表示
}

// 通報モデル
model Report {
  id          String      @id @default(cuid())
  reporterId  String      @map("reporter_id")
  targetType  ReportTargetType @map("target_type") // 何を通報するか
  targetId    String      @map("target_id")      // 通報対象のID
  reason      ReportReason // 通報理由
  description String?      @db.Text              // 詳細説明
  status      ReportStatus @default(pending)
  createdAt   DateTime @default(now()) @map("created_at")

  reporter User @relation(fields: [reporterId], references: [id], onDelete: Cascade)

  @@index([targetType, targetId]) // 同一コンテンツへの通報集計用
  @@index([status])
  @@map("reports")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/report.ts`

通報作成と自動非表示の仕組み:

```
// lib/actions/report.ts

export async function createReport(params: {
  targetType: ReportTargetType
  targetId: string
  reason: ReportReason
  description?: string
}) {
  // ... 認証・バリデーション ...

  // 通報を作成
  await prisma.report.create({
    data: {
      reporterId: session.user.id,
      targetType: params.targetType,
      targetId: params.targetId,
      reason: params.reason,
      description: params.description || null,
      status: 'pending',
    },
  })

  // 同一コンテンツへの通報数をカウント
  const reportCount = await prisma.report.count({
    where: { targetType: params.targetType, targetId: params.targetId },
  })

  // しきい値に達したら自動非表示
  if (reportCount ≥ AUTO_HIDE_THRESHOLD) {
    await autoHideContent(params.targetType, params.targetId, reportCount)
  }

  return { success: true }
}
```

通報統計の取得（管理者向け）:


```
// lib/actions/report.ts

export async function getReportStats() {
  const [pending, reviewed, resolved, dismissed, byType] = await Promise.all([
    prisma.report.count({ where: { status: 'pending' } }),
    prisma.report.count({ where: { status: 'reviewed' } }),
    prisma.report.count({ where: { status: 'resolved' } }),
    prisma.report.count({ where: { status: 'dismissed' } }),
    prisma.report.groupBy({
      by: ['targetType'],
      _count: true,
    }),
  ])

  return {
    stats: { pending, reviewed, resolved, dismissed, total: pending + reviewed + resolved + dismissed },
  }
}
```

実行結果:

```
{ stats: { pending: 5, reviewed: 2, resolved: 10, dismissed: 3, total: 20 } }
```

設計ポイント: `@@index([targetType, targetId])` は「同じコンテンツに何件の通報があるか」を高速にカウントするための複合インデックスです。自動非表示の判定で毎回使われます。

この実装で可能になること: 不適切なコンテンツをユーザーが通報でき、一定数の通報で自動的に非表示になります。

実装しない場合の影響: スпамや嫌がらせコンテンツの対処が管理者の手動作業だけに頼ることになり、対応が追いつきません。

7.6.13 管理者モデル (AdminUser / AdminLog / AdminNotification)

管理者の権限管理、操作ログ、管理者通知を担うモデル群です。

```

// prisma/schema.prisma より抜粋

// 管理者の権限レベル
enum AdminRole {
  admin          // 管理者（全権限）
  moderator      // モデレーター（コンテンツ管理のみ）
}

// 管理者
model AdminUser {
  userId      String    @id @map("user_id") // UserテーブルのIDを主キーに
  role        AdminRole
  createdAt   DateTime @default(now()) @map("created_at")

  user User            @relation(fields: [userId], references: [id], onDelete: Cascade)
  logs AdminLog[]

  @@map("admin_users")
}

// 管理者操作ログ（監査用）
model AdminLog {
  id          String    @id @default(cuid())
  adminId     String    @map("admin_id")
  action      String
  targetType  String?   @map("target_type") // 実行したアクション名
  targetId    String?   @map("target_id")   // 操作対象の種類
  details     Json?     // 操作対象のID
  createdAt   DateTime @default(now()) @map("created_at")

  admin AdminUser @relation(fields: [adminId], references: [userId], onDelete: Cascade)

  @@map("admin_logs")
}

// 管理者通知（自動非表示など）
model AdminNotification {
  id          String    @id @default(cuid())
  type        String
  targetType  String    @map("target_type") // 'auto_hidden', 'report_threshold' 等
  targetId    String    @map("target_id")
  message     String
  reportCount Int        @map("report_count")
  isRead      Boolean    @default(false) @map("is_read")
  isResolved  Boolean    @default(false) @map("is_resolved")
  resolvedAt  DateTime? @map("resolved_at")
  createdAt   DateTime @default(now()) @map("created_at")

  @@index([isRead])
  @@index([isResolved])
  @@index([createdAt])
  @@map("admin_notifications")
}

```

使用ファイル: `prisma/schema.prisma`, `lib/actions/admin.ts`, `lib/actions/report.ts`

管理ログの記録:

```
// lib/actions/report.ts (通報ステータス更新時)

await prisma.$transaction([
  prisma.report.update({
    where: { id: reportId },
    data: { status },
  }),
  prisma.adminLog.create({
    data: {
      adminId: adminUserId,
      action: 'update_report_status',
      targetType: 'report',
      targetId: reportId,
      details: JSON.stringify({
        previousStatus: report.status,
        newStatus: status,
      }),
      note,
    },
  }),
])
```

設計ポイント: `AdminUser` のIDは `User` テーブルのIDをそのまま主キーとして使います。これにより、`User` テーブルと `AdminUser` テーブルが1対1の関係になり、管理者かどうかを1回のクエリで判定できます。

この実装で可能になること: 管理者が誰がいつ何をしたかの完全な監査ログを残せます。

実装しない場合の影響: 管理者による不正操作の追跡ができず、運営の透明性が確保できません。

7.6.14 ダイレクトメッセージモデル (Conversation / Message)

ユーザー間のプライベートメッセージ機能を支えるモデル群です。

```
// prisma/schema.prisma より抜粋

// 会話（2人のユーザー間）
model Conversation {
  id          String    @id @default(cuid())
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  participants ConversationParticipant[]
  messages    Message[]

  @@map("conversations")
}

// 会話の参加者
model ConversationParticipant {
  conversationId String    @map("conversation_id")
  userId          String    @map("user_id")
  joinedAt        DateTime  @default(now()) @map("joined_at")
  lastReadAt      DateTime? @map("last_read_at") // 既読管理

  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
  user            User       @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@id([conversationId, userId]) // 複合主キー
  @@index([userId])
  @@map("conversation_participants")
}

// 個々のメッセージ
model Message {
  id          String    @id @default(cuid())
  conversationId String    @map("conversation_id")
  senderId    String    @map("sender_id")
  content      String    @db.Text
  createdAt    DateTime  @default(now()) @map("created_at")

  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
  sender        User        @relation(fields: [senderId], references: [id], onDelete: Cascade)

  @@index([conversationId])
  @@map("messages")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/message.ts`

会話の開始と既読管理:

```
// lib/actions/message.ts

// 会話を開始または既存の会話を取得
export async function startConversation(targetUserId: string) {
  // 既存の会話を検索
  const existing = await prisma.conversation.findFirst({
    where: {
      AND: [
        { participants: { some: { userId: session.user.id } } },
        { participants: { some: { userId: targetUserId } } },
      ],
    },
  })

  if (existing) return { conversationId: existing.id }

  // 新規会話を作成
  const conversation = await prisma.conversation.create({
    data: {
      participants: {
        create: [
          { userId: session.user.id },
          { userId: targetUserId },
        ],
      },
    },
  })

  return { conversationId: conversation.id }
}
```

設計ポイント: `lastReadAt` フィールドにより、「既読/未読」を管理します。`lastReadAt` より後に送信されたメッセージが未読です。LINEの「既読」表示と同じ仕組みです。

この実装で可能になること: ユーザー同士がプライベートにメッセージを交換できます。

実装しない場合の影響: ユーザー間のプライベートなやり取りが公開投稿でしかできなくなります。

7.6.15 Paymentモデル（支払い履歴）

Stripeを使った有料会員の決済履歴を管理するモデルです。

```
// prisma/schema.prisma より抜粋

// 支払いステータス
enum PaymentStatus {
  succeeded // 成功
  pending   // 処理中
  failed    // 失敗
}

// 支払い履歴
model Payment {
  id          String      @id @default(cuid())
  userId      String      @map("user_id")
  stripePaymentId String  @unique @map("stripe_payment_id") // Stripe側のID
  amount      Int          // 金額 (円)
  currency    String      @default("jpy")
  status      PaymentStatus
  description  String?
  createdAt   DateTime @default(now()) @map("created_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("payments")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/subscription.ts`

支払い履歴の取得:

```
// lib/actions/subscription.ts

export async function getPaymentHistory() {
  const session = await auth()
  if (!session?.user?.id) return { error: '認証が必要です' }

  const payments = await prisma.payment.findMany({
    where: { userId: session.user.id },
    orderBy: { createdAt: 'desc' },
    take: 20,
  })

  return { payments }
}
```

実行結果:

```
{
  payments: [
    {
      id: "pay1",
      amount: 500,
      currency: "jpy",
      status: "succeeded",
      description: "BON-LOG プレミアム月額プラン",
      createdAt: "2026-01-15T00:00:00.000Z"
    }
  ]
}
```

この実装で可能になること: プレミアム会員の決済履歴をアプリ内で確認でき、Stripeとの紐付けで正確な課金管理ができます。

実装しない場合の影響: 決済の追跡ができず、返金対応やサポート問い合わせに 대응できません。

7.6.16 盆栽成長記録モデル (Bonsai / BonsaiRecord)

ユーザーが所有する盆栽を登録し、成長記録を写真付きで管理するモデル群です。

```
// prisma/schema.prisma より抜粋

// 盆栽（本体情報）
model Bonsai {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  name        String    // 盆栽の名前
  species     String?   // 樹種
  acquiredAt  DateTime? @map("acquired_at") // 入手日
  description String?   @db.Text           // メモ
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user      User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  records   BonsaiRecord[] // 成長記録一覧
  posts     Post[]        // この盆栽に関連する投稿

  @@index([userId])
  @@map("bonsais")
}

// 盆栽の成長記録
model BonsaiRecord {
  id          String    @id @default(cuid())
  bonsaiId    String    @map("bonsai_id")
  content     String?   @db.Text           // 記録メモ
  recordAt    DateTime  @default(now()) @map("record_at") // 記録日時
  createdAt   DateTime  @default(now()) @map("created_at")

  bonsai      Bonsai      @relation(fields: [bonsaiId], references: [id], onDelete: Cascade)
  images      BonsaiRecordImage[] // 記録画像

  @@index([bonsaiId])
  @@index([recordAt])
  @@map("bonsai_records")
}

// 成長記録の画像
model BonsaiRecordImage {
  id          String    @id @default(cuid())
  recordId    String    @map("record_id")
  url         String
  sortOrder   Int       @default(0) @map("sort_order")

  record      BonsaiRecord @relation(fields: [recordId], references: [id], onDelete: Cascade)

  @@index([recordId])
  @@map("bonsai_record_images")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/bonsai.ts`

盆栽一覧の取得:

```
// lib/actions/bonsai.ts

export async function getBonsais(userId?: string) {
  const session = await auth()
  const targetUserId = userId || session?.user?.id
  if (!targetUserId) return { error: '認証が必要です' }

  const bonsais = await prisma.bonsai.findMany({
    where: { userId: targetUserId },
    include: {
      records: {
        orderBy: { recordAt: 'desc' },
        take: 1, // 最新の記録のみ
        include: {
          images: { take: 1 }, // サムネイル1枚
        },
      },
      _count: { select: { records: true } },
    },
    orderBy: { updatedAt: 'desc' },
  })

  return { bonsais }
}
```

実行結果:

```
{
  bonsais: [
    {
      id: "b1",
      name: "五葉松 寿",
      species: "五葉松",
      acquiredAt: "2020-04-01",
      records: [{ content: "春の芽摘み完了", images: [{ url: "/..." }] }],
      _count: { records: 15 }
    }
  ]
}
```

設計ポイント: `Bonsai` と `Post` の関係により、投稿時に盆栽を紐付けることで「この盆栽に関する投稿一覧」を表示できます。盆栽の成長を時系列で追える「成長日記」機能を実現しています。

この実装で可能になること: 盆栽愛好家が自分のコレクションを管理し、成長記録を写真付きで残せます。

実装しない場合の影響: 盆栽SNSとしてのコア機能である「盆栽管理・成長記録」が提供できません。

7.7 Prismaクライアントの使い方

このセクションで学ぶこと

- Prismaクライアントのシングルトンパターンとその必要性
- 4つの基本操作（CRUD: Create, Read, Update, Delete）
- `include` と `select` を使ったデータ取得の最適化
- カーソルベースのページネーション
- トランザクション（複数操作の原子的実行）
- 集計クエリ（count, aggregate）

7.7.1 シングルトンパターン

開発環境で複数のPrismaインスタンスが作成されるのを防ぐため、シングルトンパターンを使用します。

なぜシングルトンが必要？：Next.jsの開発環境では、ファイルを保存するたびにモジュールが再読み込みされます（ホットリロード）。その都度新しいPrismaインスタンスが作成されると、データベース接続が大量に発生して「Too many connections」エラーになります。シングルトンパターンを使うことで、常に1つのインスタンスだけを再利用します。

Singletonパターンとは？「アプリ全体で1つだけ」のインスタンスを保証するパターンです。データベース接続は作成コストが高いため、リクエストごとに新しく作るのではなく、1つの接続を使い回します。

Next.jsの開発モードではホットリロード時にモジュールが再読み込みされるため、`globalThis` にインスタンスを保存して重複作成を防いでいます。

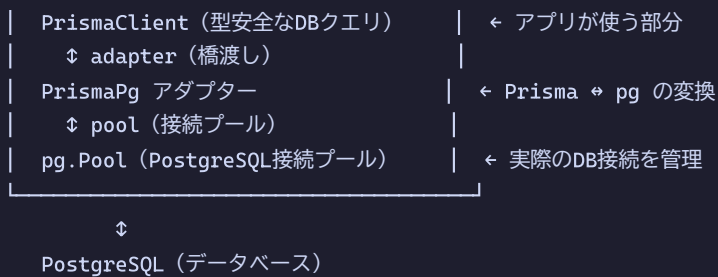
Prisma 6 の新しい接続方式: PrismaPgアダプター

BON-LOGではPrisma 6を使用しており、PostgreSQLへの接続に**PrismaPgアダプター**パターンを採用しています。これはPrisma 5以降で導入された新しい接続方式で、`pg` ライブラリの接続プール（Pool）を介してPostgreSQLと通信します。

なぜPrismaPgアダプターが必要なのか？

項目	旧方式（Prisma標準）	新方式（PrismaPgアダプター）
接続管理	Prismaが内部で管理	<code>pg.Pool</code> で管理（より柔軟）
Vercel/Supabase対応	サーバーレス環境で接続数が増える	接続プールで効率的に管理
SSL設定	限定的	<code>pg.Pool</code> のオプションで細かく制御可能
テスト環境	常に接続が必要	ダミーURL判定でモック切り替えが容易

3層の構造:



BON-LOGでの実際の実装（`lib/db.ts`）:

```

// lib/db.ts

import { PrismaClient } from '@prisma/client'

/**
 * PrismaPg: PostgreSQL用のPrismaアダプター
 * Prisma 5以降で導入された新しいアダプターシステム
 * PostgreSQLとの接続をより効率的に管理する
 */
import { PrismaPg } from '@prisma/adapters-pg'

/**
 * Pool: PostgreSQLの接続プール
 * 複数のデータベース接続を事前に作成して保持し、
 * 必要に応じて再利用することでパフォーマンスを向上させる
 */
import { Pool } from 'pg'

// =====
// グローバル変数の設定
// =====

// globalThis はブラウザの window、Node.jsの global に相当するオブジェクト
// グローバルにPrismaインスタンスを保存して、ホットリロード時の重複作成を防ぐ
const globalForPrisma = global as unknown as { prisma: PrismaClient }

// =====
// PostgreSQL接続プールの作成
// =====

/**
 * CI/テスト環境かどうかを判定
 * ダミーのDATABASE_URLの場合は実際の接続を行わない
 * → テストではモック（偽のDB）を使うので、実際のDB接続は不要
 */
const isDummyDatabase = process.env.DATABASE_URL?.includes('dummy') || false

/**
 * PostgreSQLの接続プールを作成
 *
 * 接続プールとは？: データベース接続の「貯水池」を事前に作成しておき、
 * クエリ実行時に空いている接続を借りて使い、終わったら返却する仕組み。
 * 毎回新しい接続を作成するより大幅に高速。
 */
const pool = isDummyDatabase ? null : new Pool({
  connectionString: process.env.DATABASE_URL,
  // SSL設定: 本番環境は { rejectUnauthorized: false } (自己署名証明書を許可)
  // 開発環境は false (ローカルDBはSSLなし)
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false,
})

/**
 * PrismaとPostgreSQLを接続するアダプターを作成
 * pool が存在する場合のみアダプターを作成 (CI環境ではnull)
 */
const adapter = pool ? new PrismaPg(pool) : null

```

```
// =====
// PrismaClientのエクスポート（シングルトンパターン）
// =====

// ?? はNull合体演算子：左辺がnull/undefinedなら右辺を使う
// つまり、既にインスタンスがあればそれを使い、なければ新しく作る
export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    ... (adapter && { adapter }), // アダプターがある場合のみ渡す（CI環境では省略）
    // 開発環境ではクエリログを表示（デバッグに便利）
    // 本番環境ではエラーのみ表示（パフォーマンスのため）
    log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error'],
  })

// 開発環境でのみ、作成したインスタンスをグローバルに保存
// 次回のホットリロード時に同じインスタンスを再利用するため
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

シングルトンパターンの動作:

ホットリロード1回目:

```
globalForPrisma.prisma → undefined
→ 新しいPrismaClientを作成（PrismaPgアダプター付き）
→ globalForPrisma.prismaに保存
```

ホットリロード2回目:

```
globalForPrisma.prisma → 既存のPrismaClient
→ 既存のインスタンスをそのまま使う（新規作成しない、DB接続も使い回す）
```

ホットリロード3回目:

```
globalForPrisma.prisma → 既存のPrismaClient
→ 既存のインスタンスをそのまま使う（新規作成しない）
```

CI/テスト環境での動作: `DATABASE_URL` に `'dummy'` が含まれる場合（テスト時の設定）、`Pool` とアダプターの作成をスキップします。テストコードではPrismaのモック（偽のDB操作）を使うため、実際のPostgreSQL接続が不要なためです。

BON-LOGでの使用箇所:

`lib/db.ts` はアプリ全体で共通のデータベース接続として使われます。

```
// lib/actions/post.ts (Server Action)
import { prisma } from '@lib/db'

export async function createPost(formData: FormData) {
  const post = await prisma.post.create({ ... })
}

// app/(main)/feed/page.tsx (Server Component)
import { prisma } from '@lib/db'

export default async function FeedPage() {
  const posts = await prisma.post.findMany({ ... })
  return <PostList posts={posts} />
}
```

実装しない場合の影響:

実装しない/誤った場合	発生する問題
PrismaPgアダプターを使わない (Prisma 6)	Prisma 6ではPostgreSQL接続にアダプターが必要なためエラーが発生。 Error: <code>PrismaClient is unable to connect to the database server.</code>
シングルトンパターンを使わない	開発環境のホットリロードのたびに新しいDB接続が作成され、「Too many connections」エラーが発生
SSLを本番環境で無効化	Supabase (本番DB) への接続が失敗する。本番は <code>{ rejectUnauthorized: false }</code> が必要
ダミーDB判定を省略	テスト実行時にPostgreSQL接続を試みてエラーになり、CIパイプラインが失敗する

使い方:

```
// アプリケーションのどこからでも同じインスタンスを使用
import { prisma } from '@lib/db'

// Server ComponentやServer Actionから直接呼び出せる
const users = await prisma.user.findMany()
// 返却値: User[] (Userオブジェクトの配列、0件なら空配列[])
```

7.7.2 CRUD操作

CRUD（クラッド）は、データベースの4つの基本操作の頭文字です。

操作	英語	Prismaメソッド	SQLの対応
作成	Create	<code>.create()</code>	<code>INSERT INTO ...</code>
読取	Read	<code>.findUnique()</code> 等	<code>SELECT ... FROM ...</code>
更新	Update	<code>.update()</code>	<code>UPDATE ... SET ...</code>
削除	Delete	<code>.delete()</code>	<code>DELETE FROM ...</code>

Create (作成)

```
// ===== Step 1: まず1件のユーザーを作成する =====
// data: に作成するデータの内容を指定する
const user = await prisma.user.create({
  data: {
    email: 'alice@example.com', // 必須フィールド
    nickname: 'Alice', // 必須フィールド
    password: hashedPassword, // bcryptでハッシュ化済みのパスワード
    // createdAt, updatedAt は @default(now()) と @updatedAt で自動設定
    // id は @default(cuid()) で自動生成
  },
})
console.log(user)
// 実行結果:
// {
//   id: 'cm1abc123def456gh789ij',
//   email: 'alice@example.com',
//   emailVerified: null,
//   password: '$2a$10$xK8v ... (ハッシュ値)',
//   nickname: 'Alice',
//   avatarUrl: null,
//   headerUrl: null,
//   bio: null,
//   location: null,
//   isPublic: true,
//   isPremium: false,
//   createdAt: 2025-04-15T09:30:00.000Z,
//   updatedAt: 2025-04-15T09:30:00.000Z
// }
// → id, createdAt, updatedAt, isPublic, isPremium は自動設定される
// → password, bio, location 等のオプションフィールドは指定しなければ null

// ===== Step 2: そのユーザーの投稿を作成する（ネスト作成） =====
// 投稿 + メディア + ジャンルを1回のクエリで一度に作成できる
const post = await prisma.post.create({
  data: {
    userId: user.id, // 投稿者のID (外部キー)
    content: '初めての投稿です', // 投稿テキスト
    media: {
      // create: で PostMedia レコードも同時に作成
      create: [
        { url: '/uploads/image1.jpg', type: 'image', sortOrder: 0 }, // 1枚目の画像
        { url: '/uploads/image2.jpg', type: 'image', sortOrder: 1 }, // 2枚目の画像
      ],
    },
    genres: {
      // create: で PostGenre レコード (中間テーブル) も同時に作成
      create: [
        { genreId: 'genre_松柏類' }, // ジャンル1
        { genreId: 'genre_雑木類' }, // ジャンル2
      ],
    },
  },
})
// include: で作成後のレスポンスにリレーション先のデータも含める
```



```

    include: {
      media: true, // 添付メディアも返す
      genres: { include: { genre: true } }, // ジャンル情報も返す
    },
  })
  console.log(post)
  // 実行結果:
  // {
  //   id: 'cm1xyz789abc012de345fg',
  //   userId: 'cm1abc123def456gh789ij',
  //   content: '初めての投稿です',
  //   quotePostId: null,
  //   repostPostId: null,
  //   createdAt: 2025-04-15T09:31:00.000Z,
  //   updatedAt: 2025-04-15T09:31:00.000Z,
  //   media: [
  //     { id: 'cm1med001 ... ', postId: 'cm1xyz789 ... ', url: '/uploads/image1.jpg', type: 'image', s
  //     { id: 'cm1med002 ... ', postId: 'cm1xyz789 ... ', url: '/uploads/image2.jpg', type: 'image', s
  //   ],
  //   genres: [
  //     { postId: 'cm1xyz789 ... ', genreId: 'genre_松柏類', genre: { id: 'genre_松柏類', name: '松柏類
  //     { postId: 'cm1xyz789 ... ', genreId: 'genre_雑木類', genre: { id: 'genre_雑木類', name: '雑木類
  //   ],
  // }

```

実行結果の確認方法 `npx prisma studio` を実行すると、ブラウザでデータベースの中身を確認できます。

<http://localhost:5555> でテーブル一覧が表示され、作成したユーザーや投稿のデータを目で確認できます。Prisma Studio上でレコードをクリックすると、リレーション先のデータ（メディア、ジャンル等）にもジャンプできます。

Read（読み取り）

Readは最も使用頻度の高い操作です。Prismaには用途に応じた複数の取得メソッドがあります。

Read系メソッドの使い分け：

```

findUnique → IDやユニークカラムで1件取得（見つからないとnull）
findFirst  → 条件に一致する最初の1件を取得（見つからないとnull）
findMany   → 条件に一致する複数件を取得（見つからないと空配列[]）
count      → 条件に一致するレコード数を取得

```

```
// ==== Step 1: IDで1件のユーザーを取得する ====
// findUnique は @id または @unique なカラムでのみ検索可能
const user = await prisma.user.findUnique({
  where: { id: 'cm1abc123def456gh789ij' }, // 主キーで検索
})
// 実行結果（見つかった場合）:
// {
//   id: 'cm1abc123def456gh789ij',
//   email: 'alice@example.com',
//   nickname: 'Alice',
//   avatarUrl: null,
//   bio: null,
//   isPublic: true,
//   createdAt: 2025-04-15T09:30:00.000Z,
//   updatedAt: 2025-04-15T09:30:00.000Z,
//   ... (Userモデルの全フィールド)
// }
//
// 実行結果（見つからなかった場合）:
// null

// ==== Step 2: ユニークカラムで検索する ====
const userByEmail = await prisma.user.findUnique({
  where: { email: 'alice@example.com' }, // @unique付きカラムでも検索可能
})
// 実行結果: 上記と同じUser型のオブジェクト、または null

// ==== Step 3: 条件付きで複数レコードを取得する ====
const posts = await prisma.post.findMany({
  where: {
    userId: 'cm1abc123def456gh789ij', // 特定ユーザーの投稿
    createdAt: {
      gte: new Date('2024-01-01'), // gte = Greater Than or Equal (以上)
    }, // つまり 2024年1月1日以降の投稿
  },
  orderBy: {
    createdAt: 'desc', // 新しい順に並べる (desc = 降順、asc = 昇順)
  },
  take: 20, // 最大20件のみ取得（ページネーション用）
})
// 実行結果（配列で返る）:
// [
//   {
//     id: 'cm1post003 ... ',
//     userId: 'cm1abc123 ... ',
//     content: '盆栽楽しい',
//     quotePostId: null,
//     repostPostId: null,
//     createdAt: 2025-04-16T14:00:00.000Z,
//     updatedAt: 2025-04-16T14:00:00.000Z,
//   },
//   {
//     id: 'cm1post001 ... ',
//     userId: 'cm1abc123 ... ',
//     content: '初めての投稿です',
//     ...

```

```
// },
// ]
// → createdAt の desc (降順) なので新しい投稿が先
// → 見つからない場合は空配列 [] が返る (nullではない)

// ===== Step 4: 最初の1件を取得する =====
// findUniqueと違い、ユニークでないカラムでも検索可能
const latestPost = await prisma.post.findFirst({
  where: { userId: 'cm1abc123def456gh789ij' }, // 条件
  orderBy: { createdAt: 'desc' },             // 新しい順に並べて
})                                              // 最初の1件 = 最新の投稿
// 実行結果: Post型のオブジェクト1件、または null
```

where条件の比較演算子	意味	例
<code>equals</code>	等しい (省略可能)	<code>{ userId: 'u1' }</code>
<code>not</code>	等しくない	<code>{ userId: { not: 'u1' } }</code>
<code>in</code>	いずれかに一致	<code>{ userId: { in: ['u1', 'u2'] } }</code>
<code>gt</code>	より大きい (Greater Than)	<code>{ createdAt: { gt: date } }</code>
<code>gte</code>	以上 (Greater Than or Equal)	<code>{ createdAt: { gte: date } }</code>
<code>lt</code>	より小さい (Less Than)	<code>{ createdAt: { lt: date } }</code>
<code>lte</code>	以下 (Less Than or Equal)	<code>{ createdAt: { lte: date } }</code>
<code>contains</code>	文字列を含む	<code>{ content: { contains: '盆栽' } }</code>
<code>startsWith</code>	文字列で始まる	<code>{ nickname: { startsWith: 'A' } }</code>

Update (更新)

```
// — 単一レコードの更新 —
// where: で更新対象を指定し、data: で変更内容を指定
const updatedUser = await prisma.user.update({
  where: { id: 'cm1abc123def456gh789ij' }, // この条件に一致するレコードを
  data: {
    nickname: 'Alice Updated',           // ニックネームを変更
    bio: '盆栽愛好家です',               // 自己紹介を追加
    // updatedAt は @updatedAt により自動的に現在時刻に更新される
  },
})
console.log(updatedUser)
// 実行結果 (更新後のレコード全体が返る):
// {
//   id: 'cm1abc123def456gh789ij',
//   email: 'alice@example.com',
//   nickname: 'Alice Updated',           ← 変更された
//   bio: '盆栽愛好家です',              ← 変更された (null → 値あり)
//   isPublic: true,                     ← 変更していないフィールドはそのまま
//   updatedAt: 2025-04-15T10:00:00.000Z, ← 自動更新された
//   ...
// }

// — 複数レコードの一括更新 —
// updateMany は条件に一致する全レコードを一度に更新
const result = await prisma.post.updateMany({
  where: {
    userId: 'cm1abc123def456gh789ij',    // このユーザーの投稿で
    createdAt: {
      lt: new Date('2024-01-01'),         // 2024年より前のもの
    },
  },
  data: {
    // 更新する内容を指定
    // 例: 古い投稿を非公開に (isPublicカラムがある場合)
  },
})
console.log(result)
// 実行結果 (更新件数のみ返る):
// { count: 5 }
console.log(`${result.count}件更新しました`)
```

updateとupdateManyの違い: `update` は1件のみ更新し、更新後のレコード全体を返します。`updateMany` は条件に一致する全レコードを更新し、更新件数のみを返します (`{ count: 5 }` のような形式)。

Delete (削除)

```
// — 単一レコードの削除 —
// where: で削除対象を指定
const deletedPost = await prisma.post.delete({
  where: { id: 'cm1post001abc ... ' }, // この投稿を削除
})
// 実行結果（削除されたレコードが返る）:
// {
//   id: 'cm1post001abc ... ',
//   userId: 'cm1abc123 ... ',
//   content: '初めての投稿です',
//   createdAt: 2025-04-15T09:31:00.000Z,
//   ...
// }
// ※ onDelete: Cascade が設定されているので、
//   この投稿のメディア、コメント、いいね、ブックマークも自動削除される

// — 複数レコードの一括削除 —
const result = await prisma.comment.deleteMany({
  where: {
    postId: 'cm1post002abc ... ', // この投稿へのコメントを全削除
  },
})
console.log(result)
// 実行結果（削除件数のみ返る）:
// { count: 3 }
console.log(`${result.count}件削除しました`)
```

注意: `delete` は指定した条件に一致するレコードが存在しない場合、エラーをスローします。レコードが存在しない可能性がある場合は、先に `findUnique` で存在確認するか、`deleteMany` を使いましょう（`deleteMany` は0件削除でもエラーになりません）。

CRUD操作の全体像（まとめ図）

CRUD操作をレストランの例えで理解する:

操作	レストランの例え	メソッド	説明
Create (作成)	新しい料理をメニューに追加する	<code>prisma.user.create({ data: { ... } })</code>	<code>INSERT INTO users (...) VALUES (...)</code> で新しいレコードが1件追加される
Read (読取)	メニューから料理を探す	<code>findUnique</code>	「この料理番号の料理を1つ持ってきて」
		<code>findFirst</code>	「最初に見つかった和食を1つ持ってきて」
		<code>findMany</code>	「和食メニューを全部持ってきて」
		<code>count</code>	「和食メニューはいくつある？」
Update (更新)	メニューの料理名や値段を変更する	<code>update</code>	「この1品の名前を変えて」
		<code>updateMany</code>	「全てのデザートの日曜日を10%オフにして」
Delete (削除)	メニューから料理を除外する	<code>delete</code>	「この1品を削除して」
		<code>deleteMany</code>	「売り切れの料理を全部削除して」

初心者向けポイント: CRUDの4操作はすべてのWebアプリケーションの基本です。SNSでは「投稿を作成する (Create)」「タイムラインを表示する (Read)」「プロフィールを編集する (Update)」「投稿を削除する (Delete)」と、すべての操作がCRUDに対応しています。この4つを理解すれば、どんなアプリでもデータ操作の基本はカバーできます。

7.7.3 include と select

`include` と `select` は、データ取得時に「どこまでのデータを含めるか」を制御するオプションです。

`include` と `select` の使い分け:

include: リレーション先のデータも「追加で」取得する
→ 元のフィールド全部 + 指定したリレーション

select: 指定したフィールド「だけ」を取得する
→ 必要なフィールドのみ (不要なデータを除外)

※ `include` と `select` は同時に使えません (どちらか一方のみ)

include: リレーション先のデータも取得

```
// ===== BON-LOG実例: 投稿詳細画面のデータ取得 =====
const post = await prisma.post.findUnique({
  where: { id: 'cm1xyz789abc012de345fg' },
  include: {
    user: true,    // 投稿者のUser情報も取得（全フィールド）
    media: true,   // 添付メディアも取得（全フィールド）
    _count: {      // _count は特別なフィールド: リレーション先のレコード数を取得
      select: {
        likes: true,    // いいね数
        comments: true, // コメント数
      },
    },
  },
})
console.log(JSON.stringify(post, null, 2))
// 実行結果（リレーション先のデータがネストされたオブジェクトとして返る）:
// {
//   "id": "cm1xyz789abc012de345fg",
//   "userId": "cm1abc123def456gh789ij",
//   "content": "初めての投稿です",
//   "quotePostId": null,
//   "repostPostId": null,
//   "createdAt": "2025-04-15T09:31:00.000Z",
//   "updatedAt": "2025-04-15T09:31:00.000Z",
//   "user": {                                ← include: { user: true } で追加
//     "id": "cm1abc123def456gh789ij",
//     "email": "alice@example.com",
//     "nickname": "Alice",
//     "avatarUrl": null,
//     "bio": null,
//     "isPublic": true,
//     ...
//   },
//   "media": [                                ← include: { media: true } で追加
//     {
//       "id": "cm1med001 ... ",
//       "postId": "cm1xyz789 ... ",
//       "url": "/uploads/image1.jpg",
//       "type": "image",
//       "sortOrder": 0
//     },
//     {
//       "id": "cm1med002 ... ",
//       "postId": "cm1xyz789 ... ",
//       "url": "/uploads/image2.jpg",
//       "type": "image",
//       "sortOrder": 1
//     }
//   ],
//   "_count": {                                ← _count で追加
//     "likes": 12,
//     "comments": 3
//   }
// }
```

```
// }

// 取得結果のアクセス方法:
// post.content          → '初めての投稿です'
// post.user.nickname    → 'Alice' (includeで追加されたデータ)
// post.media[0].url      → '/uploads/image1.jpg' (includeで追加されたデータ)
// post._count.likes     → 12 (_countで取得した集計値)
// post._count.comments  → 3
```

実行結果の確認方法 `npx prisma studio` でテーブルを開き、レコードをクリックすると、リレーション先のデータも確認できます。例えば posts テーブルのレコードをクリックすると、user, media, likes, comments へのリンクが表示されます。

select: 必要なフィールドのみ取得

```
// パスワードなどの機密情報を除外してユーザー情報を取得
const user = await prisma.user.findUnique({
  where: { id: 'cm1abc123def456gh789ij' },
  select: {
    id: true,          // ✅ IDは取得
    nickname: true,    // ✅ ニックネームは取得
    avatarUrl: true,   // ✅ アバター画像URLは取得
    // password → selectに含めない = 取得されない (セキュリティ上安全)
    // email → selectに含めない = 取得されない
  },
})
// 実行結果 (指定したフィールドのみ返る):
// { id: 'cm1abc123def456gh789ij', nickname: 'Alice', avatarUrl: null }
//
// user の型は { id: string; nickname: string; avatarUrl: string | null }
// password や email にはアクセスできない (型エラーになる)
```


ネストしたselect

```
// リレーション先のデータも、必要なフィールドだけを選択できる
const posts = await prisma.post.findMany({
  select: {
    id: true,           // 投稿ID
    content: true,      // 投稿テキスト
    createdAt: true,    // 作成日時
    user: {             // 投稿者情報（ネストしたselect）
      select: {
        id: true,       // ユーザーID
        nickname: true, // ニックネーム
        avatarUrl: true, // アバター画像（表示に必要な情報のみ）
        // password, email などは取得しない
      },
    },
    _count: {           // リレーション先のカウント
      select: {
        likes: true,    // いいね数
        comments: true, // コメント数
      },
    },
  },
})

// 実行結果（必要なフィールドだけが返る、軽量のレスポンス）:
// [
//   {
//     id: 'cm1post003...',
//     content: '盆栽楽しい',
//     createdAt: 2025-04-16T14:00:00.000Z,
//     user: { id: 'cm1abc123...', nickname: 'Alice', avatarUrl: null },
//     _count: { likes: 5, comments: 2 }
//   },
//   {
//     id: 'cm1post001...',
//     content: '初めての投稿です',
//     createdAt: 2025-04-15T09:31:00.000Z,
//     user: { id: 'cm1abc123...', nickname: 'Alice', avatarUrl: null },
//     _count: { likes: 12, comments: 3 }
//   },
// ]
```

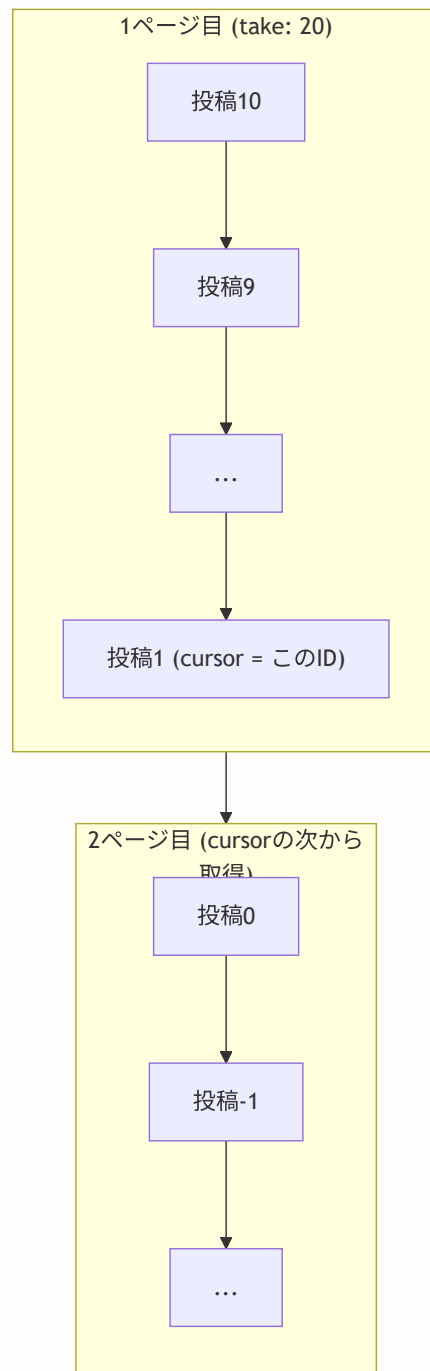
パフォーマンスのヒント: `select` を使って必要なフィールドのみ取得すると、データベースからの転送量が減り、パフォーマンスが向上します。特にユーザー一覧のような多数のレコードを取得する場面では効果的です。

7.7.4 ページネーション（カーソルベース）

カーソルベースのページネーションは無限スクロールに最適です。SNSのタイムラインのように「もっと読む」「スクロールして次を表示」という操作で使います。

2つのページネーション方式の比較:

- **オフセットベース（従来型: ページ番号方式）**：「2ページ目」＝最初の20件をスキップして次の20件を取得。問題: 新しい投稿が追加されるとズレが生じる
- **カーソルベース（SNS向き: 「ここから先」方式）** ← BON-LOGで採用: 「この投稿IDの次」から20件を取得。新しい投稿が追加されてもズレない



```

// カーソルベースページネーションの実装
export async function getPosts(cursor?: string, limit = 20) {
  const posts = await prisma.post.findMany({
    // limit + 1件取得する理由:
    // 実際に必要な件数より1件多く取得し、
    // 「まだ次のページがあるか？」を判定する
    take: limit + 1,

    // カーソルが指定されている場合 (2ページ目以降)
    ...(cursor && {
      cursor: { id: cursor }, // このIDの位置から取得開始
      skip: 1,                // カーソル自体は前のページで表示済みなのでスキップ
    }),

    orderBy: { createdAt: 'desc' }, // 新しい投稿順
    include: {
      user: {
        select: {
          id: true,
          nickname: true,
          avatarUrl: true,
        },
      },
      media: {
        orderBy: { sortOrder: 'asc' }, // メディアは表示順序順
      },
      _count: {
        select: {
          likes: true,
          comments: true,
        },
      },
    },
  })

  // limit + 1件取得できた = まだ次のページがある
  const hasMore = posts.length > limit
  // hasMoreの場合、余分に取得した1件を除外する
  const data = hasMore ? posts.slice(0, -1) : posts
  // 次のページのカーソル = 最後の投稿のID
  const nextCursor = hasMore ? data[data.length - 1]?.id : undefined

  return { posts: data, nextCursor, hasMore }
}

```

使い方:

```
// ==== Step 1: 初回取得（カーソルなし = 最新の投稿から） ====
const page1 = await getPosts()
console.log(page1)
// 実行結果:
// {
//   posts: [
//     { id: 'cm1post100 ... ', content: '最新の投稿', user: { nickname: 'Alice', ... }, ... },
//     { id: 'cm1post099 ... ', content: '2番目の投稿', user: { nickname: 'Bob', ... }, ... },
//     ... (計20件)
//     { id: 'cm1post081 ... ', content: '20番目の投稿', user: { nickname: 'Charlie', ... }, ... },
//   ],
//   nextCursor: 'cm1post081 ... ',    ← 20件目の投稿のID
//   hasMore: true                    ← まだ投稿がある
// }

// ==== Step 2: 次のページを取得（前回の最後のIDをカーソルとして渡す） ====
const page2 = await getPosts(page1.nextCursor)
// 実行結果:
// {
//   posts: [ ... (21~40件目の投稿、計20件) ],
//   nextCursor: 'cm1post061 ... ',
//   hasMore: true
// }

// ==== Step 3: 最後のページ ====
const pageLast = await getPosts(someLastCursor)
// 実行結果:
// {
//   posts: [ ... (残り8件のみ) ],
//   nextCursor: undefined,    ← もう次のページがない
//   hasMore: false            ← これが最後のページ
// }
```

7.7.5 トランザクション

複数の操作を原子的（アトミック）に実行します。「全て成功するか、全て失敗するか」のどちらかになり、中途半端な状態を防ぎます。

トランザクション（Transaction）とは？: 英語で「取引」を意味する言葉です。銀行で「振込」という1つの取引が「引き落とし」と「入金」の2つの操作から成り立つように、データベースでも複数の操作を1つの「取引」としてまとめて扱う仕組みです。

トランザクションのたとえ: 銀行の振込を想像してください。「Aさんの口座から1万円引く」と「Bさんの口座に1万円足す」は、必ず両方成功するか、両方失敗する必要があります。もしAさんの口座から引いたのにBさんに足されなかったら、1万円が消えてしまいます。トランザクションはこのような「一緒に成功・失敗すべき操作のまとまり」を保証します。

トランザクションの4つの保証（ACID特性）：

A - Atomicity（原子性）

→ 全ての操作が成功するか、全てが取り消されるか。中途半端な状態にならない。

例：いいね追加 + 通知作成 → 両方成功 or 両方なかったことに

C - Consistency（一貫性）

→ トランザクション前後でデータの整合性が保たれる。

例：ユニーク制約に違反するデータは挿入されない

I - Isolation（分離性）

→ 同時に実行される複数のトランザクションが互いに干渉しない。

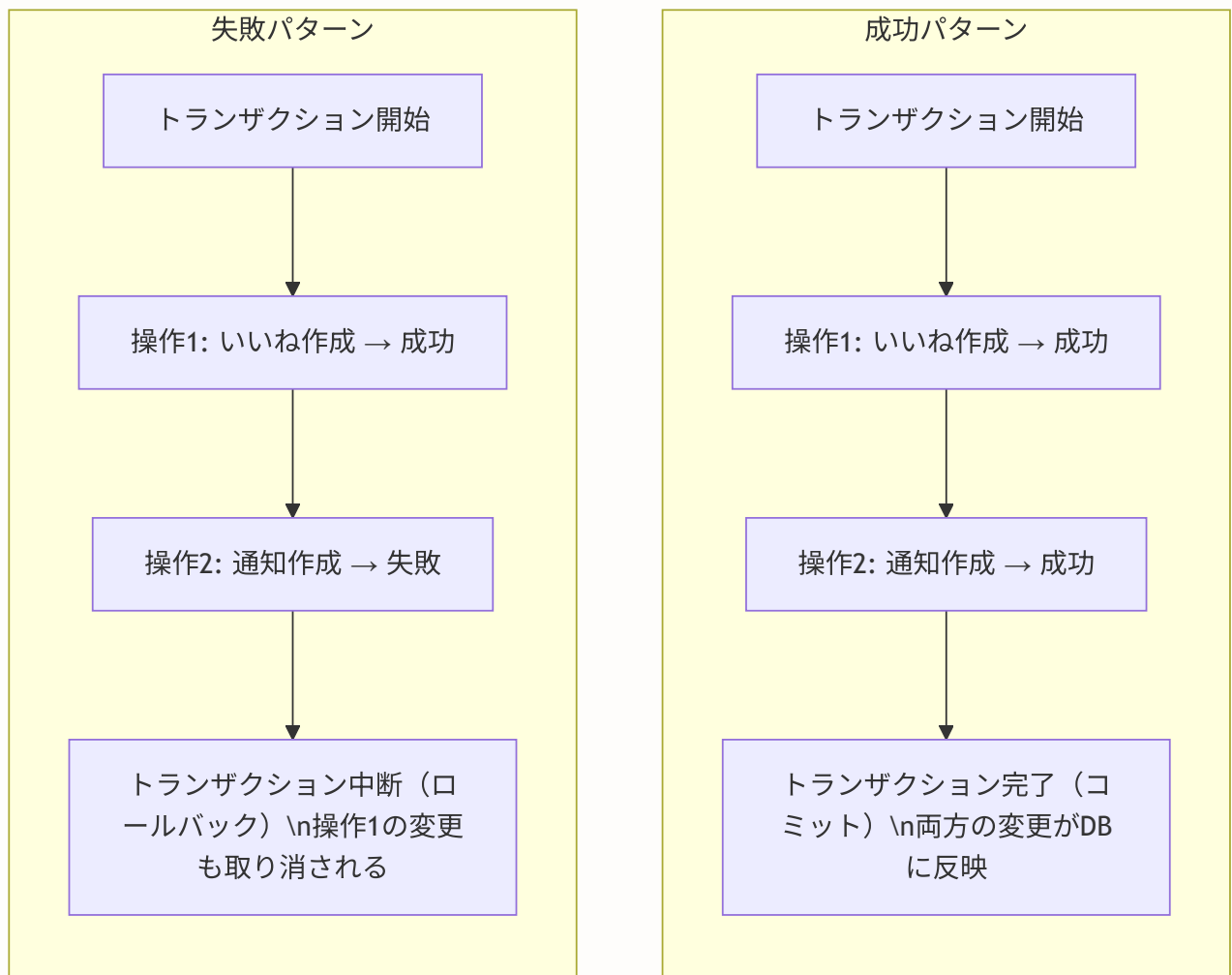
例：AliceとBobが同時に同じ投稿にいいねしても正しくカウントされる

D - Durability（永続性）

→ 完了したトランザクションの結果は、システム障害が起きても失われない。

例：「いいね完了」と表示された後にサーバーが落ちても、いいねは保存されている

初心者向けポイント: ACIDは覚えなくても大丈夫ですが、「トランザクション = 複数の操作を安全にまとめて実行する仕組み」ということは覚えておいてください。BON-LOGでは「いいね + 通知」「フォロー + 通知」のようにセットで行うべき操作に使用しています。



```
// ==== BON-LOG実例: いいねボタンを押した時の処理 ====
// トグル: 押すたびにON/OFFが切り替わる
export async function toggleLike(postId: string, userId: string) {
  // Step 1: 既にいいいねしているか確認
  const existingLike = await prisma.like.findUnique({
    where: {
      userId_postId: { userId, postId }, // 複合ユニークキーで検索
    },
  })
  // 実行結果: Like | null
  // いいいね済み → { id: 'cm1like001...', userId: 'u1', postId: 'p1', ... }
  // 未いいいね → null

  if (existingLike) {
    // いいいね削除
    await prisma.$transaction([
      prisma.like.delete({
        where: { id: existingLike.id },
      }),
      // 通知も削除 (通知機能がある場合)
      prisma.notification.deleteMany({
        where: {
          type: 'like',
          postId,
          actorId: userId,
        },
      }),
    ])
  } else {
    // いいいね追加
    const post = await prisma.post.findUnique({
      where: { id: postId },
      select: { userId: true },
    })

    await prisma.$transaction([
      prisma.like.create({
        data: { userId, postId },
      }),
      // 自分の投稿でなければ通知を作成
      ... (post && post.userId !== userId
        ? [
            prisma.notification.create({
              data: {
                userId: post.userId,
                actorId: userId,
                postId,
                type: 'like',
              },
            }),
          ]
        : []),
    ])
  }
}
```

```

    }
  }
}

```

7.7.6 集計クエリ

データの件数や統計情報を取得するためのクエリです。

```

// — count: レコード数を取得 —
// ユーザーの投稿数を取得
const postCount = await prisma.post.count({
  where: { userId: 'cm1abc123def456gh789ij' }, // このユーザーの投稿を数える
})
// 実行結果: 42 (数値が直接返る)
// → このユーザーは42件投稿している

// — 特定投稿のいいね数 —
const likeCount = await prisma.like.count({
  where: { postId: 'cm1xyz789abc012de345fg' }, // この投稿へのいいねを数える
})
// 実行結果: 15 (数値が直接返る)

// — aggregate: 統計情報を一括取得 —
const stats = await prisma.post.aggregate({
  _count: { id: true }, // 総投稿数
  _max: { createdAt: true }, // 最新の投稿日時
  _min: { createdAt: true }, // 最古の投稿日時
})
console.log(stats)
// 実行結果:
// {
//   _count: { id: 1234 },
//   _max: { createdAt: 2025-04-15T10:30:00.000Z },
//   _min: { createdAt: 2025-01-01T00:00:00.000Z }
// }
// stats._count.id → 1234 (全投稿数)
// stats._max.createdAt → 最新投稿の日時
// stats._min.createdAt → 最初の投稿の日時

```

▶ 理解度チェック: Prismaクライアントの使い方

7.8 マイグレーション管理

マイグレーションとは？: データベースの「リフォーム工事記録」のようなものです。家を建てた後に「部屋を追加したい」「壁を取り壊したい」と思ったとき、工事の記録を残しておくことで、同じ家を別の場所に建てるときにも同じ手順で再現できます。データベースのマイグレーションも同様に、テーブル構造の変更内容をファイルとして記録し、どの環境でも同じ状態を再現できるようにする仕組みです。

マイグレーションの役割:

Gitがソースコードの変更履歴を管理するように、マイグレーションはデータベース構造の変更履歴を管理します。

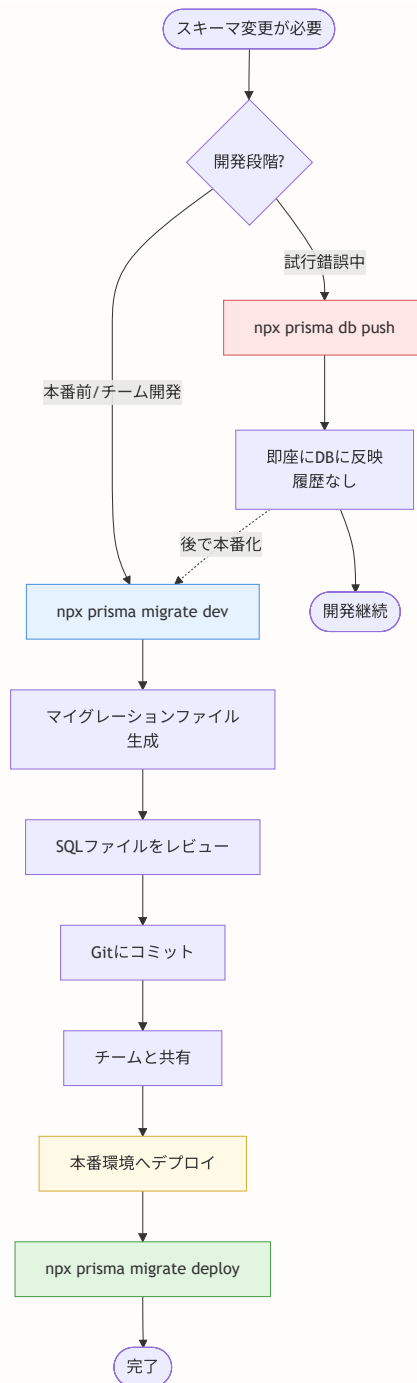
Git	マイグレーション
commit 1: 初期コード	migration 1: 初期テーブル
commit 2: 機能追加	migration 2: カラム追加
commit 3: バグ修正	migration 3: インデックス

どちらも「いつ、何を変更したか」の記録を保持し、過去の状態を再現できます。

Prismaコマンドの使い分け

- 開発中: `prisma db push` (手軽だが履歴なし)
- 本番環境: `prisma migrate deploy` (履歴付き、安全)

####マイグレーションワークフロー図



ワークフローの読み方:

1. **開発段階:** `db push` で素早く 試行錯誤
2. **本番前:** `migrate dev` で履歴を残す
3. **チーム共有:** SQLファイルをGitで管理
4. **本番デプロイ:** `migrate deploy` で安全に適用

db push vs migrate dev の使い分け:

コマンド	<code>npx prisma db push</code>	<code>npx prisma migrate dev</code>
用途	開発時の試行錯誤向き	本番・チーム開発向き
フロー	schema.prisma を編集 → 即座にDBに反映	schema.prisma を編集 → SQL ファイルを生成 → DBに反映
マイグレーション ファイル	作成されない	<code>prisma/migrations/</code> に作成される
メリット	・ 素早くスキーマを試せる	・ 変更履歴が残る（Gitで管理可能） ・ チームメンバーが同じマイグレーションを適用できる ・ ロールバック（巻き戻し）の参考になる
デメリット	・ 変更履歴が残らない ・ データが消える可能性がある（テーブル再作成時）	-
適した場面	1人で開発中に「ちょっとカラム追加してみよう」という場面	チーム開発、本番デプロイ前の最終確認

7.8.1 開発環境でのワークフロー

```
# スキーマを編集したら、データベースに即座に反映
npx prisma db push

# Prismaクライアントを再生成
npx prisma generate
```

実行結果の例（db push でカラムを追加した場合）：

```
$ npx prisma db push
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "bonsai_sns", schema "public"
  at "localhost:5432"

Your database is now in sync with your Prisma schema. Done in 0.89s

✓ Generated Prisma Client (v6.2.1) to ./node_modules/@prisma/client in 178ms
```

`db push` は開発中に素早くスキーマを試す際に便利ですが、マイグレーション履歴は残りません。

7.8.2 本番環境でのワークフロー

```
# マイグレーションファイルを作成
npx prisma migrate dev --name add_premium_field

# これにより以下が実行されます:
# 1. prisma/migrations/ にSQLファイルが作成される
# 2. データベースにマイグレーションが適用される
# 3. Prismaクライアントが再生成される
```

実行結果の例:

```
$ npx prisma migrate dev --name add_premium_field
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "bonsai_sns", schema "public"
  at "localhost:5432"

Applying migration `20250415_add_premium_field`

The following migration(s) have been created and applied from new schema changes:

migrations/
├─ 20250415093000_add_premium_field/
└─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (v6.2.1) to ./node_modules/@prisma/client in 201ms
```

`prisma/migrations/` フォルダに生成されたSQLファイルはGitにコミットして、チームメンバーと共有します。

7.8.3 本番デプロイ

```
# CI/CDパイプラインやデプロイスクリプトで実行
npx prisma migrate deploy
```

これは未適用のマイグレーションのみを実行します。

7.8.4 BON-LOGの実際のマイグレーション履歴

BON-LOGプロジェクトの `prisma/migrations/` ディレクトリには、以下のマイグレーションファイルが存在します。これらはプロジェクトの進化の歴史そのものです。

```
prisma/migrations/
├─ 0_init/                                # 初期スキーマ（全テーブル一括作成）
│   └─ migration.sql
├─ 20240201000000_add_fts_indexes/        # 全文検索インデックスの追加
│   └─ migration.sql
├─ 20260127_add_birth_date/              # Userに生年月日カラムを追加
│   └─ migration.sql
├─ 20260128_add_system_settings/         # SystemSetting + ブラックリスト + UserDevice
│   └─ migration.sql
├─ 20260129_add_notification_preferences/ # Userに通知設定JSONカラムを追加
│   └─ migration.sql
├─ 20260129_enable_rls/                  # テーブルにRow Level Securityを有効化
│   └─ migration.sql
├─ 20260203_add_polls/                   # 投票機能（Poll/PollOption/PollVote）
│   └─ migration.sql
├─ 20260208_add_scheduled_posts_composite_index/ # 予約投稿のCronジョブ用複合インデックス
│   └─ migration.sql
├─ 20260222_add_enum_types/              # String型をEnum型に一括変換
│   └─ migration.sql
├─ fulltext_search_indexes.sql           # 全文検索インデックス定義（手動適用）
└─ migration_lock.toml                  # Prismaのマイグレーションロック
```

各マイグレーションの詳細:

マイグレーション	内容	影響テーブル
<code>0_init</code>	初期スキーマ。全テーブルとインデックスを一括作成。CREATE TABLE文が約50個。 ScheduledPostStatus enumも定義	全テーブル
<code>add_fts_indexes</code>	pg_trgm拡張を有効化し、posts.content、 users.nickname、users.bio、 bonsai_shops.name/address、 events.title/description、 bonsais.name/species/description、 hashtags.name、comments.content にGINインデックスを作成	posts, users, bonsai_shops, events, bonsais, hashtags, comments
<code>add_birth_date</code>	<code>ALTER TABLE "users" ADD COLUMN "birth_date" DATE;</code> ユーザーの生年月日を追加	users
<code>add_system_settings</code>	SystemSetting、EmailBlacklist、DeviceBlacklist、 UserDeviceテーブルを作成。管理機能とセキュリティ強化	新規4テーブル
<code>add_notification_preferences</code>	<code>ALTER TABLE "users" ADD COLUMN "notification_preferences" JSONB DEFAULT '{}';</code> 通知設定をJSON型で追加	users
<code>enable_rls</code>	全50テーブルに <code>ENABLE ROW LEVEL SECURITY</code> を適用。Supabaseのセキュリティレイヤー	全テーブル
<code>add_polls</code>	投票機能のPoll、PollOption、PollVoteテーブルを作成	新規3テーブル
<code>add_scheduled_posts_composite_index</code>	<code>CREATE INDEX "scheduled_posts_status_scheduled_at_idx"</code> Cronジョブの検索を最適化する複合インデックス	scheduled_posts
<code>add_enum_types</code>	MediaType、NotificationType、ReportTargetType 等10個のEnum型を作成し、既存のString型カラム をEnum型に変換。VARCHAR制約も追加	多数のテーブル

マイグレーションの読み方（0_init の例）：

```
-- prisma/migrations/0_init/migration.sql (抜粋)

-- Enum型の定義 (PostgreSQLのカスタム型)
CREATE TYPE "ScheduledPostStatus" AS ENUM ('pending', 'published', 'failed', 'cancelled');

-- テーブル作成 (CREATE TABLE)
CREATE TABLE "users" (
  "id" TEXT NOT NULL,           -- 主キー (cuid形式)
  "email" TEXT NOT NULL,       -- ユニーク制約付き
  "nickname" TEXT NOT NULL,
  "created_at" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  -- ... (他のカラム)
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")
);

-- ユニーク制約 (重複を許さない)
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");

-- 外部キーとインデックス
CREATE TABLE "posts" (
  "id" TEXT NOT NULL,
  "user_id" TEXT NOT NULL,     -- usersテーブルへの外部キー
  -- ...
  CONSTRAINT "posts_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "posts_user_id_idx" ON "posts"("user_id");

-- 外部キー制約の追加
ALTER TABLE "posts" ADD CONSTRAINT "posts_user_id_fkey"
  FOREIGN KEY ("user_id") REFERENCES "users"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

初心者の方へ: マイグレーションファイルの中身はSQLです。Prismaが `schema.prisma` を読んで自動生成してくれるので、手動でSQLを書く必要はありません。ただし、中身を読めるようになると「何が行われるか」を事前に確認でき、トラブルシューティングにも役立ちます。

Row Level Security (RLS) マイグレーションの例:

```
-- prisma/migrations/20260129_enable_rls/migration.sql

-- 全テーブルにRLSを有効化 (Supabaseのセキュリティ機能)
ALTER TABLE public.users ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.comments ENABLE ROW LEVEL SECURITY;
-- ... 全50テーブルに対して適用 ...
```

RLS (Row Level Security) とは？: 「このユーザーはこの行だけ見てもいい」というアクセス制御をデータベースレベルで行う機能です。Supabaseではアプリケーション層だけでなく、DB層でもセキュリティを確保するために使用します。PrismaはDBオーナーとして接続するため、RLSによるブロックは受けませんが、Supabaseのダッシュボードから直接SQLを実行する場合に保護層として機能します。

Enum型移行マイグレーションの例:

```
-- prisma/migrations/20260222_add_enum_types/migration.sql (抜粋)

-- 1. Enum型を定義
CREATE TYPE "MediaType" AS ENUM ('image', 'video');
CREATE TYPE "NotificationType" AS ENUM ('like', 'comment', 'follow', 'quote', 'reply', ...);

-- 2. 既存カラムをString型からEnum型に変換
ALTER TABLE "post_media" ALTER COLUMN "type"
  TYPE "MediaType" USING "type"::"MediaType";

-- 3. デフォルト値のあるカラムは一度削除して再設定
ALTER TABLE "scheduled_post_media" ALTER COLUMN "type" DROP DEFAULT;
ALTER TABLE "scheduled_post_media" ALTER COLUMN "type"
  TYPE "MediaType" USING "type"::"MediaType";
ALTER TABLE "scheduled_post_media" ALTER COLUMN "type"
  SET DEFAULT 'image'::"MediaType";
```

String型からEnum型への移行: プロジェクト初期は柔軟性のためにString型で定義し、モデルが安定してきたタイミングでEnum型に移行するのは実践的なアプローチです。Enum型にすることで、不正な値の挿入を防ぎ、型安全性が向上します。

7.9 Prisma Studio (GUI)

Prisma Studioは、データベースをブラウザで可視化・編集できるツールです。

```
# Prisma Studioを起動
npx prisma studio
```

実行結果:

```
$ npx prisma studio
Prisma Studio is up on http://localhost:5555
```


ブラウザで `http://localhost:5555` が開き、以下が可能になります:

- テーブルのデータを一覧表示 (Excelのような表形式で表示される)
- レコードの作成・編集・削除 (「Add record」ボタンで新規追加、セルをクリックで編集)
- リレーション先への移動 (外部キーのリンクをクリックすると関連テーブルにジャンプ)
- データのフィルタリング・ソート (カラムヘッダーをクリックでソート、フィルタ欄で条件指定)

実行結果の確認方法 Prisma Studioは、本章で学んだCRUD操作の結果を目で確認するのに最適なツールです。コードでデータを作成・更新した後に Prisma Studio を開くと、データが正しく保存されているか確認できます。特にリレーション (user → posts → likes のような紐付け) が正しく設定されているかの確認に便利です。

開発中のデバッグやテストデータの投入に便利です。

7.10 実践演習

演習1: 投稿の取得 (基礎)

以下の条件で投稿を取得するServer Actionを作成してください。

要件:

- 特定ユーザーの投稿を取得
- 新しい順に並べる
- 投稿者情報 (id, nickname, avatarUrl) も含める
- メディアも含める
- いいね数とコメント数も取得

```
// lib/actions/post.ts
'use server'

import { prisma } from '@lib/db'

export async function getUserPosts(userId: string) {
  // ここにコードを書いてください
  const posts = await prisma.post.findMany({
    // ...
  })

  return posts
}
```

▶ 解答例

演習2: フォロー機能（トランザクション）

ユーザーをフォロー/アンフォローする関数を作成してください。

要件:

- 既にフォロー中ならアンフォロー、未フォローならフォロー
- フォロー時は通知も作成（自分自身のフォローは通知不要）
- トランザクションを使って原子性を保証

```
// lib/actions/follow.ts
'use server'

import { prisma } from '@lib/db'
import { auth } from '@lib/auth'

export async function toggleFollow(targetUserId: string) {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  const followerId = session.user.id

  // 自分自身はフォローできない
  if (followerId === targetUserId) {
    return { error: '自分自身をフォローできません' }
  }

  // ここにコードを書いてください
}
```

▶ 解答例

演習3: タイムラインの取得（応用）

フォロー中のユーザーの投稿を取得するタイムライン機能を実装してください。

要件:

- 自分の投稿 + フォロー中のユーザーの投稿を取得
- カーソルベースのページネーション対応
- 引用投稿・リポスの元投稿も含める

- いいね済み・ブックマーク済みフラグも含める

```
// lib/actions/feed.ts
'use server'

import { prisma } from '@lib/db'
import { auth } from '@lib/auth'

export async function getFeed(cursor?: string, limit = 20) {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  const userId = session.user.id

  // ここにコードを書いてください
  // ヒント:
  // 1. フォロー中のユーザーIDを取得
  // 2. where条件で userId in [自分, ... フォロー中のユーザー]
  // 3. カーソルベースページネーション
  // 4. いいね・ブックマーク済みをSELECTで判定
}
```

▶ 解答例

7.11 追加モデル詳細

このセクションで学ぶこと

- 7.6で紹介しきれなかったモデルの設計意図
- Poll/PollOption/PollVote（投票機能）の仕組み
- Hashtag（ハッシュタグ）のカウント管理
- UserAnalytics（ユーザー分析）の日別集計設計
- SystemSetting（システム設定）のキーバリュー設計
- ブラックリスト系・下書き・予約投稿モデル
- NextAuth.js認証用モデル（Account, Session等）
- CommentMedia、ShopChangeRequest、お問い合わせ等のサポートモデル

7.6節では主要なモデル（User, Post, Follow, Like, Comment, Bookmark, Block, Mute, Notification, BonsaiShop, Event, Report, AdminUser, Conversation/Message, Payment, Bonsai等）を解説しましたが、BON-LOGにはさらに多くのモデルが存在します。ここでは、残りの重要なモデルをカテゴリ別に解説します。

7.11.1 投票機能（Poll / PollOption / PollVote）

投稿にアンケート（投票）を付けられる機能を実現する3つのモデルです。X（旧Twitter）の投票機能と同じ仕組みです。

```
// prisma/schema.prisma より抜粋

// アンケート/投票
model Poll {
  id          String      @id @default(cuid())
  postId      String      @unique @map("post_id") // 1つの投稿に1つだけ投票
  duration    Int          // 投票期間 (秒数)
  expiresAt   DateTime     @map("expires_at")      // 投票期限
  createdAt   DateTime     @default(now()) @map("created_at")

  post        Post         @relation(fields: [postId], references: [id], onDelete: Cascade)
  options     PollOption[] // 選択肢一覧
  votes       PollVote[]   // 投票一覧

  @@map("polls")
}

model PollOption {
  id          String      @id @default(cuid())
  pollId      String      @map("poll_id")
  text        String      @db.VarChar(50) // 選択肢テキスト (最大50文字)
  sortOrder   Int         @default(0) @map("sort_order") // 表示順序

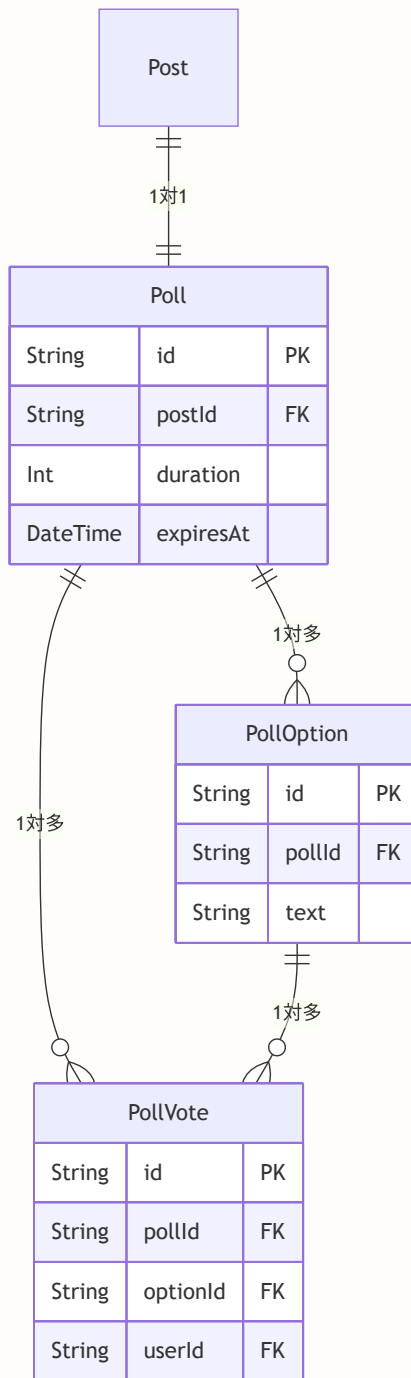
  poll        Poll         @relation(fields: [pollId], references: [id], onDelete: Cascade)
  votes       PollVote[] // この選択肢への投票一覧

  @@index([pollId])
  @@map("poll_options")
}

model PollVote {
  id          String      @id @default(cuid())
  pollId      String      @map("poll_id")
  optionId    String      @map("option_id")
  userId      String      @map("user_id")
  createdAt   DateTime     @default(now()) @map("created_at")

  poll        Poll         @relation(fields: [pollId], references: [id], onDelete: Cascade)
  option      PollOption   @relation(fields: [optionId], references: [id], onDelete: Cascade)

  @@unique([pollId, userId]) // 1つの投票に1人1回だけ投票可能
  @@index([pollId])
  @@map("poll_votes")
}
```



`@@unique([pollId, userId])` により、同じユーザーは同じ投票に1回のみ参加可能。

例: 「好きな松柏の樹種は？」

要素	内容
Post	好きな松柏は？
Poll	duration: 86400秒（24時間の投票）
Option1	黒松（5票）
Option2	五葉松（3票）
Option3	真柏（7票）

設計ポイント:

- `@@unique([pollId, userId])`: データベースレベルで「1人1票」を保証。アプリケーションコードにバグがあっても重複投票を防げます
- `postId` の `@unique`: 1つの投稿に対して作成できる投票は1つだけ
- `duration`（秒数）と `expiresAt`（期限日時）の両方を持つことで、投票期間の表示と期限判定の両方に対応
- `@db.VarChar(50)`: 選択肢テキストの長さをデータベースレベルで制限

投票機能のCRUD操作

投票機能は「投稿作成」「投票」「結果取得」の3つの操作に分けられます。それぞれの具体的なPrismaコードを見ていきましょう。

投票付き投稿の作成:

```

// lib/actions/poll.ts

// 投票付き投稿を作成するServer Action
export async function createPostWithPoll(formData: FormData) {
  // ① 認証チェック (投稿者を特定するため必須)
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // ② フォームからデータを取得
  const content = formData.get('content') as string // 投稿本文
  const options = formData.getAll('options') as string[] // 選択肢一覧
  const duration = parseInt(formData.get('duration') as string) // 投票期間 (秒)

  // ③ バリデーション
  // options: 選択肢は最低2つ、最大4つ
  if (options.length < 2 || options.length > 4) {
    return { error: '選択肢は2~4個で指定してください' }
  }

  // ④ 投票期限を計算
  // duration は秒数なので、現在時刻に秒数を加算して期限日時を求める
  const expiresAt = new Date()
  expiresAt.setSeconds(expiresAt.getSeconds() + duration)

  // ⑤ 投稿 + 投票 + 選択肢をネストして一括作成
  // Prismaのネスト作成機能を使うと、関連するデータをまとめて作成できる
  const post = await prisma.post.create({
    data: {
      userId: session.user.id, // 投稿者ID
      content, // 投稿本文
      poll: {
        // create: Postに紐づくPollを同時に作成
        create: {
          duration, // 投票期間 (秒数)
          expiresAt, // 投票期限 (計算済み)
          options: {
            // create: Pollに紐づくPollOptionを複数同時に作成
            create: options.map((text, index) => ({
              text, // 選択肢のテキスト
              sortOrder: index, // 表示順序 (0始まり)
            })),
          },
        },
      },
    },
  })

  // 作成結果にリレーション先のデータも含める
  include: {
    poll: {
      include: {
        options: true, // 選択肢一覧も取得
      },
    },
  },
})
}

```



```
return { success: true, postId: post.id }
}
```

ネスト作成の構造:

```
prisma.post.create({
  data: {
    userId: ...,           ← Postレコードのデータ
    content: ...,
    poll: {
      create: {             ← Postに紐づくPollを作成
        duration: ...,
        expiresAt: ...,
        options: {
          create: [          ← Pollに紐づくPollOptionを作成
            { text: '黒松', sortOrder: 0 },
            { text: '五葉松', sortOrder: 1 },
            { text: '真柏', sortOrder: 2 },
          ]
        }
      }
    }
  }
})
```

実行結果のデータベース:

```
posts: [{ id: 'p1', content: '好きな松柏は?' }]
polls: [{ id: 'poll1', postId: 'p1', duration: 86400, expiresAt: '...' }]
poll_options: [
  { id: 'opt1', pollId: 'poll1', text: '黒松', sortOrder: 0 },
  { id: 'opt2', pollId: 'poll1', text: '五葉松', sortOrder: 1 },
  { id: 'opt3', pollId: 'poll1', text: '真柏', sortOrder: 2 },
]
```

投票する (PollVoteの作成):

```

// lib/actions/poll.ts

// ユーザーが投票するServer Action
export async function votePoll(pollId: string, optionId: string) {
  // ① 認証チェック
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // ② 投票期限チェック
  // findUniqueで投票データを取得し、期限切れでないか確認
  const poll = await prisma.poll.findUnique({
    where: { id: pollId },
  })

  if (!poll) {
    return { error: '投票が見つかりません' }
  }

  // expiresAtが現在時刻より前なら期限切れ
  if (poll.expiresAt < new Date()) {
    return { error: 'この投票は終了しています' }
  }

  // ③ 投票を作成（トランザクション内で実行）
  // tryブロックでユニーク制約違反をキャッチ
  try {
    await prisma.pollVote.create({
      data: {
        pollId, // どの投票に対して
        optionId, // どの選択肢に
        userId: session.user.id, // 誰が投票したか
      },
    })
    return { success: true }
  } catch (error) {
    // @@unique([pollId, userId]) 制約に違反した場合
    // → 既に投票済みのユーザーが再度投票しようとした
    if ((error as { code?: string }).code === 'P2002') {
      return { error: '既に投票済みです' }
    }
    throw error // その他のエラーは再スロー
  }
}

```

投票時のエラーハンドリング:

ケース	操作	結果
ケース1: 正常に投票	<pre>pollVote.create({ pollId: 'poll1', optionId: 'opt2', userId: 'u1' })</pre>	成功
ケース2: 同じユーザーが再度投票	<pre>pollVote.create({ pollId: 'poll1', optionId: 'opt3', userId: 'u1' })</pre> ← 別の選択肢でも同じユーザー	P2002エラー（ユニーク違反）→ '既に投票済みです'

P2002はPrismaのエラーコード:

- ユニーク制約（`@@unique`）に違反した場合に発生
- `@@unique([pollId, userId])` により、同じ組み合わせは作成不可

投票結果の取得:

```

// lib/actions/poll.ts

// 投票結果を取得する関数
export async function getPollResults(postId: string, currentUserId?: string) {
  // ① 投稿に紐づく投票データを取得
  const post = await prisma.post.findUnique({
    where: { id: postId },
    include: {
      poll: {
        include: {
          options: {
            include: {
              // 各選択肢の投票数をカウント
              _count: {
                select: { votes: true },
              },
            },
          },
          orderBy: { sortOrder: 'asc' }, // 表示順序でソート
        },
        // ログインユーザーの投票情報を取得
        votes: currentUserId
          ? {
              where: { userId: currentUserId },
              select: { optionId: true }, // どの選択肢に投票したかのみに絞り込む
            }
          : false, // 未ログインの場合は取得しない
        // 全投票数をカウント
        _count: {
          select: { votes: true },
        },
      },
    },
  })

  // ② 結果を整形して返す
  if (!post?.poll) return null

  const totalVotes = post.poll._count.votes // 全体の投票数
  const userVotedOptionId = post.poll.votes?.[0]?.optionId || null // ユーザーの投票先

  return {
    pollId: post.poll.id,
    expiresAt: post.poll.expiresAt,
    isExpired: post.poll.expiresAt < new Date(), // 期限切れかどうか
    totalVotes,
    userVotedOptionId,
    options: post.poll.options.map((option) => ({
      id: option.id,
      text: option.text,
      voteCount: option._count.votes,
      // パーセンテージ計算 (0除算を防ぐ)
      percentage: totalVotes > 0
        ? Math.round((option._count.votes / totalVotes) * 100)
        : 0,
    })),
  }
}

```

```
}
}
```

取得結果のイメージ:

```
{
  pollId: 'poll1',
  expiresAt: '2024-03-16T12:00:00Z',
  isExpired: false,
  totalVotes: 15,
  userVotedOptionId: 'opt2',      ← ログインユーザーは五葉松に投票済み
  options: [
    { id: 'opt1', text: '黒松',   voteCount: 5,  percentage: 33 },
    { id: 'opt2', text: '五葉松', voteCount: 7,  percentage: 47 }, ← 最多得票
    { id: 'opt3', text: '真柏',   voteCount: 3,  percentage: 20 },
  ]
}
```

表示イメージ: 「好きな松柏は？」

選択肢	得票率	備考
黒松	33%	
五葉松	47%	あなたが投票
真柏	20%	

15票 / 残り12時間

`_count`の活用: `_count: { select: { votes: true } }` を使うことで、投票数を取得するために別クエリを実行する必要がありません。Prismaが自動的にCOUNTサブクエリを生成し、1回のクエリで選択肢と投票数の両方を取得します。

7.11.2 ハッシュタグ (Hashtag / PostHashtag)

投稿にハッシュタグを付けて分類する機能です。多対多のリレーションを中間テーブルで実現しています。

```
// prisma/schema.prisma より抜粋

model Hashtag {
  id          String   @id @default(cuid())
  name        String   @unique           // ハッシュタグ名 (例: "黒松")
  count       Int      @default(0)       // このタグが使われている投稿数
  createdAt   DateTime @default(now()) @map("created_at")

  posts PostHashtag[] // このハッシュタグが付いた投稿一覧

  @@index([count])     // カウント順でのソートを高速化
  @@map("hashtags")
}

model PostHashtag {
  postId      String @map("post_id")
  hashtagId   String @map("hashtag_id")

  post        Post    @relation(fields: [postId], references: [id], onDelete: Cascade)
  hashtag     Hashtag @relation(fields: [hashtagId], references: [id], onDelete: Cascade)

  @@id([postId, hashtagId]) // 複合主キー: 同じ投稿に同じタグは1つだけ
  @@map("post_hashtags")
}
```

設計ポイント:

- `count` フィールド: ハッシュタグの使用回数をキャッシュとして保持。毎回集計クエリを実行する代わりに、このカウンターを参照することでトレンド表示が高速になります
- `@@index([count])`: カウント順でソートするクエリ (トレンドハッシュタグの取得) を高速化するインデックス
- `@@id([postId, hashtagId])`: PostGenreと同じ複合主キーパターン。同じ投稿に同じハッシュタグが2回付くことを防止

ハッシュタグのCRUD操作

ハッシュタグは投稿作成時に自動抽出し、トレンド表示に活用します。

投稿テキストからハッシュタグを抽出して登録:

```

// lib/actions/hashtag.ts

// 投稿テキストからハッシュタグを抽出するユーティリティ関数
export function extractHashtags(content: string): string[] {
  // 正規表現で #タグ のパターンを全て抽出
  // (?<=#) : '#' の直後にマッチ
  // [^\s#]+ : 空白と '#' 以外の1文字以上にマッチ
  const matches = content.match(/#(?:[^\s#]+)/g)

  if (!matches) return [] // ハッシュタグがなければ空配列

  // '#' を除去し、重複を排除
  return [...new Set(matches.map((tag) => tag.slice(1)))]
}

// 例: '#盆栽 #五葉松 今日の手入れ #盆栽'
// → ['盆栽', '五葉松'] (重複除去済み)

// 投稿にハッシュタグを登録するトランザクション
export async function saveHashtags(postId: string, hashtags: string[]) {
  // トランザクション内で複数操作をまとめて実行
  // → 途中で失敗した場合、全ての変更がロールバック（取り消し）される
  await prisma.$transaction(async (tx) => {
    for (const tagName of hashtags) {
      // ① ハッシュタグの取得 or 作成
      // upsert: 既にあれば取得、なければ作成
      const hashtag = await tx.hashtag.upsert({
        where: { name: tagName },
        update: {}, // 既にある場合は何も更新しない
        create: {
          name: tagName,
          count: 0, // カウントは後で更新
        },
      })

      // ② 投稿とハッシュタグの紐付けを作成
      // createManyは複合主キーの重複を無視するskipDuplicatesが使える
      await tx.postHashtag.create({
        data: {
          postId,
          hashtagId: hashtag.id,
        },
      })

      // ③ ハッシュタグのカウントを更新
      // increment: 現在の値に1を加算（排他制御込みで安全）
      await tx.hashtag.update({
        where: { id: hashtag.id },
        data: {
          count: { increment: 1 },
        },
      })
    }
  })
}

```

ハッシュタグ登録の流れ:

入力: '五葉松の植え替え完了 #盆栽 #五葉松 #植え替え'

Step 1: 抽出 - `extractHashtags()` → ['盆栽', '五葉松', '植え替え']

Step 2: トランザクション内で各タグを処理

タグ	upsert結果	PostHashtag作成	count更新
'盆栽'	既存 (id: 'h1', count: 42)	postId: 'p1', hashtagId: 'h1'	42 → 43
'五葉松'	既存 (id: 'h2', count: 15)	postId: 'p1', hashtagId: 'h2'	15 → 16
'植え替え'	新規作成 (id: 'h99', count: 0)	postId: 'p1', hashtagId: 'h99'	0 → 1

トランザクション成功 → 全ての変更がコミット

トレンドハッシュタグの取得:

```
// lib/actions/hashtag.ts

// トrendハッシュタグを取得する関数
export async function getTrendingHashtags(limit: number = 10) {
  // countが多い順にハッシュタグを取得
  const hashtags = await prisma.hashtag.findMany({
    where: {
      count: { gt: 0 }, // 使用回数が1以上
    },
    orderBy: {
      count: 'desc', // 使用回数が多い順
    },
    take: limit, // 上位N件のみ
    select: {
      id: true,
      name: true,
      count: true,
    },
  })

  return hashtags
}

// 結果例:
// [
//   { id: 'h1', name: '盆栽', count: 156 },
//   { id: 'h2', name: '黒松', count: 89 },
//   { id: 'h3', name: '五葉松', count: 72 },
//   { id: 'h4', name: '植え替え', count: 45 },
//   ...
// ]
```


ハッシュタグ検索（特定のタグが付いた投稿一覧）：

```
// lib/actions/hashtag.ts

// 特定のハッシュタグが付いた投稿を取得
export async function getPostsByHashtag(
  tagName: string,
  cursor?: string,
  limit: number = 20
) {
  const posts = await prisma.post.findMany({
    where: {
      isHidden: false, // 非表示でない
      hashtags: {
        // some: リレーション先に「1つでも条件を満たすものがある」
        some: {
          hashtag: {
            name: tagName, // ハッシュタグ名で絞り込み
          },
        },
      },
    },
    orderBy: { createdAt: 'desc' }, // 新しい順
    take: limit + 1, // 「次のページがあるか」判定用に+1
    ...(cursor && {
      cursor: { id: cursor },
      skip: 1, // カーソル自体をスキップ
    }),
    include: {
      user: {
        select: { id: true, nickname: true, avatarUrl: true },
      },
      _count: {
        select: { likes: true, comments: true },
      },
    },
  })

  // ページネーション処理
  const hasMore = posts.length > limit
  const data = hasMore ? posts.slice(0, -1) : posts
  const nextCursor = hasMore ? data[data.length - 1]?.id : undefined

  return { posts: data, nextCursor, hasMore }
}
```

someフィルタの動作:

```
where: {
  hashtags: {
    some: {           ← 「1つでも条件を満たすPostHashtagがある」
      hashtag: {
        name: '盆栽' ← 「hashtagテーブルのnameが '盆栽」
      }
    }
  }
}
```

SQLに変換されると:

```
SELECT posts.*
FROM posts
WHERE posts.id IN (
  SELECT post_id FROM post_hashtags
  INNER JOIN hashtags ON post_hashtags.hashtag_id = hashtags.id
  WHERE hashtags.name = '盆栽'
)
AND posts.is_hidden = false
ORDER BY posts.created_at DESC
LIMIT 21
```

someフィルタとは: Prismaの `some` フィルタは、リレーション先のレコードの中に「1つでも条件を満たすもの」があるかをチェックします。 `every` (全て満たす) や `none` (1つも満たさない) というフィルタもあります。

7.11.3 ユーザーアナリティクス (UserAnalytics)

プレミアム会員向けに、プロフィールの閲覧数や投稿の反応を日別で記録するモデルです。

```
// prisma/schema.prisma より抜粋

model UserAnalytics {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  date        DateTime  @db.Date              // 記録対象の日付（時刻なし）
  profileViews Int       @default(0) @map("profile_views") // プロフィール閲覧数
  postViews   Int       @default(0) @map("post_views")     // 投稿閲覧数
  likesReceived Int     @default(0) @map("likes_received") // 受け取ったいいね数
  newFollowers Int     @default(0) @map("new_followers")    // 新規フォロワー数
  createdAt   DateTime  @default(now()) @map("created_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([userId, date]) // 1ユーザー1日1レコード
  @@index([userId])
  @@index([date])
  @@map("user_analytics")
}
```

UserAnalyticsのデータイメージ（user_analytics テーブル）：

id	userId	date	profileViews	postViews	likesReceived	newFollowers
a1	u1	2024-03-01	12	45	8	2
a2	u1	2024-03-02	8	30	5	1
a3	u1	2024-03-03	15	60	12	3

- 3月1日~3日のプロフィール閲覧数の推移: 12 → 8 → 15
- グラフ表示に最適なデータ構造

設計ポイント：

- `@@unique([userId, date])`：複合ユニーク制約で「1ユーザー・1日・1レコード」を保証。この制約があることで `upsert`（あれば更新、なければ作成）が安全に使えます
- `@db.Date`：PostgreSQLのDATE型（時刻を含まない日付のみ）。日別集計のために時刻情報は不要
- 各カウンターは `@default(0)`：新規レコード作成時のデフォルト値
- 日別にレコードを分けることで、期間指定での集計やグラフ表示が容易

アナリティクスのCRUD操作

アナリティクスは「記録」と「集計表示」の2つの操作が中心です。

プロフィール閲覧の記録:

```
// lib/actions/analytics.ts

// プロフィール閲覧を記録する関数
export async function recordProfileView(userId: string) {
  // 今日の日付（時刻を0時に設定）
  const today = new Date()
  today.setHours(0, 0, 0, 0)

  // upsert: 今日のレコードがあれば閲覧数+1、なければ新規作成
  await prisma.userAnalytics.upsert({
    where: {
      // 複合ユニークキーを使った検索
      // @@unique([userId, date]) に対応
      userId_date: {
        userId: userId,
        date: today,
      },
    },
    update: {
      // increment: 現在の値に1を加算
      // SQLでは SET profile_views = profile_views + 1 に相当
      profileViews: { increment: 1 },
    },
    create: {
      userId: userId,
      date: today,
      profileViews: 1,    // 初回閲覧
      postViews: 0,
      likesReceived: 0,
      newFollowers: 0,
    },
  })
}
```

incrementの安全性:

{ increment: 1 } は以下のSQLに変換される:

```
UPDATE user_analytics
SET profile_views = profile_views + 1
WHERE user_id = 'u1' AND date = '2024-03-15'
```

なぜ「現在値を取得して+1してから保存」ではダメなのか？

❌ 危険なパターン:

リクエストA: 値を読む → `profile_views = 5`
リクエストB: 値を読む → `profile_views = 5` (Aの更新前に読んでしまう)
リクエストA: `5 + 1 = 6` を書き込み
リクエストB: `5 + 1 = 6` を書き込み
結果: 6 (期待値は7)

✅ increment を使うパターン:

リクエストA: `SET profile_views = profile_views + 1` (DB内で計算)
リクエストB: `SET profile_views = profile_views + 1` (DB内で計算)
結果: 7 (正しい)

→ データベースが排他制御してくれるので、同時アクセスでも正確

アナリティクスデータの集計取得:

```

// lib/actions/analytics.ts

// 期間指定でアナリティクスデータを取得する関数
export async function getUserAnalytics(
  userId: string,
  days: number = 30 // 過去何日分を取得するか
) {
  // 開始日を計算
  const startDate = new Date()
  startDate.setDate(startDate.getDate() - days)
  startDate.setHours(0, 0, 0, 0)

  // ① 日別データの取得
  const dailyData = await prisma.userAnalytics.findMany({
    where: {
      userId,
      date: { gte: startDate }, // startDate以降のレコード
    },
    orderBy: { date: 'asc' }, // 古い順にソート（グラフ表示用）
    select: {
      date: true,
      profileViews: true,
      postViews: true,
      likesReceived: true,
      newFollowers: true,
    },
  })

  // ② 合計値の算出
  // aggregate: SUM, AVG, MIN, MAX 等の集計関数を実行
  const totals = await prisma.userAnalytics.aggregate({
    where: {
      userId,
      date: { gte: startDate },
    },
    _sum: {
      profileViews: true, // プロフィール閲覧の合計
      postViews: true, // 投稿閲覧の合計
      likesReceived: true, // いいね受取の合計
      newFollowers: true, // 新規フォロワーの合計
    },
    _avg: {
      profileViews: true, // 1日あたりの平均閲覧数
    },
  })

  return {
    dailyData,
    totals: {
      profileViews: totals._sum.profileViews ?? 0,
      postViews: totals._sum.postViews ?? 0,
      likesReceived: totals._sum.likesReceived ?? 0,
      newFollowers: totals._sum.newFollowers ?? 0,
      avgDailyViews: Math.round(totals._avg.profileViews ?? 0),
    },
  }
}

```

```
}
}
```

集計結果のイメージ:

```
dailyData: [
  { date: '2024-03-01', profileViews: 12, postViews: 45, ... },
  { date: '2024-03-02', profileViews: 8, postViews: 30, ... },
  { date: '2024-03-03', profileViews: 15, postViews: 60, ... },
  ...
]

totals: {
  profileViews: 350,    ← 30日間の合計
  postViews: 1200,
  likesReceived: 89,
  newFollowers: 24,
  avgDailyViews: 12,   ← 1日平均
}
```

グラフ表示（閲覧数の推移）:

3/1: 15 → 3/5: 45 → 3/10: 60 → 3/15: 30
dailyDataの配列をフロントエンドのチャートライブラリで描画

aggregateとgroupByの使い分け: `aggregate` はテーブル全体（またはwhere条件に一致するレコード全体）に対してSUM/AVG等を計算します。`groupBy` は特定のフィールドでグループ分けした上で、各グループごとに集計を行います。「全体の合計」にはaggregate、「日別・種類別の集計」にはgroupByを使います。

7.11.4 システム設定（SystemSetting）

メンテナンスモードの切り替えなど、システム全体の設定を管理するキーバリューストア型のモデルです。

```
// prisma/schema.prisma より抜粋

model SystemSetting {
  id      String  @id @default(cuid())
  key     String  @unique           // 設定キー（例: "maintenance_mode"）
  value   Json    // 設定値（JSON形式で柔軟に保存）
  updatedBy String? @map("updated_by") // 最後に更新した管理者ID
  updatedAt DateTime @updatedAt @map("updated_at")
  createdAt DateTime @default(now()) @map("created_at")

  @@map("system_settings")
}
```

キーバリューストア方式の利点:

設定ごとにカラムを追加する方式（スキーマ変更が必要）:

maintenance_mode	max_posts	...新しい設定	備考
false	20	...	カラム追加 = マイグレーション必要

キーバリューストア方式（スキーマ変更不要） -- BON-LOGで採用:

key	value (JSON)	備考
maintenance_mode	<code>{ "enabled": false }</code>	
max_posts	<code>{ "limit": 20 }</code>	
new_setting	<code>{ ... }</code>	行を追加するだけ

設計ポイント:

- `key` に `@unique`: 同じキーの重複を防止
- `value` が `Json` 型: 設定の値をJSON形式で柔軟に保存。 `{ "enabled": true, "message": "メンテナンス中" }` のように構造化されたデータも格納可能
- 新しい設定を追加するときにスキーマ変更（マイグレーション）が不要

システム設定のCRUD操作

システム設定はキーバリューストア方式なので、汎用的な取得・更新関数を作成します。

設定値の取得と更新:


```

// lib/actions/system-settings.ts

// 設定値を取得する汎用関数
export async function getSystemSetting<T>(key: string): Promise<T | null> {
  // keyで検索 (keyはユニーク制約が付いている)
  const setting = await prisma.systemSetting.findUnique({
    where: { key },
  })

  // valueフィールドはJson型なので、型アサーションでT型に変換
  return setting ? (setting.value as T) : null
}

// 設定値を更新する汎用関数
export async function setSystemSetting(
  key: string,
  value: unknown,          // JSON形式で保存できる任意の値
  adminId: string          // 更新した管理者のID (監査証跡)
) {
  // upsert: 設定キーが既にあれば更新、なければ作成
  await prisma.systemSetting.upsert({
    where: { key },
    update: {
      value: value as object, // Json型にキャスト
      updatedBy: adminId,     // 最後に更新した管理者を記録
    },
    create: {
      key,
      value: value as object,
      updatedBy: adminId,
    },
  })
}

// 使用例: メンテナンスモードの管理
// -----

// メンテナンスモードの設定値の型定義
type MaintenanceConfig = {
  enabled: boolean
  message: string
  allowedIps: string[] // メンテナンス中もアクセス可能なIP
}

// メンテナンスモード状態の確認
export async function isMaintenanceMode(): Promise<boolean> {
  const config = await getSystemSetting<MaintenanceConfig>('maintenance_mode')
  return config?.enabled ?? false // 設定がなければfalse (通常運用)
}

// メンテナンスモードの切り替え
export async function toggleMaintenance(
  enabled: boolean,
  message: string,
  adminId: string
) {

```

```

    await setSystemSetting(
      'maintenance_mode',
      {
        enabled,
        message,
        allowedIps: ['127.0.0.1'], // ローカルホストは常にアクセス可能
      },
      adminId
    )
  }

// 使用例: 投稿制限の管理
// -----

type PostLimitConfig = {
  maxPostsPerDay: number // 1日の最大投稿数
  maxCommentsPerDay: number // 1日の最大コメント数
}

export async function getPostLimits(): Promise<PostLimitConfig> {
  const config = await getSystemSetting<PostLimitConfig>('post_limits')
  // デフォルト値を返す (設定がない場合)
  return config ?? { maxPostsPerDay: 20, maxCommentsPerDay: 100 }
}

```

SystemSettingの実際のデータ例:

system_settings テーブル

key	value (JSON)	updatedBy
maintenance_mode	<code>{ "enabled": false, "message": "", "allowedIps": ["127.0.0.1"] }</code>	admin1
post_limits	<code>{ "maxPostsPerDay": 20, "maxCommentsPerDay": 100 }</code>	admin1
feature_flags	<code>{ "polls": true, "dm": true, "scheduledPosts": true }</code>	admin2

メリット:

- 新しい設定を追加するとき、テーブル構造を変更する必要がない
- 行を追加するだけで新しい設定を管理できる
- JSON形式なので、設定値の構造を自由に設計できる

7.11.5 ブラックリスト系モデル (EmailBlacklist / DeviceBlacklist)

不正利用者の再登録を防ぐためのブラックリスト機能です。

```
// prisma/schema.prisma より抜粋

// メールアドレスブラックリスト
model EmailBlacklist {
  id          String   @id @default(cuid())
  email       String   @unique
  reason      String?  @db.Text              // ブロック理由（管理用メモ）
  createdBy   String   @map("created_by")    // 登録した管理者ID
  createdAt   DateTime @default(now()) @map("created_at")

  @@index([email])
  @@map("email_blacklist")
}

// デバイスブラックリスト（フィンガープリントベース）
model DeviceBlacklist {
  id          String   @id @default(cuid())
  fingerprint String   @unique              // デバイスフィンガープリント（ハッシュ）
  reason      String?  @db.Text              // ブロック理由
  originalEmail String? @map("original_email") // 元のブロックユーザーのメール
  createdBy   String   @map("created_by")
  createdAt   DateTime @default(now()) @map("created_at")

  @@index([fingerprint])
  @@map("device_blacklist")
}
```

設計ポイント:

- **EmailBlacklist**: 停止されたアカウントのメールアドレスでの再登録を防止。新規ユーザー登録時にこのテーブルを照合します
- **DeviceBlacklist**: メールアドレスを変えて再登録する不正利用者を検出するため、ブラウザのフィンガープリント（デバイス固有の識別情報のハッシュ値）を記録
- **createdBy**: 誰（どの管理者）がブロックしたかの監査証跡を残す

ブラックリスト照合の実装

ブラックリストは「ユーザー登録時の照合」と「管理者によるブロック操作」の2つの場面で使います。

新規登録時のブラックリスト照合:

```

// lib/actions/auth.ts

// 新規ユーザー登録時にブラックリストを照合する関数
export async function checkBlacklist(
  email: string,
  fingerprint?: string
): Promise<{ blocked: boolean; reason?: string }> {
  // ① メールアドレスのブラックリスト照合
  const emailBlocked = await prisma.emailBlacklist.findUnique({
    where: { email },
  })

  if (emailBlocked) {
    return {
      blocked: true,
      reason: 'このメールアドレスは使用できません',
    }
  }

  // ② デバイスフィンガープリントのブラックリスト照合
  if (fingerprint) {
    const deviceBlocked = await prisma.deviceBlacklist.findUnique({
      where: { fingerprint },
    })

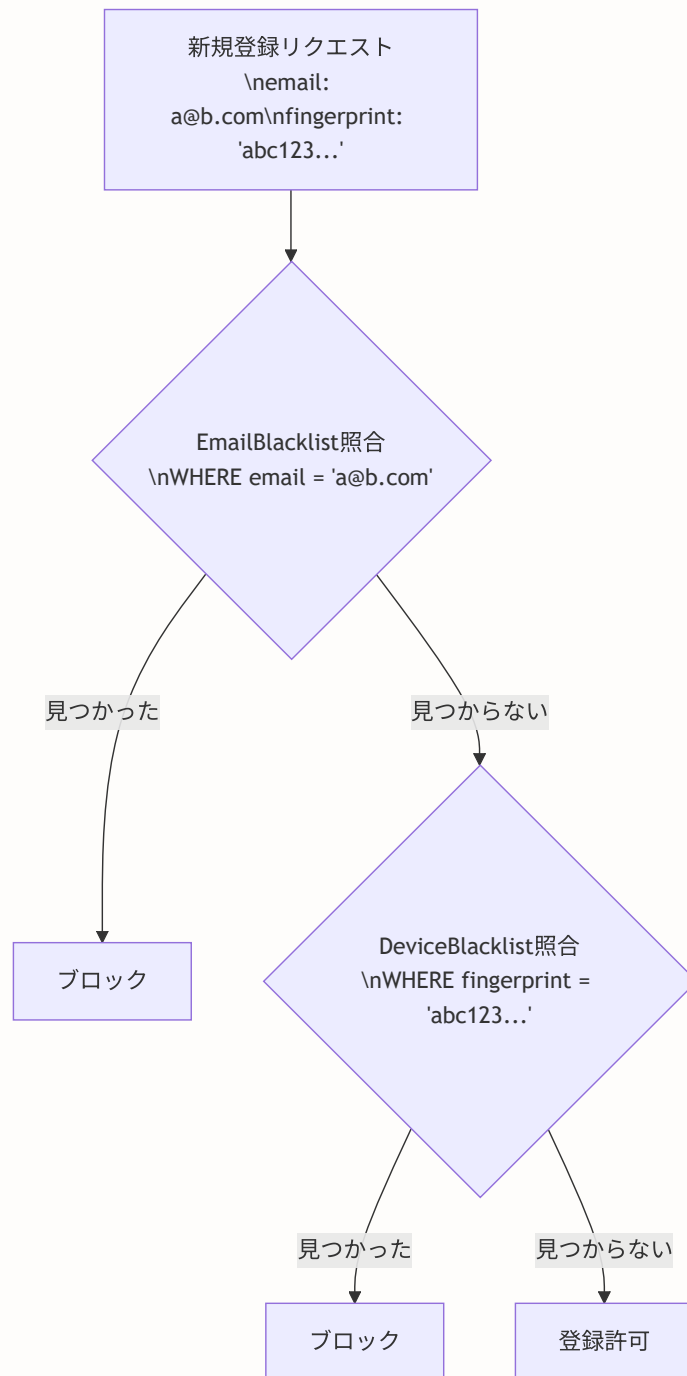
    if (deviceBlocked) {
      return {
        blocked: true,
        reason: 'このデバイスからの登録は制限されています',
      }
    }
  }

  return { blocked: false }
}

// 登録フローでの使用例:
// const { blocked, reason } = await checkBlacklist(email, fingerprint)
// if (blocked) return { error: reason }
// ... 通常の登録処理を続行

```

ブラックリスト照合の流れ:



管理者がユーザーをブラックリストに追加:

```

// lib/actions/blacklist.ts

// ユーザーを停止し、関連デバイスもブラックリストに追加する管理者操作
export async function banUser(userId: string, reason: string, adminId: string) {
  // トランザクションで全操作をまとめて実行
  await prisma.$transaction(async (tx) => {
    // ① ユーザー情報の取得
    const user = await tx.user.findUnique({
      where: { id: userId },
      select: { email: true },
    })

    if (!user) throw new Error('ユーザーが見つかりません')

    // ② ユーザーを停止状態にする
    await tx.user.update({
      where: { id: userId },
      data: {
        isSuspended: true,
        suspendedAt: new Date(),
      },
    })

    // ③ メールアドレスをブラックリストに追加
    await tx.emailBlacklist.upsert({
      where: { email: user.email },
      update: { reason, createdBy: adminId },
      create: {
        email: user.email,
        reason,
        createdBy: adminId,
      },
    })

    // ④ ユーザーの全デバイスをブラックリストに追加
    const devices = await tx.userDevice.findMany({
      where: { userId },
      select: { fingerprint: true },
    })

    for (const device of devices) {
      await tx.deviceBlacklist.upsert({
        where: { fingerprint: device.fingerprint },
        update: { reason, createdBy: adminId },
        create: {
          fingerprint: device.fingerprint,
          reason,
          originalEmail: user.email,
          createdBy: adminId,
        },
      },
    )
  })
}

```

トランザクションの重要性: ユーザー停止+ブラックリスト追加は必ずセットで行う必要があります。途中で失敗した場合（例えばブラックリスト追加でエラーが発生した場合）、ユーザーの停止状態だけが反映されてブラックリストが不完全になるのを防ぐため、トランザクション内で実行しています。

7.11.6 下書き・予約投稿（DraftPost / ScheduledPost）

投稿の下書き保存と、指定日時に自動投稿する機能を実現するモデルです。

```
// prisma/schema.prisma より抜粋

// 下書き投稿
model DraftPost {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  content     String?   @db.Text
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  media       DraftPostMedia[]
  genres      DraftPostGenre[]

  @@index([userId])
  @@map("draft_posts")
}

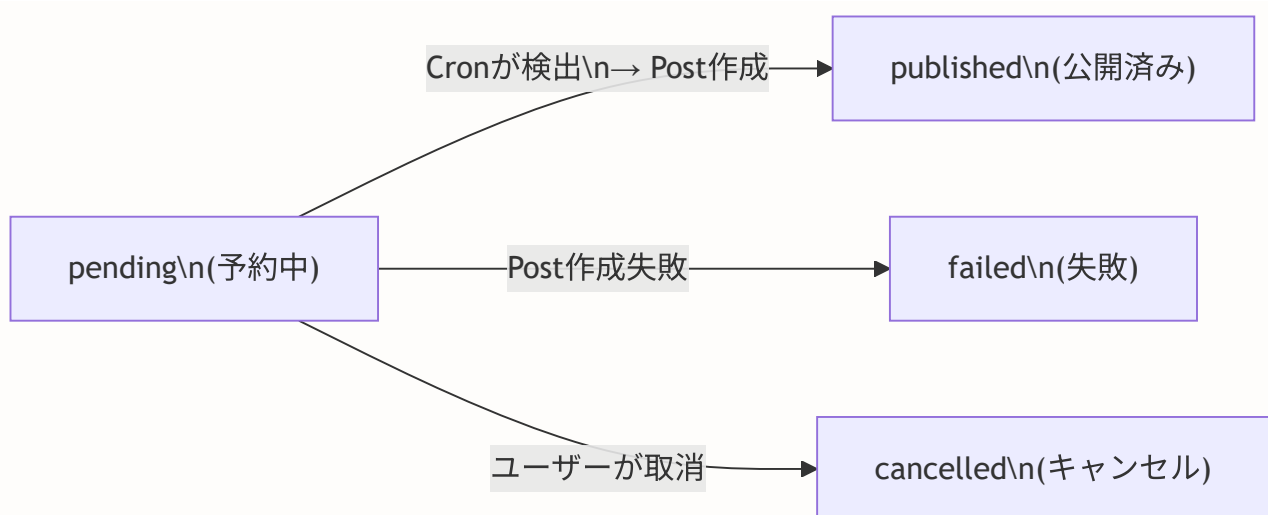
// 有料会員 - 予約投稿
model ScheduledPost {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  content     String?   @db.Text
  scheduledAt  DateTime  @map("scheduled_at") // 投稿予定日時
  status      ScheduledPostStatus @default(pending) // 状態管理
  publishedPostId String? @unique @map("published_post_id") // 公開後の投稿ID
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  publishedPost Post?     @relation(fields: [publishedPostId], references: [id], onDelete: Cascade)
  media       ScheduledPostMedia[]
  genres      ScheduledPostGenre[]

  @@index([userId])
  @@index([scheduledAt])
  @@index([status])
  @@index([status, scheduledAt]) // Cronジョブのクエリ最適化用
  @@map("scheduled_posts")
}

// 予約投稿のステータスを列挙型で定義
enum ScheduledPostStatus {
  pending    // 予約中（まだ投稿されていない）
  published  // 公開済み（正常に投稿された）
  failed     // 公開失敗（エラーが発生した）
  cancelled  // キャンセル（ユーザーが取り消した）
}
```

予約投稿のライフサイクル:



Cronジョブの検索条件: `WHERE status = 'pending' AND scheduled_at ≤ NOW()` → `@@index([status, scheduledAt])` が効く

設計ポイント:

- `DraftPost` はPostモデルと似た構造を持つが、独立したテーブルとして設計。下書きは未完成のデータであり、postsテーブルのデータ品質を保つため分離しています
- `ScheduledPost` の `status` はenum型。文字列で自由に値を入れるよりも、決まった選択肢だけを許可することでデータの整合性を保ちます
- `@@index([status, scheduledAt])`: Cronジョブが「pendingかつ予定時刻を過ぎた予約投稿」を効率的に検索するための複合インデックス
- `publishedPostId` の `@unique`: 1つの予約投稿から作成される実際の投稿は1つだけ

下書きのCRUD操作

下書き機能では「保存」「一覧取得」「下書きから投稿に変換」の3つが主要な操作です。

下書きの保存（自動保存対応）:

```
// lib/actions/draft.ts

// 下書きを保存する（新規作成 or 既存更新）
export async function saveDraft(
  userId: string,
  data: {
    id?: string // 既存の下書きIDがあれば更新
    content: string
    genreIds: string[]
  }
) {
  if (data.id) {
    // — 既存の下書きを更新 —
    const draft = await prisma.draftPost.update({
      where: {
        id: data.id,
        userId, // 自分の下書きのみ更新可能（認可チェック）
      },
      data: {
        content: data.content,
        // ジャンルの関連を「全削除→再作成」で更新
        genres: {
          deleteMany: {}, // 既存の関連を全て削除
          create: data.genreIds.map((genreId) => ({ genreId })), // 新しい関連を作成
        },
      },
    })
    return draft
  } else {
    // — 新しい下書きを作成 —
    const draft = await prisma.draftPost.create({
      data: {
        userId,
        content: data.content,
        genres: {
          create: data.genreIds.map((genreId) => ({ genreId })),
        },
      },
    })
    return draft
  }
}
```

下書きのジャンル更新パターン（deleteMany + create）：

更新前の状態 -- draft_post_genres テーブル:

draftPostId	genreId	ジャンル
draft1	g1	松柏類
draft1	g2	雑木類

ユーザーが「松柏類」を外して「道具」を追加した場合:

Step 1: deleteMany → 全て削除 (draft_post_genres テーブル: 空)

Step 2: create → 新しい関連を作成

draftPostId	genreId	ジャンル	備考
draft1	g2	雑木類	残った
draft1	g5	道具	新しく追加

なぜ「差分更新」ではなく「全削除→再作成」？

- 差分計算のロジックが複雑になるため
- 中間テーブルのレコード数は少ない（最大3件）ので性能問題なし
- 実装がシンプルで理解しやすい

下書きから投稿に変換:

```

// lib/actions/draft.ts

// 下書きを投稿に変換して公開する
export async function publishDraft(draftId: string, userId: string) {
  // トランザクション: 投稿作成 + 下書き削除をセットで実行
  const post = await prisma.$transaction(async (tx) => {
    // ① 下書きデータを取得
    const draft = await tx.draftPost.findUnique({
      where: { id: draftId, userId }, // 自分の下書きのみ
      include: {
        media: true, // 添付メディア
        genres: true, // ジャンル
      },
    })

    if (!draft) throw new Error('下書きが見つかりません')

    // ② 下書きデータを元に投稿を作成
    const newPost = await tx.post.create({
      data: {
        userId,
        content: draft.content,
        // メディアをコピー
        media: {
          create: draft.media.map((m) => ({
            url: m.url,
            type: m.type,
            sortOrder: m.sortOrder,
          })),
        },
        // ジャンルをコピー
        genres: {
          create: draft.genres.map((g) => ({
            genreId: g.genreId,
          })),
        },
      },
    })

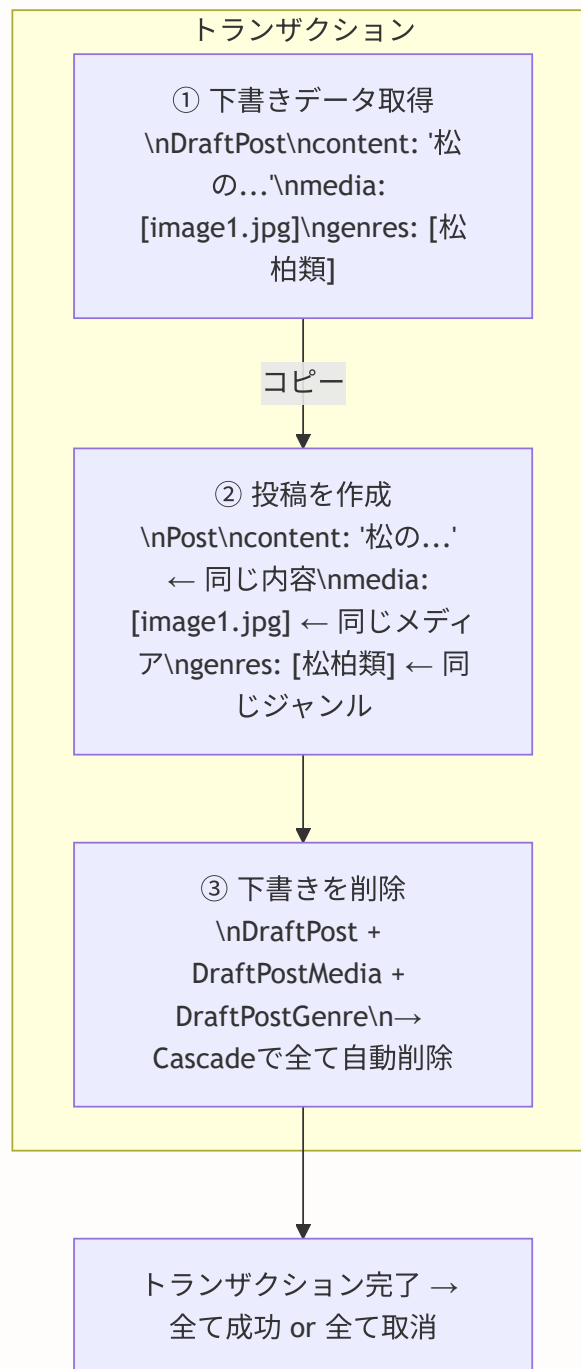
    // ③ 下書きを削除 (Cascadeで関連するmedia, genresも自動削除)
    await tx.draftPost.delete({
      where: { id: draftId },
    })

    return newPost
  })

  return post
}

```

下書き → 投稿の変換フロー:



予約投稿のCronジョブ処理

予約投稿は、バックグラウンドのCronジョブ（定期実行タスク）が期限を過ぎた予約を検出し、自動的に投稿に変換します。

```

// lib/actions/scheduled-post.ts

// Cronジョブから呼ばれる：期限を過ぎた予約投稿を公開する
export async function publishScheduledPosts() {
  // ① 公開すべき予約投稿を検索
  // WHERE status = 'pending' AND scheduled_at ≤ NOW()
  // → @@index([status, scheduledAt]) が効くクエリ
  const scheduledPosts = await prisma.scheduledPost.findMany({
    where: {
      status: 'pending', // 予約中のもの
      scheduledAt: { lte: new Date() }, // 予定時刻を過ぎたもの
    },
    include: {
      media: true,
      genres: true,
    },
    take: 10, // 一度に処理する件数を制限（サーバー負荷対策）
  })

  let successCount = 0
  let failCount = 0

  // ② 各予約投稿を1件ずつ処理
  for (const scheduled of scheduledPosts) {
    try {
      await prisma.$transaction(async (tx) => {
        // 投稿を作成
        const post = await tx.post.create({
          data: {
            userId: scheduled.userId,
            content: scheduled.content,
            media: {
              create: scheduled.media.map((m) => ({
                url: m.url,
                type: m.type,
                sortOrder: m.sortOrder,
              })),
            },
            genres: {
              create: scheduled.genres.map((g) => ({
                genreId: g.genreId,
              })),
            },
          },
        })

        // 予約投稿のステータスを更新
        await tx.scheduledPost.update({
          where: { id: scheduled.id },
          data: {
            status: 'published', // 公開済みに変更
            publishedPostId: post.id, // 作成された投稿のIDを記録
          },
        })
      })

      successCount++
    } catch (error) {
      failCount++
    }
  }
}

```

```
    } catch (error) {  
      // 個別の予約投稿が失敗しても、他の投稿は処理を続行  
      await prisma.scheduledPost.update({  
        where: { id: scheduled.id },  
        data: { status: 'failed' }, // 失敗ステータスに変更  
      })  
      console.error(`予約投稿 ${scheduled.id} の公開に失敗:`, error)  
      failCount++  
    }  
  }  
  
  return { processed: scheduledPosts.length, successCount, failCount }  
}
```

Cronジョブの実行イメージ:

毎分実行: `publishScheduledPosts()`

scheduled_posts テーブル (実行前、現在時刻: 2024-03-15 10:00) :

id	userId	scheduledAt	status	判定
sp1	u1	2024-03-15 09:00	pending	過ぎた → 処理対象
sp2	u2	2024-03-15 09:30	pending	過ぎた → 処理対象
sp3	u3	2024-03-15 12:00	pending	まだ → スキップ
sp4	u1	2024-03-14 18:00	published	処理済み → スキップ

処理結果:

- sp1 → Post作成成功 → status = 'published'
- sp2 → Post作成失敗 → status = 'failed'
- sp3 → まだ時間前 → スキップ
- sp4 → 既に処理済み → スキップ

結果: `{ processed: 2, successCount: 1, failCount: 1 }`

個別エラーハンドリング: 1件の予約投稿の公開に失敗しても、ループ全体は中断せず次の予約投稿の処理を続けます。これにより、1件の障害が他の正常な予約投稿に影響を与えるのを防いでいます。

7.11.7 NextAuth.js用モデル (Account / Session / VerificationToken / PasswordResetToken)

NextAuth.js (Auth.js v5) が内部的に使用する認証関連モデルです。これらは認証ライブラリが自動的に管理するため、開発者が直接操作することは少ないですが、スキーマに含める必要があります。


```
// prisma/schema.prisma より抜粋

// OAuthプロバイダーのアカウント情報
model Account {
  id          String  @id @default(cuid())
  userId      String  @map("user_id")
  type        String           // "oauth" | "credentials"
  provider    String           // "google" | "github" 等
  providerAccountId String @map("provider_account_id")
  refresh_token String? @db.Text
  access_token String? @db.Text
  expires_at   Int?
  token_type   String?
  scope        String?
  id_token     String? @db.Text
  session_state String?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([provider, providerAccountId]) // 同じプロバイダーで同じアカウントは1つだけ
  @@map("accounts")
}

// セッション管理（JWT戦略の場合は使用されない）
model Session {
  id          String  @id @default(cuid())
  sessionToken String  @unique @map("session_token")
  userId      String  @map("user_id")
  expires     DateTime
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@map("sessions")
}

// メール確認トークン
model VerificationToken {
  identifier String
  token      String  @unique
  expires    DateTime

  @@unique([identifier, token])
  @@map("verification_tokens")
}

// パスワードリセットトークン
model PasswordResetToken {
  id          String  @id @default(cuid())
  email       String
  token       String  @unique
  expires     DateTime
  createdAt   DateTime @default(now()) @map("created_at")

  @@unique([email, token])
}
```

```
    @@map("password_reset_tokens")
  }
```

使用ファイル: `prisma/schema.prisma`, `lib/auth.ts`

設計ポイント:

- `Account` モデルはGoogle, GitHub等のOAuthログイン情報を保存します。 `@@unique([provider, providerAccountId])` で同じOAuthアカウントの重複登録を防ぎます
- BON-LOGではJWT戦略を使用しているため、`Session` モデルは実際にはほぼ使われません（JWTはサーバー側にセッション情報を保存しないため）
- `PasswordResetToken` はNextAuth標準ではなく、BON-LOG独自に追加したモデルです。パスワードリセットメールのリンクに含まれるトークンを一時保存し、有効期限で自動失効させます

この実装で可能になること: メール+パスワード認証、Googleログイン、パスワードリセットなどの認証フローが動作します。

実装しない場合の影響: NextAuth.jsが動作せず、ユーザーのログイン・登録ができません。

7.11.8 CommentMediaモデル（コメントメディア）

コメントに添付された画像・動画を管理するモデルです。PostMediaと同じ構造で、コメント版です。

```
// prisma/schema.prisma より抜粋

model CommentMedia {
  id          String   @id @default(cuid())
  commentId   String   @map("comment_id")
  url         String
  type        MediaType // image | video
  sortOrder   Int      @default(0) @map("sort_order")

  comment Comment @relation(fields: [commentId], references: [id], onDelete: Cascade)

  @@index([commentId])
  @@map("comment_media")
}

// メディアタイプの列挙型 (PostMediaと共通)
enum MediaType {
  image
  video
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/comment.ts`

コメント作成時のメディア保存:

```
// lib/actions/comment.ts (抜粋)

const comment = await prisma.comment.create({
  data: {
    postId,
    userId,
    parentId: parentId || null,
    content,
    media: {
      create: mediaUrls.map((url, index) => ({
        url,
        type: mediaTypes[index] as MediaType,
        sortOrder: index,
      })),
    },
  },
})
```

この実装で可能になること: コメントにも画像を添付でき、より豊かなコミュニケーションが可能になります。

実装しない場合の影響: コメントがテキストのみに限定され、画像を使った回答や説明ができません。

7.11.9 ShopChangeRequestモデル（盆栽園変更リクエスト）

ユーザーが盆栽園の情報変更を管理者にリクエストする仕組みです。

```
// prisma/schema.prisma より抜粋

model ShopChangeRequest {
  id          String      @id @default(cuid())
  shopId      String      @map("shop_id")
  userId      String      @map("user_id")
  status      RequestStatus @default(pending)
  requestedChanges Json    @map("requested_changes") // 変更内容（JSON形式）
  reason      String?     @db.Text                  // 変更理由
  adminComment String?     @db.Text @map("admin_comment")
  resolvedAt  DateTime?    @map("resolved_at")
  createdAt   DateTime     @default(now()) @map("created_at")

  shop BonsaiShop @relation(fields: [shopId], references: [id], onDelete: Cascade)
  user User        @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([shopId])
  @@index([userId])
  @@index([status])
  @@map("shop_change_requests")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/shop.ts`

設計ポイント:

- `requestedChanges` はJson型で変更内容を柔軟に保存します。例: `{ "name": "新しい名前", "phone": "03-xxxx-xxxx" }`
- 管理者が `adminComment` でフィードバックを残し、`status` を `approved` に変更すると実際のShop情報が更新されます

この実装で可能になること: 登録者以外のユーザーでも盆栽園の情報修正をリクエストでき、管理者が承認する形で情報の正確性を保てます。

実装しない場合の影響: 盆栽園の情報が古くなっても登録者本人しか更新できず、情報の鮮度が保てません。

7.11.10 UserHiddenPostモデル（投稿非表示）

ユーザーが特定の投稿を自分のタイムラインから非表示にする機能です。

```
// prisma/schema.prisma より抜粋

model UserHiddenPost {
  id          String   @id @default(cuid())
  userId      String   @map("user_id")
  postId      String   @map("post_id")
  createdAt   DateTime @default(now()) @map("created_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  post Post @relation(fields: [postId], references: [id], onDelete: Cascade)

  @@unique([userId, postId]) // 同じ投稿を2回非表示にはしない
  @@index([userId])
  @@map("user_hidden_posts")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/hide-post.ts`

投稿を非表示にする実装:

```
// lib/actions/hidden-post.ts

export async function hidePost(postId: string) {
  const { userId, error: authError } = await requireAuth()
  if (!userId) return { error: authError! }

  // 自分の投稿は非表示にできない
  const post = await prisma.post.findUnique({ where: { id: postId } })
  if (post?.userId === userId) {
    return { error: '自分の投稿は非表示にできません' }
  }

  // upsertで幕等に処理（既に非表示でもエラーにならない）
  await prisma.userHiddenPost.upsert({
    where: { userId_postId: { userId, postId } },
    create: { userId, postId },
    update: {},
  })

  revalidatePath('/feed')
  return { success: true }
}

// 非表示にした投稿IDの一覧を取得（タイムラインフィルタ用）
export async function getHiddenPostIds(userId: string): Promise<string[]> {
  const hidden = await prisma.userHiddenPost.findMany({
    where: { userId },
    select: { postId: true },
  })
  return hidden.map((h) => h.postId)
}
```

この実装で可能になること: 興味のない投稿を個別に非表示にでき、タイムラインの質を自分でコントロールできます。

実装しない場合の影響: 不快な投稿をミュートやブロック以外の方法で回避できず、ユーザー体験が低下します。

7.11.11 ContactInquiryモデル（お問い合わせ）

未ログインユーザーも含めたお問い合わせを管理するモデルです。

```
// prisma/schema.prisma より抜粋

model ContactInquiry {
  id          String   @id @default(cuid())
  name        String   @db.VarChar(50)
  email       String   @db.VarChar(100)
  category    String   // general, account, bug, feature, premium, other
  subject     String   @db.VarChar(100)
  message     String   @db.Text
  status      String   @default("pending") // pending, in_progress, resolved, closed
  adminNote   String?  @db.Text @map("admin_note")
  respondedAt DateTime? @map("responded_at")
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  @@index([status])
  @@index([createdAt])
  @@map("contact_inquiries")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/contact.ts`

お問い合わせ送信:

```
// lib/actions/contact.ts

export async function submitContactInquiry(data: {
  name: string
  email: string
  category: string
  subject: string
  message: string
}) {
  const inquiry = await prisma.contactInquiry.create({
    data: {
      name: data.name.trim(),
      email: data.email.trim(),
      category: data.category,
      subject: data.subject.trim(),
      message: data.message.trim(),
    },
  })

  // 送信者へ確認メール + 管理者へ通知メールを送信
  // ...

  return { success: true }
}
```

この実装で可能になること: ログインしていないユーザーも含めて、バグ報告や機能リクエストを送信できます。

実装しない場合の影響: ユーザーからのフィードバック収集手段がなくなり、サービス改善が困難になります。

7.11.12 UserDeviceモデル（デバイス履歴）

ユーザーのデバイスフィンガープリントを記録し、不正登録の検出に使用するモデルです。

```
// prisma/schema.prisma より抜粋

model UserDevice {
  id          String   @id @default(cuid())
  userId      String   @map("user_id")
  fingerprint String    // デバイスフィンガープリント（ハッシュ）
  userAgent   String?  @map("user_agent") @db.Text
  ipAddress   String?  @map("ip_address")
  lastSeenAt  DateTime @default(now()) @map("last_seen_at")
  createdAt   DateTime @default(now()) @map("created_at")

  @@unique([userId, fingerprint]) // 同じデバイスは1ユーザーにつき1レコード
  @@index([userId])
  @@index([fingerprint])
  @@map("user_devices")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/blacklist.ts`

設計ポイント:

- `@@unique([userId, fingerprint])` で同じユーザーの同じデバイスが重複登録されるのを防ぎます
- `fingerprint` へのインデックスにより、デバイスブラックリスト照合が高速に行えます
- `lastSeenAt` でデバイスの最終使用日時を記録し、アクティブなデバイスを把握できます

この実装で可能になること: ブロックされたユーザーが新しいアカウントを作成しても、同じデバイスから登録できないようになります。

実装しない場合の影響: 悪質なユーザーがアカウントを何度も作り直して嫌がらせを続けることを防げません。

7.11.13 CommentThreadMuteモデル（コメントスレッドミュート）

特定のコメントスレッドの通知をミュートする機能です。

```
// prisma/schema.prisma より抜粋

model CommentThreadMute {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  commentId   String    @map("comment_id")
  createdAt   DateTime  @default(now()) @map("created_at")

  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  comment     Comment   @relation(fields: [commentId], references: [id], onDelete: Cascade)

  @@unique([userId, commentId])
  @@map("comment_thread_mutes")
}
```

使用ファイル: `prisma/schema.prisma`, `lib/actions/comment-thread-mute.ts`

設計ポイント:

- コメントへの返信通知が多くなった場合に、特定のスレッドだけ通知を止められます
- ユーザーミュートとは異なり、そのユーザーの他のコメントの通知は引き続き受け取れます

この実装で可能になること: 盛り上がっているコメントスレッドの通知を個別にミュートでき、通知疲れを防げます。

実装しない場合の影響: 活発なディスカッションのコメントスレッドで通知が大量に届き、重要な通知が埋もれてしまいます。

7.11.14 メディア・ジャンルのサブモデル一覧

投稿、予約投稿、下書きにはそれぞれメディアとジャンルの中間テーブルがあります。すべて同じ構造パターンを共有しています。

```
// prisma/schema.prisma より抜粋

// — 予約投稿のメディア —
model ScheduledPostMedia {
  id          String    @id @default(cuid())
  scheduledPostId String @map("scheduled_post_id")
  url         String
  type        MediaType @default(image)
  sortOrder   Int       @default(0) @map("sort_order")

  scheduledPost ScheduledPost @relation(fields: [scheduledPostId], references: [id], onDelete: Cascade)

  @@index([scheduledPostId])
  @@map("scheduled_post_media")
}

// — 予約投稿のジャンル —
model ScheduledPostGenre {
  scheduledPostId String @map("scheduled_post_id")
  genreId          String @map("genre_id")

  scheduledPost ScheduledPost @relation(fields: [scheduledPostId], references: [id], onDelete: Cascade)
  genre          Genre        @relation(fields: [genreId], references: [id], onDelete: Cascade)

  @@id([scheduledPostId, genreId])
  @@map("scheduled_post_genres")
}

// — 下書きのメディア —
model DraftPostMedia {
  id          String    @id @default(cuid())
  draftPostId String    @map("draft_post_id")
  url         String
  type        MediaType @default(image)
  sortOrder   Int       @default(0) @map("sort_order")

  draftPost DraftPost @relation(fields: [draftPostId], references: [id], onDelete: Cascade)

  @@index([draftPostId])
  @@map("draft_post_media")
}

// — 下書きのジャンル —
model DraftPostGenre {
  draftPostId String @map("draft_post_id")
  genreId      String @map("genre_id")

  draftPost DraftPost @relation(fields: [draftPostId], references: [id], onDelete: Cascade)
  genre      Genre    @relation(fields: [genreId], references: [id], onDelete: Cascade)

  @@id([draftPostId, genreId])
  @@map("draft_post_genres")
}
```

なぜ同じ構造のテーブルを複数作るのか？:

テーブル	親テーブル	用途
<code>PostMedia</code>	<code>Post</code>	公開済みの投稿メディア
<code>ScheduledPostMedia</code>	<code>ScheduledPost</code>	予約投稿のメディア
<code>DraftPostMedia</code>	<code>DraftPost</code>	下書きのメディア
<code>CommentMedia</code>	<code>Comment</code>	コメントのメディア

設計上、各状態（下書き、予約中、公開済み）のデータを別テーブルで管理することで、公開済みの `posts` テーブルに未完成のデータが混入するのを防いでいます。下書きから投稿への変換時には、`DraftPostMedia` のデータを `PostMedia` にコピーし、元の下書きデータはCascade削除されます。

この実装で可能になること: 下書き・予約投稿・公開投稿それぞれに独立したメディア管理ができます。

実装しない場合の影響: メディア管理が複雑になり、下書きのメディアと公開済みのメディアが混在してデータ整合性が崩れます。

► 理解度チェック: 未カバーモデル

7.12 高度なPrismaパターン

このセクションで学ぶこと

- `groupBy`を使った集計クエリ
- `$queryRaw`による生SQLの実行
- `updateMany`による一括更新
- `createMany`によるバッチインサート
- `upsert`（あれば更新、なければ作成）
- `connectOrCreate`パターン

7.7節では基本的なCRUD操作を学びましたが、実際のアプリケーションではより高度なクエリパターンが必要になります。ここでは、BON-LOGで実際に使われている高度なPrismaパターンを紹介します。

7.12.1 groupBy（集計クエリ）

`groupBy` は、SQLの `GROUP BY` に相当するPrismaの集計機能です。特定のフィールドでレコードをグループ化し、各グループの統計を計算します。

実例: トレンドジャンルの取得

BON-LOGでは、過去48時間で投稿が多いジャンルを「トレンドジャンル」として表示しています。

```
// lib/cache.ts より抜粋（トレンドジャンルの集計）

// 過去48時間の基準日時を計算
const fortyEightHoursAgo = new Date()
fortyEightHoursAgo.setHours(fortyEightHoursAgo.getHours() - 48)

// ジャンルごとの投稿数を集計
const trendingGenres = await prisma.postGenre.groupBy({
  by: ['genreId'],           // genreIdでグループ化
  where: {
    post: {
      createdAt: { gte: fortyEightHoursAgo }, // 48時間以内の投稿に絞る
    },
  },
  _count: {
    genreId: true,           // 各グループのレコード数をカウント
  },
  orderBy: {
    _count: {
      genreId: 'desc',       // 多い順にソート
    },
  },
  take: 10,                 // 上位10件のみ取得
})

// 結果例:
// [
//   { genreId: 'g1', _count: { genreId: 45 } }, // 松柏類: 45投稿
//   { genreId: 'g2', _count: { genreId: 32 } }, // 雑木類: 32投稿
//   { genreId: 'g3', _count: { genreId: 18 } }, // 道具: 18投稿
// ]
```

groupByの動作イメージ:

post_genres テーブル（入力）:

postId	genreId
p1	g1
p2	g1
p3	g2
p4	g1
p5	g2

GROUP BY genreId →

集計結果（出力）：

genreId	count
g1	3
g2	2

実例: 通報種類ごとの集計

```
// lib/actions/report.ts より抜粋

// 対象種類ごとの通報数を集計
const reportsByType = await prisma.report.groupBy({
  by: ['targetType'], // targetType ('post', 'comment', 'user'等) でグループ化
  _count: true,       // 各グループのレコード数をカウント
})
console.log(reportsByType)
// 実行結果（グループごとのオブジェクト配列）：
// [
//   { targetType: 'post',    _count: { _all: 23 } },
//   { targetType: 'comment', _count: { _all: 8 } },
//   { targetType: 'user',    _count: { _all: 5 } },
// ]
// → targetType でグループ化され、各グループのレコード数が _count に入る
```

実例: いいね数でのフィルタリング（having句）

```
// lib/actions/search.ts より抜粋

// いいね数が一定以上の投稿を検索
const likesResult = await prisma.like.groupBy({
  by: ['postId'],
  _count: { postId: true },
  having: {
    postId: {
      _count: { gte: minLikes }, // いいね数がminLikes以上のグループのみ
    },
  },
})
// 実行結果 (minLikes = 10 の場合) :
// [
//   { postId: 'cm1post042...', _count: { postId: 25 } },
//   { postId: 'cm1post017...', _count: { postId: 12 } },
// ]
// → いいね数が10未満の投稿はフィルタされて結果に含まれない

// havingはWHEREと似ているが、集計後のフィルタリングに使う
// WHERE: グループ化前にフィルタ (個別レコードに対して)
// HAVING: グループ化後にフィルタ (集計結果に対して)
```

7.12.2 \$queryRaw (生SQL)

Prismaのクエリビルダーでは表現できない複雑なクエリには、`$queryRaw` で生のSQLを実行できます。BON-LOGでは全文検索機能で使用しています。

```
// lib/search/fulltext.ts より抜粋

// PostgreSQLの拡張機能が利用可能かチェック
const result = await prisma.$queryRaw<{ available: boolean }[]>`
  SELECT EXISTS(
    SELECT 1 FROM pg_extension WHERE extname = ${extension}
  ) as available
`
// テンプレートリテラル内の ${extension} は自動的にパラメータ化される
// → SQLインジェクション対策済み
```

`$queryRaw` の使い分け:

	Prismaクエリ	\$queryRaw
用途	通常のCRUD操作、リレーション取得、フィルタ・ソート、ページネーション	複雑なJOIN、全文検索（pg_trgm等）、PostgreSQL固有機能、パフォーマンス最適化
型安全	型安全、自動補完あり	型はジェネリクスで指定
使用頻度	ほとんどのケースで十分	特殊なケースのみ使用

全文検索の実例:

```
// lib/search/fulltext.ts より抜粋

// pg_trgm を使った類似度検索（日本語対応の全文検索）
const postIds = await prisma.$queryRaw<{ id: string }[]>`
  SELECT p.id
  FROM posts p
  WHERE p.is_hidden = false
  AND (
    p.content % ${escapedQuery}          -- 類似度が閾値を超えるか
    OR p.content ILIKE '%' || ${escapedQuery} || '%' -- 部分一致も含む
  )
  ${excludedUserIds.length > 0
    ? Prisma.sql`AND p.user_id NOT IN (${Prisma.join(excludedUserIds)})`
    : Prisma.empty}
  ORDER BY similarity(p.content, ${escapedQuery}) DESC -- 類似度順
  LIMIT ${limit}
`
```

GINインデックスの作成:

```
// lib/search/fulltext.ts より抜粋

// $executeRaw: データを返さないSQL（DDL文など）に使用
await prisma.$executeRaw`
  CREATE INDEX IF NOT EXISTS posts_content_trgm_idx
  ON posts USING gin (content gin_trgm_ops)
`

// CREATE INDEX: インデックスを作成するDDL文
// IF NOT EXISTS: 既にあればスキップ
// USING gin: GIN（汎用転置インデックス）方式
// gin_trgm_ops: pg_trgm用の演算子クラス
```

\$queryRawと\$executeRawの違い: `$queryRaw` はSELECT文のようにデータを返すクエリに使用します。
`$executeRaw` はCREATE INDEXやUPDATE文のようにデータを返さないクエリに使用します。

注意: `$queryRawUnsafe` と `$executeRawUnsafe` は文字列を直接SQLとして実行するため、SQLインジェクションのリスクがあります。使用する場合は、必ず入力値を事前にバリデーションしてください。

7.12.3 updateMany（一括更新）

`updateMany` は、条件に一致する複数のレコードを一度に更新します。

```
// lib/premium.ts より抜粋

// プレミアム期限切れユーザーを一括で無料プランに戻す
export async function checkPremiumExpiry(): Promise<number> {
  const result = await prisma.user.updateMany({
    where: {
      isPremium: true,
      premiumExpiresAt: {
        lt: new Date(), // lt: less than = 現在時刻より前 = 期限切れ
      },
    },
    data: {
      isPremium: false, // プレミアムフラグを無効化
    },
  })
  // 実行結果: { count: 3 } (更新されたレコード数のみ返る)

  return result.count // 返却値: 3
}
```

`updateMany` の動作:

更新前:

id	isPremium	premiumExpiresAt	判定
u1	true	2024-02-28	期限切れ → 更新対象
u2	true	2024-04-30	まだ有効
u3	true	2024-01-15	期限切れ → 更新対象
u4	false	null	非プレミアム

更新後（現在日時 = 2024-03-15 とする）：

id	isPremium	premiumExpiresAt	結果
u1	false	2024-02-28	更新された
u2	true	2024-04-30	変化なし
u3	false	2024-01-15	更新された
u4	false	null	変化なし

```
result.count = 2
```

update との違い：

- **update** : 1件のみ更新。更新後のレコード全体を返す。対象が見つからないとエラー
- **updateMany** : 条件に一致する全件を更新。 `{ count: N }` のみ返す。0件でもエラーにならない

7.12.4 createMany（バッチインサート）

createMany は、複数のレコードを1回のクエリで一括挿入します。ループで1件ずつ **create** するよりも大幅に高速です。

```
// lib/actions/blacklist.ts より抜粋

// デバイスフィンガープリントを一括でブラックリストに追加
const result = await prisma.deviceBlacklist.createMany({
  data: newDevices.map((d) => ({
    fingerprint: d.fingerprint,
    reason: reason || `ユーザー ${userId} のデバイスを一括ブロック`,
    originalEmail: user?.email || null,
    createdBy: session.user.id,
  })),
})
// 実行結果: { count: 5 } (作成されたレコード数のみ返る)
// ※ createMany は create と違い、作成されたレコードの中身は返らない
```

createMany vs ループ create の比較:

❌ ループで1件ずつcreate (N回のクエリ → 遅い) :

```
for (const device of newDevices) {
  await prisma.deviceBlacklist.create({ data: { ... } }) // 毎回DBに問い合わせ
}
// 10件 → 10回のDBクエリ → 約100ms
```

✅ createManyで一括挿入 (1回のクエリ → 速い) :

```
await prisma.deviceBlacklist.createMany({
  data: newDevices.map(d => ({ ... })))
})
// 10件 → 1回のDBクエリ → 約10ms
```

パフォーマンス差: 件数が増えるほど顕著

レコード数	ループcreate	createMany
10件	約100ms	約10ms
100件	約1,000ms	約20ms
1,000件	約10,000ms	約50ms

createManyの制限: `createMany` はリレーション先のデータを同時に作成できません (ネストした `create` は不可)。リレーション先も含めて作成する場合は、トランザクション内で `create` を使います。

7.12.5 upsert (あれば更新、なければ作成)

`upsert` は「Update + Insert」の造語で、レコードが存在すれば更新し、存在しなければ作成する操作です。

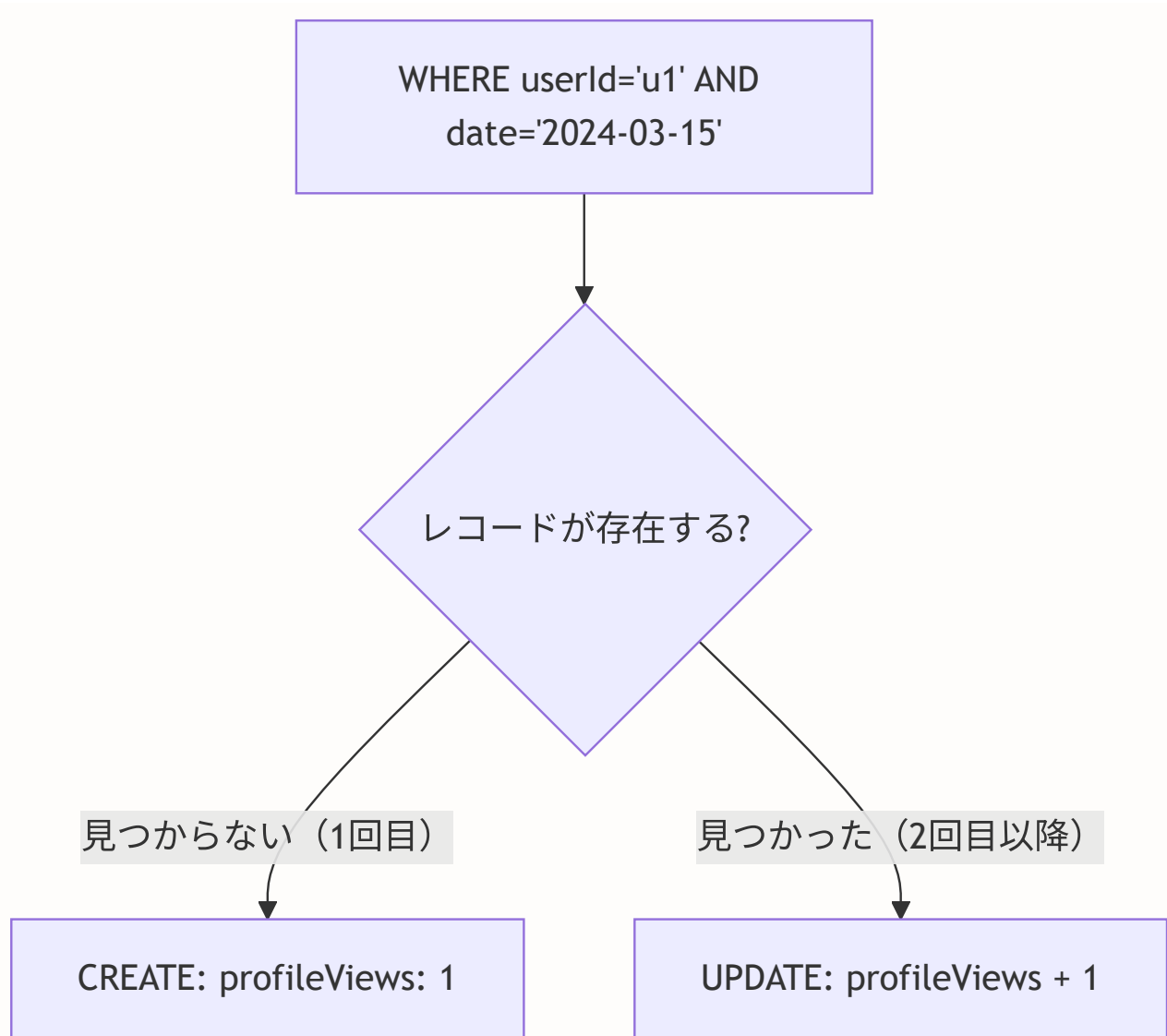
```
// lib/actions/analytics.ts より抜粋

// プロフィール閲覧を記録
const today = new Date()
today.setHours(0, 0, 0, 0) // 今日の0時を基準にする

const analytics = await prisma.userAnalytics.upsert({
  where: {
    userId_date: { userId, date: today }, // 複合ユニークキーで検索
  },
  update: {
    profileViews: { increment: 1 }, // 既存レコード → profileViewsを+1
  },
  create: {
    userId,
    date: today,
    profileViews: 1, // 新規レコード → profileViews=1で作成
  },
})

// 実行結果（1回目のアクセス = create が実行された場合）：
// {
//   id: 'cm1ana001 ... ',
//   userId: 'cm1abc123 ... ',
//   date: 2025-04-15T00:00:00.000Z,
//   profileViews: 1,
//   postViews: 0,
//   likesReceived: 0,
//   newFollowers: 0,
//   createdAt: 2025-04-15T09:30:00.000Z
// }
//
// 実行結果（2回目のアクセス = update が実行された場合）：
// {
//   ... 同じレコード,
//   profileViews: 2, ← 1 → 2 にインクリメントされた
// }
```

upsertの動作フロー：



- 1回目のアクセス: レコードなし → CREATE (profileViews: 1)
- 2回目のアクセス: レコードあり → UPDATE (profileViews: 2)
- 3回目のアクセス: レコードあり → UPDATE (profileViews: 3)

seed.tsでのupsertの活用:

```
// prisma/seed.ts より抜粋

// ジャンルのシードデータ投入
// upsertを使うことで「既にあればスキップ、なければ作成」を実現
for (const genre of postGenres) {
  await prisma.genre.upsert({
    where: {
      id: `${genre.category}-${genre.name}`.replace(/[/\./g, '-'),
    },
    update: genre,    // 既にあれば最新データで更新
    create: {
      id: `${genre.category}-${genre.name}`.replace(/[/\./g, '-'),
      ... genre,      // なければ新規作成
    },
  })
}
// → シードを何回実行しても安全（冪等性がある）
```

upsertの利点（冪等性）： `upsert` を使うと、同じ操作を何回実行しても結果が同じになります（冪等性）。シードデータの投入やアナリティクスの記録など、「重複を気にせず安全に実行したい」場面で重宝します。

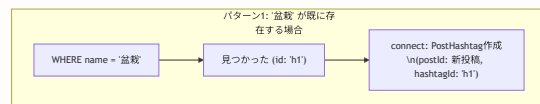
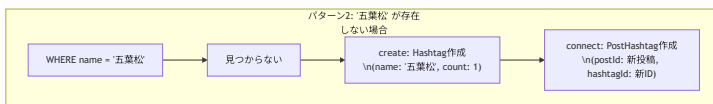
7.12.6 connectOrCreate

`connectOrCreate` は、リレーション先のレコードが既に存在すれば接続し、存在しなければ新規作成する操作です。ハッシュタグの管理で特に有効です。

```
// ハッシュタグの connectOrCreate パターン

// 投稿作成時にハッシュタグを関連付け
const post = await prisma.post.create({
  data: {
    userId: session.user.id,
    content: '五葉松の植え替え #盆栽 #五葉松',
    hashtags: {
      create: [
        {
          hashtag: {
            // "盆栽" タグが既にあれば接続、なければ作成
            connectOrCreate: {
              where: { name: '盆栽' },
              create: { name: '盆栽', count: 1 },
            },
          },
        },
        {
          hashtag: {
            connectOrCreate: {
              where: { name: '五葉松' },
              create: { name: '五葉松', count: 1 },
            },
          },
        },
      ],
    },
  },
})
```

connectOrCreate の動作:



7.12.7 aggregate (集計関数)

aggregate はSQLの集計関数 (SUM, AVG, MIN, MAX, COUNT) に対応する機能です。テーブル全体またはフィルタ後のレコードに対して数値的な集計を行います。

```

// lib/actions/analytics.ts

// ユーザーの統計情報を取得
export async function getUserStats(userId: string) {
  // ① 投稿に対する集計
  const postStats = await prisma.post.aggregate({
    where: {
      userId,
      isHidden: false,
    },
    _count: true, // レコード数 (= 投稿数)
  })

  // ② いいねの集計 (そのユーザーの投稿が受け取ったいいね数)
  const likeStats = await prisma.like.aggregate({
    where: {
      post: {
        userId, // リレーション先のuserIdでフィルタ
      },
    },
    _count: true, // 合計いいね数
  })

  // ③ 盆栽園レビューの評価平均
  const reviewStats = await prisma.shopReview.aggregate({
    where: {
      userId,
      isHidden: false,
    },
    _avg: {
      rating: true, // ratingの平均値
    },
    _min: {
      rating: true, // 最低評価
    },
    _max: {
      rating: true, // 最高評価
    },
    _count: true, // レビュー数
  })

  return {
    totalPosts: postStats._count,
    totalLikesReceived: likeStats._count,
    reviewCount: reviewStats._count,
    avgRating: reviewStats._avg.rating
      ? Math.round(reviewStats._avg.rating * 10) / 10 // 小数第1位に丸める
      : null,
    minRating: reviewStats._min.rating,
    maxRating: reviewStats._max.rating,
  }
}

// 結果例:
// {
//   totalPosts: 42,

```

```
// totalLikesReceived: 156,
// reviewCount: 8,
// avgRating: 4.2,
// minRating: 3,
// maxRating: 5,
// }
```

aggregate の集計関数一覧:

関数	説明
_count	レコード数をカウント
_sum	数値カラムの合計値
_avg	数値カラムの平均値
_min	数値カラムの最小値
_max	数値カラムの最大値

SQLとの対応:

```
prisma.shopReview.aggregate({  →  SELECT
  _avg: { rating: true },      AVG(rating),
  _min: { rating: true },      MIN(rating),
  _max: { rating: true },      MAX(rating),
  _count: true                 COUNT(*)
})                             FROM shop_reviews
```

7.12.8 distinct（重複排除）

`distinct` はSQLの `SELECT DISTINCT` に相当し、指定したフィールドの値が重複するレコードを除外して取得します。


```
// lib/actions/search.ts

// フォロワーが投稿しているジャンル一覧を取得（重複なし）
export async function getFollowingGenres(userId: string) {
  // フォローしているユーザーの投稿に付けられたジャンルIDを、重複なしで取得
  const genres = await prisma.postGenre.findMany({
    where: {
      post: {
        user: {
          followers: {
            some: {
              followerId: userId, // 自分がフォローしているユーザー
            },
          },
        },
      },
    },
    distinct: ['genreId'], // genreIdの重複を除外
    select: {
      genre: {
        select: {
          id: true,
          name: true,
          category: true,
        },
      },
    },
  })

  return genres.map((g) => g.genre)
}

// 実行結果（重複が除去されたジャンルの配列）：
// [
//   { id: 'g1', name: '黒松', category: '松柏類' },
//   { id: 'g2', name: '楓', category: '雑木類' },
//   { id: 'g3', name: '道具', category: '用品・道具' },
// ]
// → distinct がなければ、同じジャンルが投稿数分だけ重複して返る
```

distinctの動作:

post_genres テーブル（フォロー先ユーザーの投稿のみ）:

postId	genreId	ジャンル	備考
p1	g1	黒松	
p2	g1	黒松	重複
p3	g2	楓	
p4	g1	黒松	重複
p5	g3	道具	

distinct: ['genreId'] 適用後:

genreId	ジャンル	備考
g1	黒松	1件目のみ残る
g2	楓	
g3	道具	

7.12.9 ネストした書き込み（Nested Writes）

Prismaの強力な機能の1つが、リレーション先のデータを同時に作成・更新・削除する「ネストした書き込み」です。複数のテーブルにまたがる操作を1回のクエリでまとめて実行できます。

```

// lib/actions/bonsai.ts

// 盆栽と成長記録を同時に作成する
export async function createBonsaiWithRecord(
  userId: string,
  data: {
    name: string
    species?: string
    description?: string
    recordContent: string
    recordImages: string[]
  }
) {
  const bonsai = await prisma.bonsai.create({
    data: {
      userId,
      name: data.name,
      species: data.species,
      description: data.description,
      // ネストしたcreate: Bonsaiと同時にBonsaiRecordも作成
      records: {
        create: {
          content: data.recordContent,
          recordAt: new Date(),
          // さらにネスト: BonsaiRecordと同時にBonsaiRecordImageも作成
          images: {
            create: data.recordImages.map((url, index) => ({
              url,
              sortOrder: index,
            })),
          },
        },
      },
    },
    // 作成結果にリレーション先も含める
    include: {
      records: {
        include: { images: true },
      },
    },
  })

  return bonsai
}

```

ネスト作成の階層構造:

```
prisma.bonsai.create({
  data: {
    name: '五葉松 1号',           ← Level 1: Bonsai
    records: {
      create: {
        content: '植え替え完了', ← Level 2: BonsaiRecord
        images: {
          create: [
            { url: '...jpg' }, ← Level 3: BonsaiRecordImage
          ]
        }
      }
    }
  }
})
```

Prismaが生成するSQL (簡略化) :

1. INSERT INTO bonsais (id, user_id, name, ...) VALUES (...)
 2. INSERT INTO bonsai_records (id, bonsai_id, content, ...) VALUES (...)
 3. INSERT INTO bonsai_record_images (id, record_id, url, ...) VALUES (...)
- 3回のINSERTを自動的に正しい順序で実行
→ 外部キー (bonsai_id, record_id) も自動的に設定

ネスト書き込みで使えるキーワード:

キーワード	説明
create	新規レコードを作成
createMany	複数レコードを一括作成
connect	既存レコードをリレーションに接続
connectOrCreate	あれば接続、なければ作成
disconnect	リレーションの切断
set	リレーションを置き換え
update	リレーション先を更新
updateMany	リレーション先を一括更新
delete	リレーション先を削除
deleteMany	リレーション先を一括削除
upsert	あれば更新、なければ作成

7.12.10 バッチ処理のパターン

大量のデータを処理する場合、全件を一度に処理するのではなく、バッチ（小分け）に分けて処理することで、メモリ使用量を抑え、データベースへの負荷を分散できます。

```

// lib/actions/batch.ts

// 古い通知を一括削除するバッチ処理
export async function cleanupOldNotifications(
  daysToKeep: number = 90 // 何日より前の通知を削除するか
) {
  const cutoffDate = new Date()
  cutoffDate.setDate(cutoffDate.getDate() - daysToKeep)

  // バッチサイズ: 一度に処理する件数
  const BATCH_SIZE = 1000
  let totalDeleted = 0

  // deleteMany を繰り返し実行して、バッチで削除
  while (true) {
    const result = await prisma.notification.deleteMany({
      where: {
        createdAt: { lt: cutoffDate }, // 指定日より前
        isRead: true, // 既読のみ削除
      },
      // Note: deleteManyには直接的なlimitがないため、
      // 実際にはfindManyで先にIDを取得してから削除する方法を使う
    })

    totalDeleted += result.count

    // 削除件数がバッチサイズ未満なら、全件削除完了
    if (result.count < BATCH_SIZE) break
  }

  return { totalDeleted }
}

// より精密なバッチ処理: IDを先に取得してから削除
export async function cleanupOldNotificationsBatch(daysToKeep: number = 90) {
  const cutoffDate = new Date()
  cutoffDate.setDate(cutoffDate.getDate() - daysToKeep)

  const BATCH_SIZE = 500
  let totalDeleted = 0

  while (true) {
    // ① 削除対象のIDを取得
    const idsToDelete = await prisma.notification.findMany({
      where: {
        createdAt: { lt: cutoffDate },
        isRead: true,
      },
      select: { id: true }, // IDのみ取得 (メモリ節約)
      take: BATCH_SIZE,
    })

    if (idsToDelete.length === 0) break // もう対象がない

    // ② IDを指定して削除
    const result = await prisma.notification.deleteMany({

```

```

    where: {
      id: { in: idsToDelete.map((n) => n.id) },
    },
  })

  totalDeleted += result.count
  console.log(`${totalDeleted}件削除済み ... `)

  // 小休止 (DBへの負荷を分散)
  await new Promise((resolve) => setTimeout(resolve, 100))
}

return { totalDeleted }
}

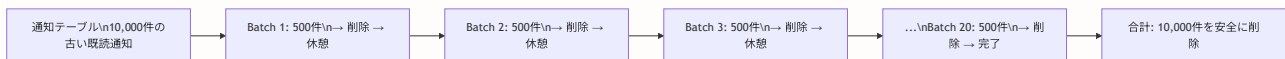
```

バッチ処理の流れ:

古い通知 10,000件を削除する場合

一括削除（危険）: `deleteMany({ where: { ... } })` → 10,000件を一度に削除 → DBロックが長時間継続 → 他のクエリがブロックされる

バッチ削除（安全）:



短いロックが20回 → 他のクエリへの影響が最小限

なぜ休憩（await setTimeout）を入れるのか: バッチ処理で削除を連続実行すると、データベースのI/O負荷が集中します。100msの小休止を入れることで、他のユーザーからの通常リクエスト（投稿の閲覧、いいね等）が処理される余地を確保します。

► 理解度チェック: 高度なPrismaパターン

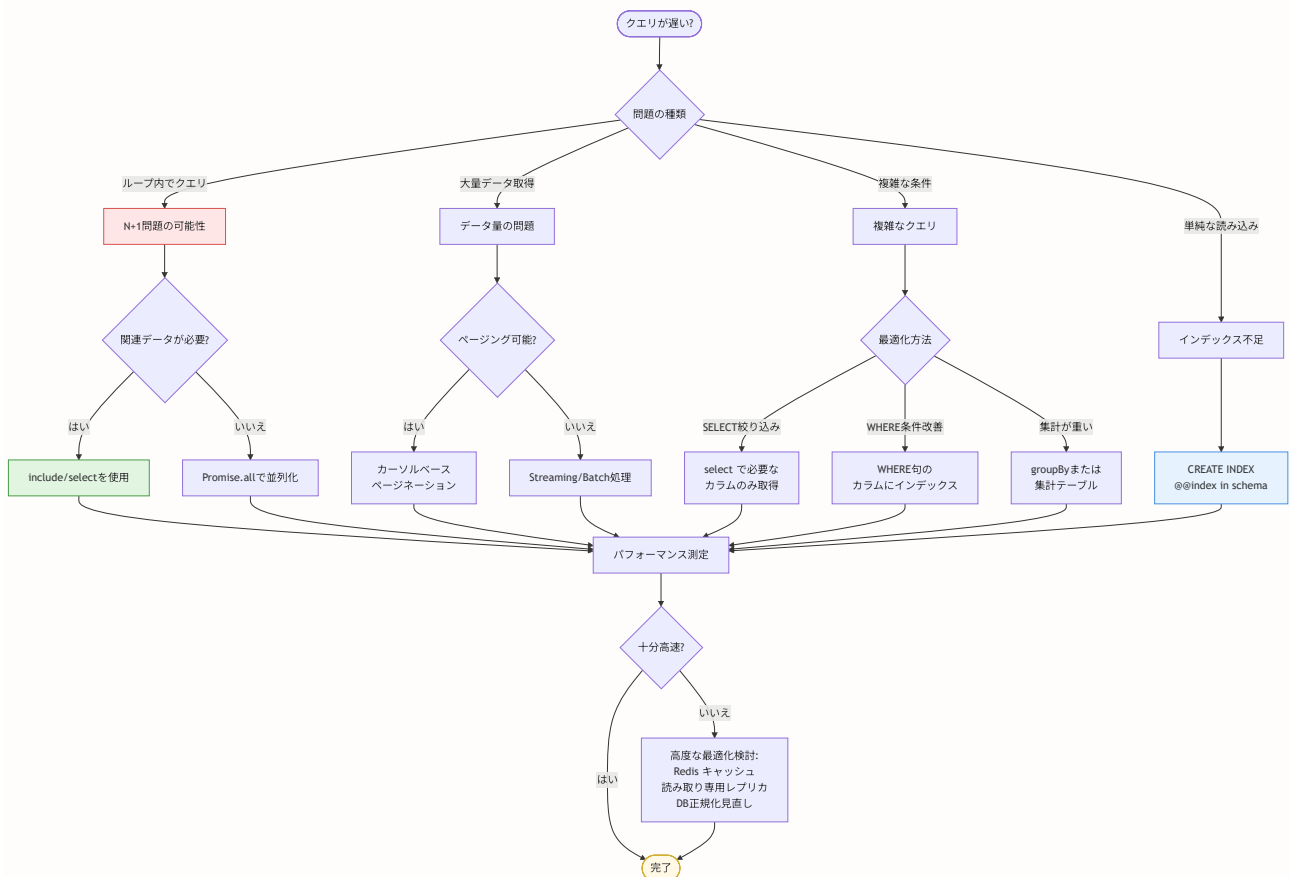
7.13 データベース最適化

このセクションで学ぶこと

- N+1問題とは何か、どう解決するか
- インデックス設計の考え方と実践
- コネクションプーリングの仕組み
- クエリパフォーマンスの改善手法

アプリケーションが成長してユーザーやデータが増えると、データベースのパフォーマンスが重要になります。ここでは、BON-LOGで実際に対策している最適化手法を解説します。

####クエリ最適化決定フローチャート



最適化の優先順位:

1. **N+1問題の解決** - 最も効果が高い
2. **インデックスの追加** - WHERE/ORDER BY句のカラムに
3. **SELECT句の最適化** - 必要なカラムのみ取得
4. **ページネーション** - 大量データは分割取得

5. キャッシュ導入 - 頻繁に読まれる静的データ

7.13.1 N+1問題と解決策

N+1問題は、データベースパフォーマンスの最も一般的な落とし穴です。初心者が最も陥りやすいミスであり、理解しておくことが非常に重要です。

N+1問題を日常の例えで理解する: あなたがクラスの出席簿を作るとします。まず「クラスの生徒30人の名前一覧」を取得します（1回の問い合わせ）。次に、各生徒の住所を調べるために、1人ずつ教務課に「この生徒の住所は？」と聞きに行きます（30回の問い合わせ）。合計31回の問い合わせです。しかし、最初から「名前と住所の両方が載った名簿」を1回で取得すれば、たった1回の問い合わせで済みます。これがN+1問題の本質です。

N+1問題とは？ 10件の投稿とその作者を取得する場合を考えます：

```
// ❌ N+1問題（11回のクエリが実行される）
const posts = await prisma.post.findMany({ take: 10 }) // 1回目
for (const post of posts) {
  const user = await prisma.user.findUnique({           // 2~11回目
    where: { id: post.userId }
  })
}

// ✅ 解決策: includeで1回のクエリにまとめる（2回のクエリ）
const posts = await prisma.post.findMany({
  take: 10,
  include: { user: true } // JOINで一括取得
})
```

図書館で例えると：N+1は「本のリストをもらい、1冊ずつ書庫に取りに行く」。`include`は「本のリストと一緒に本も持ってきてもらう」。

N+1問題とは:

「投稿一覧を20件取得し、各投稿の投稿者名も表示する」場合

❌ N+1問題が発生するパターン（計21回のクエリ）:

```
1回目: SELECT * FROM posts LIMIT 20           -- 投稿一覧取得
2回目: SELECT * FROM users WHERE id = 'u1'      -- 投稿1の投稿者
3回目: SELECT * FROM users WHERE id = 'u2'      -- 投稿2の投稿者
...
21回目: SELECT * FROM users WHERE id = 'u20'    -- 投稿20の投稿者
```

→ 1回（投稿取得） + N回（各投稿の投稿者取得） = N+1回のクエリ
→ 投稿が100件なら101回のクエリ！

✅ includeで解決（計1回のクエリ）:

```
1回目: SELECT posts.*, users.* FROM posts
      LEFT JOIN users ON posts.user_id = users.id
      LIMIT 20
→ JOINで1回のクエリで全データを取得
```

Prismaでの解決方法:

```
// ❌ N+1問題が発生するコード
const posts = await prisma.post.findMany({ take: 20 })
// この時点では投稿データのみ（投稿者情報なし）

for (const post of posts) {
  // ループ内で毎回クエリが実行される → 20件なら20回のクエリ
  const user = await prisma.user.findUnique({
    where: { id: post.userId },
  })
  console.log(`${user?.nickname}: ${post.content}`)
}
// 合計: 1 + 20 = 21回のクエリ

// ✅ includeで解決（1回のクエリ）
const posts = await prisma.post.findMany({
  take: 20,
  include: {
    user: {
      select: { id: true, nickname: true, avatarUrl: true },
    },
  },
})
// 1回のクエリで投稿と投稿者を同時に取得
for (const post of posts) {
  console.log(`${post.user.nickname}: ${post.content}`)
}
// 合計: 1回のクエリ
```

Promise.allを使った並列取得:

```
// ✅ 独立したデータを並列に取得
const [posts, trendingGenres, userCount] = await Promise.all([
  prisma.post.findMany({ take: 20 }), // 投稿一覧
  getTrendingGenres(),               // トレンドジャンル
  prisma.user.count(),                // ユーザー数
])
// 3つのクエリを同時に実行 → 個別に await するより高速

// ❌ 逐次実行 (遅い)
const posts = await prisma.post.findMany({ take: 20 }) // 50ms
const trending = await getTrendingGenres()             // 30ms
const userCount = await prisma.user.count()             // 10ms
// 合計: 50 + 30 + 10 = 90ms

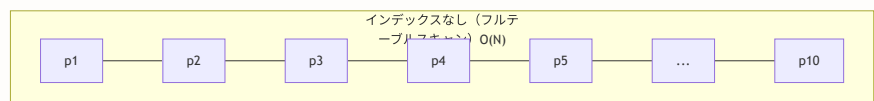
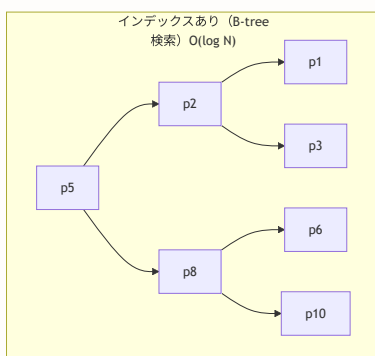
// ✅ 並列実行 (速い)
const [posts, trending, userCount] = await Promise.all([ ... ])
// 合計: max(50, 30, 10) = 50ms (最も遅いクエリの時間のみ)
```

7.13.2 インデックス設計

インデックス (Index) とは? データベースの「索引」です。辞書で「盆栽」という単語を探すとき、1ページ目から順番に読むのではなく、「は行」の索引を見て該当ページにジャンプしますよね。データベースのインデックスも同じ原理で、対象のデータがどこにあるかを素早く見つけるためのデータ構造です。

インデックスは、データベースの「索引」です。本の巻末の索引があれば、全ページを読まなくても目的の情報を見つけれられるのと同じです。

インデックスの効果:



- インデックスなし: 10万件あれば10万件チェック → $O(N)$
- インデックスあり: 10万件でも17回程度のチェック → $O(\log N)$

インデックスの仕組み インデックスなしの検索は、本の全ページを1ページずつ読んで目的の単語を探すようなものです。インデックスありの検索は、巻末の索引で「盆栽 → p.42」と直接ジャンプするようなものです。

データが少ない時（数百件）は差が出ませんが、数万件以上になると検索速度が100倍以上変わることがあります。ただし、インデックスはデータ更新時にも維持する必要があるため、むやみに追加しすぎると書き込みが遅くなります。

BON-LOGのインデックス設計（全一覧）：

BON-LOGでは、合計70以上のインデックスが定義されています。以下にPrismaスキーマ上のインデックスとSQLマイグレーションで追加した全文検索インデックスの両方を網羅します。

Prismaスキーマ上のインデックス (`@@index` / `@@unique`)

```
// prisma/schema.prisma のインデックス定義 (全モデル)

// =====
// 認証系
// =====

model Account {
  @@unique([provider, providerAccountId]) // プロバイダ + アカウントIDの一意性
}

model VerificationToken {
  @@unique([identifier, token]) // 検証トークンの一意性
}

model PasswordResetToken {
  @@unique([email, token]) // パスワードリセットトークンの一意性
}

// =====
// 投稿系
// =====

model Post {
  // 単一カラムインデックス
  @@index([userId]) // ユーザーの投稿一覧取得: WHERE user_id = ?
  @@index([createdAt]) // 時系列ソート: ORDER BY created_at DESC
  @@index([isHidden]) // 非表示フィルタ: WHERE is_hidden = false
  @@index([bonsaiId]) // 盆栽紐づけ投稿: WHERE bonsai_id = ?

  // 複合インデックス (最重要 -- タイムラインクエリ用)
  @@index([userId, isHidden, createdAt(sort: Desc)])
  // ↑ タイムラインクエリ:
  //   WHERE userId IN (...) AND isHidden = false ORDER BY createdAt DESC
  //   → 3カラムを1つのインデックスでカバー (カバリングインデックス)
}

model PostMedia {
  @@index([postId]) // 投稿に紐づくメディア取得
}

model PostGenre {
  @@index([type]) // ジャンルタイプでフィルタ (post/shop)
}

// =====
// コメント系
// =====

model Comment {
  @@index([postId]) // 投稿のコメント一覧: WHERE post_id = ?
  @@index([userId]) // ユーザーのコメント一覧
  @@index([parentId]) // 返信の取得: WHERE parent_id = ?
}
```

```

model CommentMedia {
  @@index([commentId]) // コメントのメディア取得
}

// =====
// いいね・ブックマーク
// =====

model Like {
  @@unique([userId, postId]) // 1ユーザー1投稿に1いいね
  @@unique([userId, commentId]) // 1ユーザー1コメントに1いいね
  @@index([postId]) // 投稿のいいね数取得
  @@index([userId]) // ユーザーがいいねした一覧
}

model Bookmark {
  @@unique([userId, postId]) // 1ユーザー1投稿に1ブックマーク
  @@index([userId]) // ユーザーのブックマーク一覧
}

// =====
// フォロー・ブロック・ミュート
// =====

model Follow {
  @@id([followerId, followingId]) // 複合主キー
  @@index([followerId]) // フォロー中の一覧
  @@index([followingId]) // フォロワー一覧
}

model Block {
  @@id([blockerId, blockedId]) // 複合主キー
  @@index([blockedId]) // ブロックされた側からの検索
}

model Mute {
  @@id([muterId, mutedId]) // 複合主キー
  @@index([mutedId]) // ミュートされた側からの検索
}

model FollowRequest {
  @@unique([requesterId, targetId]) // 重複リクエスト防止
  @@index([requesterId]) // 送信したリクエスト一覧
  @@index([targetId]) // 受信したリクエスト一覧
  @@index([status]) // ステータスでフィルタ: WHERE status = 'pending'
}

// =====
// 通知
// =====

model Notification {
  @@index([userId]) // ユーザーの通知一覧
  @@index([userId, isRead]) // 未読通知の取得（複合インデックス）
  // ↑ WHERE userId = ? AND isRead = false ORDER BY createdAt DESC
  // → 未読通知バッジの表示や、未読通知一覧の取得を高速化

```

```

}

// =====
// 盆栽園・レビュー
// =====

model BonsaiShop {
  @@index([isHidden]) // 公開盆栽園の一覧
  @@index([createdBy]) // 登録者の盆栽園一覧
}

model ShopReview {
  @@index([shopId]) // 盆栽園のレビュー一覧
  @@index([userId]) // ユーザーのレビュー一覧
  @@index([isHidden]) // 公開レビューのフィルタ
}

model ShopReviewImage {
  @@index([reviewId]) // レビューの画像取得
}

model ShopChangeRequest {
  @@index([shopId]) // 盆栽園の変更リクエスト一覧
  @@index([userId]) // ユーザーの変更リクエスト一覧
  @@index([status]) // ステータスでフィルタ
}

// =====
// イベント
// =====

model Event {
  @@index([startDate]) // 開催日でソート/フィルタ
  @@index([prefecture]) // 都道府県でフィルタ
  @@index([isHidden]) // 公開イベントのフィルタ
}

// =====
// 通報
// =====

model Report {
  @@index([targetType, targetId]) // 対象ごとの通報一覧（複合インデックス）
  @@index([status]) // ステータスでフィルタ
}

// =====
// 管理者
// =====

model AdminNotification {
  @@index([isRead]) // 未読通知フィルタ
  @@index([isResolved]) // 未解決通知フィルタ
  @@index([createdAt]) // 時系列ソート
}

// =====

```

```

// 投稿非表示
// =====

model UserHiddenPost {
    @@unique([userId, postId]) // 重複非表示防止
    @@index([userId])          // ユーザーの非表示一覧
}

// =====
// お問い合わせ
// =====

model ContactInquiry {
    @@index([status]) // ステータスでフィルタ
    @@index([createdAt]) // 時系列ソート
}

// =====
// DM (ダイレクトメッセージ)
// =====

model ConversationParticipant {
    @@id([conversationId, userId]) // 複合主キー
    @@index([userId])             // ユーザーの会話一覧
}

model Message {
    @@index([conversationId]) // 会話のメッセージ一覧
}

// =====
// 課金・支払い
// =====

model Payment {
    @@index([userId]) // ユーザーの支払い履歴
}

// =====
// 予約投稿・下書き
// =====

model ScheduledPost {
    @@index([userId]) // ユーザーの予約投稿一覧
    @@index([scheduledAt]) // 公開予定日でソート
    @@index([status]) // ステータスでフィルタ
    @@index([status, scheduledAt]) // Cronジョブの検索最適化（複合インデックス）
    // ↑ WHERE status = 'pending' AND scheduled_at ≤ NOW()
    // → 名前付きインデックス: "scheduled_posts_status_scheduled_at_idx"
    // → テーブルスキャン(1.12秒) → インデックススキャン(数十ms)に高速化
}

model ScheduledPostMedia {
    @@index([scheduledPostId]) // 予約投稿のメディア取得
}

model DraftPost {

```

```

    @@index([userId]) // ユーザーの下書き一覧
}

model DraftPostMedia {
    @@index([draftPostId]) // 下書きのメディア取得
}

// =====
// ハッシュタグ
// =====

model Hashtag {
    @@index([count]) // トレンド取得: ORDER BY count DESC
}

// =====
// 盆栽成長記録
// =====

model Bonsai {
    @@index([userId]) // ユーザーの盆栽一覧
}

model BonsaiRecord {
    @@index([bonsaiId]) // 盆栽の成長記録一覧
    @@index([recordAt]) // 記録日でソート
}

model BonsaiRecordImage {
    @@index([recordId]) // 記録の画像取得
}

// =====
// アナリティクス
// =====

model UserAnalytics {
    @@unique([userId, date]) // 1ユーザー1日1レコード
    @@index([userId]) // ユーザーのアナリティクス一覧
    @@index([date]) // 日付でフィルタ
}

// =====
// デバイス管理
// =====

model UserDevice {
    @@unique([userId, fingerprint]) // 1ユーザー1デバイスの一意性
    @@index([userId]) // ユーザーのデバイス一覧
    @@index([fingerprint]) // フィンガープリントでの検索
}

// =====
// コメントスレッドミュート
// =====

model CommentThreadMute {

```



```

    @@unique([userId, commentId]) // 重複ミュート防止
}

// =====
// 投票機能
// =====

model PollOption {
  @@index([pollId]) // 投票の選択肢取得
}

model PollVote {
  @@unique([pollId, userId]) // 1ユーザー1投票
  @@index([pollId]) // 投票の集計
}

```

全文検索インデックス (pg_trgm GINインデックス)

pg_trgm (トライグラム) とは? テキストを3文字ずつの組み合わせ (tri-gram) に分割して検索する仕組みです。例えば「盆栽」は「盆栽」(2文字だが前後にパディング) のトライグラムに分割されます。これにより、部分一致検索や曖昧検索 (タイポ許容) が高速に行えます。

Prismaの `@@index` ではサポートされない特殊なインデックスは、SQLマイグレーションで直接定義します。

ファイル: `prisma/migrations/20240201000000_add_fts_indexes/migration.sql`

```

-- pg_trgm拡張を有効化（Supabaseでは標準で利用可能）
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 投稿検索用
CREATE INDEX IF NOT EXISTS posts_content_trgm_idx
  ON posts USING gin (content gin_trgm_ops);

-- ユーザー検索用
CREATE INDEX IF NOT EXISTS users_nickname_trgm_idx
  ON users USING gin (nickname gin_trgm_ops);
CREATE INDEX IF NOT EXISTS users_bio_trgm_idx
  ON users USING gin (bio gin_trgm_ops);

-- 盆栽園検索用
CREATE INDEX IF NOT EXISTS bonsai_shops_name_trgm_idx
  ON bonsai_shops USING gin (name gin_trgm_ops);
CREATE INDEX IF NOT EXISTS bonsai_shops_address_trgm_idx
  ON bonsai_shops USING gin (address gin_trgm_ops);

-- イベント検索用
CREATE INDEX IF NOT EXISTS events_title_trgm_idx
  ON events USING gin (title gin_trgm_ops);
CREATE INDEX IF NOT EXISTS events_description_trgm_idx
  ON events USING gin (description gin_trgm_ops);

-- 盆栽成長記録検索用
CREATE INDEX IF NOT EXISTS bonsais_name_trgm_idx
  ON bonsais USING gin (name gin_trgm_ops);
CREATE INDEX IF NOT EXISTS bonsais_species_trgm_idx
  ON bonsais USING gin (species gin_trgm_ops);
CREATE INDEX IF NOT EXISTS bonsais_description_trgm_idx
  ON bonsais USING gin (description gin_trgm_ops);

-- ハッシュタグ検索用
CREATE INDEX IF NOT EXISTS hashtags_name_trgm_idx
  ON hashtags USING gin (name gin_trgm_ops);

-- コメント検索用
CREATE INDEX IF NOT EXISTS comments_content_trgm_idx
  ON comments USING gin (content gin_trgm_ops);

```

GINインデックスの使われ方（Prismaから利用）：

```
// lib/actions/search.ts (実際のコード例)
// pg_trgmのGINインデックスはPrismaの contains/mode:'insensitive' で自動的に使われる

const posts = await prisma.post.findMany({
  where: {
    content: {
      contains: searchQuery,      // pg_trgm GINインデックスが使われる
      mode: 'insensitive',        // 大文字小文字を区別しない
    },
    isHidden: false,
  },
  orderBy: { createdAt: 'desc' },
  take: 20,
})
```

全文検索インデックスの効果:

インデックスなし:

```
SELECT * FROM posts WHERE content ILIKE '%盆栽%'
→ フルテーブルスキャン: 10万件すべてをチェック (数秒)
```

pg_trgm GINインデックスあり:

```
SELECT * FROM posts WHERE content ILIKE '%盆栽%'
→ GINインデックスで候補を絞り込み (数十ms)
→ 日本語でも英語でも高速に検索可能
```

pg_trgm vs pg_bigm (日本語全文検索の選択肢):

項目	pg_trgm	pg_bigm
分割単位	3文字 (tri-gram)	2文字 (bi-gram)
日本語精度	良好 (3文字以上のクエリで高精度)	より高精度 (2文字から対応)
クラウド対応	Supabase/RDS等で標準利用可能	別途インストールが必要
BON-LOGの選択	採用 (Supabase対応のため)	参考実装あり

BON-LOGの選定理由: pg_bigmの方が日本語検索の精度は高いですが、Supabase (本番環境) では標準で利用できないため、pg_trgmを採用しています。 `fulltext_search_indexes.sql` にはpg_bigmの代替実装もコメントアウトで残しており、自前サーバーに移行した場合に切り替え可能です。

インデックス設計のガイドライン:

いつインデックスを作るべきか:

✅ インデックスが効果的なケース:

- WHERE句で頻繁に使われるカラム → `@@index([userId])`
- ORDER BYで使われるカラム → `@@index([createdAt])`
- JOINの結合条件で使われるカラム → 外部キーは自動的にインデックス化
- 複数カラムの組み合わせで検索される → `@@index([userId, createdAt])`

❌ インデックスが逆効果になるケース:

- テーブルのレコード数が少ない (100件以下)
- ほぼ全レコードが条件に一致する (選択性が低い)
- INSERT/UPDATE/DELETEが非常に頻繁 (インデックス更新コスト)

インデックスの代償:

| | 内容 |

├──┤

| メリット | SELECTクエリが高速化 |

| デメリット | INSERT/UPDATE時にインデックスも更新されるため、書き込みが若干遅くなる。ディスク容量を追加で消費する
→ 読み取りが多いSNSアプリでは、適切なインデックスは必須

7.13.3 コネクションプーリング

データベース接続の管理は、本番環境でのパフォーマンスに大きく影響します。

コネクションプーリングとは:

❌ 接続プーリングなし:

リクエスト1 → [接続作成] → DB → [接続破棄]
 リクエスト2 → [接続作成] → DB → [接続破棄] ← 毎回、接続・切断のコスト
 リクエスト3 → [接続作成] → DB → [接続破棄]

✅ 接続プーリングあり:

リクエスト1 → [プールから借りる] → DB → [プールに返す]
 リクエスト2 → [プールから借りる] → DB → [プールに返す] ← 接続を再利用
 リクエスト3 → [プールから借りる] → DB → [プールに返す]

コネクションプール (事前に接続を確保):

| 接続1 | 接続2 | 接続3 | 接続4 |

├──┤├──┤├──┤├──┤

| 使用中 | 空き | 空き | 使用中 |

BON-LOGでの設定:

```
// prisma/schema.prisma

datasource db {
  provider = "postgresql"
  url       = env("DATABASE_URL")    // プーリング経由の接続URL
  directUrl = env("DIRECT_URL")      // 直接接続URL (マイグレーション用)
}
```

SupabaseでのURL使い分け:

DATABASE_URL (プーリング経由):
 postgresql:// ...@aws-0-ap-northeast-1.pooler.supabase.com:6543/postgres
 → アプリケーションの通常クエリ用 (Prisma Clientが使用)
 → 多数の同時接続に対応

DIRECT_URL (直接接続):
 postgresql:// ...@aws-0-ap-northeast-1.supabase.com:5432/postgres
 → マイグレーション用 (Prisma Migrateが使用)
 → スキーマ変更にはDDLロックが必要なため直接接続を使用

Prismaのシングルトンパターン (復習):

BON-LOGではPrisma 6 + PrismaPgアダプターパターンを使用します (詳細は7.7.1参照)。

```
// lib/db.ts (概要: 詳細は7.7.1を参照)
import { PrismaClient } from '@prisma/client'
import { PrismaPg } from '@prisma/adapter-pg'
import { Pool } from 'pg'

const globalForPrisma = global as unknown as { prisma: PrismaClient }

// CI/テスト環境ではPoolを作成しない (ダミーDB判定)
const isDummyDatabase = process.env.DATABASE_URL?.includes('dummy') || false
const pool = isDummyDatabase ? null : new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false,
})

const adapter = pool ? new PrismaPg(pool) : null

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({ ... (adapter && { adapter }) })

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
// → ホットリロードのためにPrismaClientが新規作成されるのを防止
// → Prisma 6ではPrismaPgアダプターが必要 (直接new PrismaClient()は動作しない)
```

7.13.4 クエリパフォーマンスの改善手法

パフォーマンス改善のチェックリスト:

1. `select` で必要なフィールドのみ取得
 - ✗ `prisma.user.findMany()` -- 全カラム取得
 - ✓ `prisma.user.findMany({ select: { id: true, nickname: true } })` -- 必要なカラムのみ
2. `include` で N+1 問題を回避
 - ✗ ループ内で `findUnique` -- N+1 回のクエリ
 - ✓ `findMany` に `include` を付ける -- 1 回の JOIN クエリ
3. 適切なページネーション
 - ✗ `prisma.post.findMany()` -- 全件取得
 - ✓ `prisma.post.findMany({ take: 20, ... })` -- 必要な件数のみ
4. `Promise.all` で並列実行
 - ✗ `const a = await query1()`
`const b = await query2()` -- 逐次実行
 - ✓ `const [a, b] = await Promise.all([query1(), query2()])` -- 並列実行
5. `_count` で集計 (別クエリを避ける)
 - ✗ `const likes = await prisma.like.count({ where: { postId } })` -- 別クエリ
 - ✓ `include: { _count: { select: { likes: true } } }` -- JOIN で取得
6. カーソルベースのページネーション
 - ✗ `skip: pageNumber * pageSize` -- 大きい skip は遅い
 - ✓ `cursor: { id: lastId }, skip: 1` -- インデックスが効く

開発環境でのクエリログ:

```
// lib/db.ts のログ設定

export const prisma = new PrismaClient({
  log: process.env.NODE_ENV === 'development'
    ? ['query', 'error', 'warn'] // 開発環境: 全クエリをログ出力
    : ['error'],                // 本番環境: エラーのみ
})

// 開発環境のターミナル出力例:
// prisma:query SELECT "users"."id", "users"."nickname" FROM "users" WHERE ...
// prisma:query 実行時間: 3ms
// → 遅いクエリ (100ms以上) がないか確認する
```

7.13.5 selectとincludeの使い分け戦略

`select` と `include` は似ていますが、使い分けを間違えるとパフォーマンスに大きな差が出ます。

```
// —— パターン1: includeを使う場合 ——
// 「全フィールド + リレーション先」を取得
const post = await prisma.post.findUnique({
  where: { id: postId },
  include: {
    user: true,          // userの全フィールドを含む
    comments: true,      // commentsの全フィールドを含む
  },
})
// 結果: post.id, post.content, post.createdAt, ... (Post全フィールド)
//       post.user.id, post.user.email, post.user.password, ... (User全フィールド ← 危険!)
//       post.comments[0].id, ... (Comment全フィールド)

// —— パターン2: selectを使う場合 ——
// 「必要なフィールドだけ」を取得
const post = await prisma.post.findUnique({
  where: { id: postId },
  select: {
    id: true,
    content: true,
    createdAt: true,
    user: {
      select: {
        id: true,
        nickname: true,
        avatarUrl: true,
        // ← email, passwordは取得しない
      },
    },
    _count: {
      select: { comments: true, likes: true },
    },
  },
})
// 結果: post.id, post.content, post.createdAt のみ
//       post.user.id, post.user.nickname, post.user.avatarUrl のみ
//       post._count.comments, post._count.likes のみ
```

selectとincludeの使い分けガイド:

状況	推奨
APIレスポンスに使うデータ	select を使う → 不要なフィールドを除外 → パスワード等の機密情報の漏洩を防止
Server Componentで表示するデータ	select を使う → 必要なフィールドのみ取得で高速化 → 転送データ量の削減
内部処理で使うデータ	include でも可 → 全フィールドが必要な場合（例: 投稿の編集）
リレーションのカウン特的み必要	_count を使う → include で全件取得するよりも大幅に高速

パフォーマンス比較（投稿一覧20件取得の場合）：

クエリパターン	転送データ量	実行時間
<code>include: { user: true }</code>	約 50KB	約 15ms
select（必要なフィールドのみ）	約 8KB	約 5ms
include なし	約 3KB	約 3ms

7.13.6 複合インデックスの設計指針

複合インデックスは、複数のカラムを組み合わせたインデックスです。カラムの順序が非常に重要で、間違えるとインデックスが効かなくなります。

```
// prisma/schema.prisma

model Post {
  // ...

  // 複合インデックスの例
  @@index([userId, isHidden, createdAt(sort: Desc)])
}
```


複合インデックスのカラム順序ルール:

インデックス: `@@index([userId, isHidden, createdAt(sort: Desc)])`

このインデックスが効くクエリ:

- ✓ `WHERE userId = 'u1'` (左端のみ)
- ✓ `WHERE userId = 'u1' AND isHidden = false` (左から2つ)
- ✓ `WHERE userId = 'u1' AND isHidden = false ORDER BY createdAt DESC` (左から3つ全て)

このインデックスが効かないクエリ:

- ✗ `WHERE isHidden = false` (左端がない)
- ✗ `WHERE createdAt > '2024-01-01'` (左端がない)
- ✗ `WHERE isHidden = false AND createdAt > '2024-01-01'` (左端がない)

覚え方:

複合インデックスは「電話帳」と同じ。

電話帳は「姓 → 名 → 住所」の順で索引されている。

- 「田中」で検索 → 効く ✓ (左端から)
- 「田中 太郎」で検索 → 効く ✓ (左端から2つ)
- 「太郎」だけで検索 → 効かない ✗ (左端がない)
- 「東京都」だけで検索 → 効かない ✗ (左端がない)

BON-LOGの主要インデックス設計とその根拠:

テーブル	インデックス	主な用途
Post	<code>[userId]</code>	プロフィール画面でユーザーの投稿一覧を取得
Post	<code>[createdAt]</code>	新着順ソート
Post	<code>[userId, isHidden, createdAt(sort: Desc)]</code>	タイムラインの主要クエリ
Follow	<code>[followerId]</code> / <code>[followingId]</code>	フォロワー一覧 / フォロワー一覧
Notification	<code>[userId]</code>	通知一覧取得
ScheduledPost	<code>[status, scheduledAt]</code>	Cronジョブの予約投稿検索
Hashtag	<code>[count]</code>	トレンド表示
Report	<code>[targetType, targetId]</code> / <code>[status]</code>	特定対象の通報 / 未処理通報検索

7.13.7 クエリ分析とデバッグ

開発中に遅いクエリを発見し、原因を特定するためのテクニックです。

Prismaのクエリログを有効化:

```
// lib/db.ts

// 開発環境で全クエリのログを出力する設定
export const prisma = new PrismaClient({
  log: [
    { level: 'query', emit: 'event' }, // クエリをイベントとして発行
    { level: 'error', emit: 'stdout' },
    { level: 'warn', emit: 'stdout' },
  ],
})

// クエリの実行時間をログに出力するイベントリスナー
prisma.$on('query', (e) => {
  // 100ms以上かかったクエリを警告として出力
  if (e.duration > 100) {
    console.warn(`⚠️ 遅いクエリ検出 (${e.duration}ms):`)
    console.warn(`  SQL: ${e.query}`)
    console.warn(`  Params: ${e.params}`)
  }
})
```

クエリログの読み方:

ターミナルに出力される例:

```
prisma:query SELECT "posts"."id", "posts"."content",
  "posts"."user_id", "posts"."created_at"
FROM "posts"
WHERE "posts"."is_hidden" = $1
ORDER BY "posts"."created_at" DESC
LIMIT $2
prisma:query パラメータ: [false, 20]
prisma:query 実行時間: 3ms
```

確認ポイント:

1. 実行時間が100ms以上のクエリがないか
2. 同じクエリが繰り返し実行されていないか (N+1問題の兆候)
3. 不要なカラムを取得していないか
4. LIMITが付いているか (全件取得していないか)

PostgreSQLのEXPLAIN ANALYZEで実行計画を確認:

```
// $queryRawを使って実行計画を確認する
const explain = await prisma.$queryRaw`
  EXPLAIN ANALYZE
  SELECT * FROM posts
  WHERE user_id = 'u1'
  AND is_hidden = false
  ORDER BY created_at DESC
  LIMIT 20
`

console.log(explain)
```

EXPLAIN ANALYZE の出力例と読み方:

```
Limit (cost=0.42..12.85 rows=20 ...)
  → Index Scan Backward using posts_user_id_is_hidden_created_at_idx
     on posts (cost=0.42..50.23 rows=80 ...)
     Index Cond: ((user_id = 'u1') AND (is_hidden = false))
     Actual rows: 20
     Actual time: 0.1ms..0.3ms
```

読み方:

表示	意味
Index Scan	インデックスが使われている（良好）
Seq Scan	全件スキャン（遅い可能性あり）
Actual time	実際の実行時間
Actual rows	実際に返された行数
Index Cond	インデックス条件

「Index Scan」が出ていればインデックスが効いている証拠。「Seq Scan」ばかり出る場合は、インデックスの追加を検討する。

7.13.8 メモリ効率の良いデータ取得

大量のレコードを処理する場合、全件を一度にメモリに読み込むとOutOfMemoryエラーが発生する可能性があります。カーソルベースのページネーションを応用して、少しずつ処理する方法を紹介します。

```

// lib/utils/batch-processor.ts

// カーソルベースで全レコードを順次処理するジェネレータ関数
export async function* iterateAllPosts(batchSize: number = 100) {
  let cursor: string | undefined = undefined

  while (true) {
    // batchSize件ずつ取得
    const posts = await prisma.post.findMany({
      take: batchSize,
      ... (cursor && {
        cursor: { id: cursor },
        skip: 1, // カーソル自体をスキップ
      }),
      orderBy: { id: 'asc' }, // IDの昇順で安定したソート
      select: {
        id: true,
        content: true,
        userId: true,
      },
    })

    // 取得件数が0なら終了
    if (posts.length === 0) break

    // 取得したバッチをyield（呼び出し元に返す）
    yield posts

    // 次のバッチのカーソルを設定
    cursor = posts[posts.length - 1].id

    // 最後のバッチ（batchSize未満）なら終了
    if (posts.length < batchSize) break
  }
}

// 使用例：全投稿を順次処理
async function processAllPosts() {
  let processedCount = 0

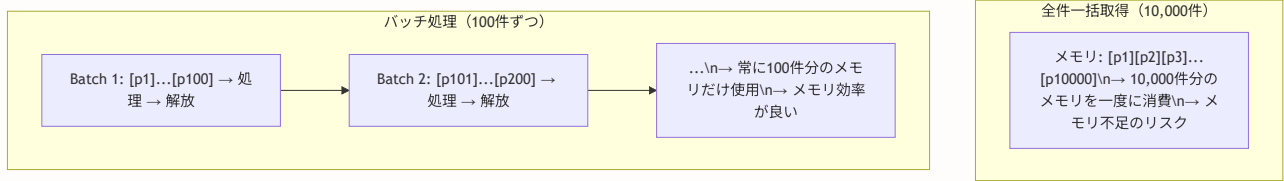
  // for await ... of でジェネレータからバッチを1つずつ受け取る
  for await (const batch of iterateAllPosts(100)) {
    // バッチ内の各投稿を処理
    for (const post of batch) {
      // 何らかの処理（例：ハッシュタグの再抽出）
      await processPost(post)
    }

    processedCount += batch.length
    console.log(`${processedCount}件処理済み ...`)
  }

  console.log(`合計: ${processedCount}件処理完了`)
}

```

カーソルベース順次処理のメモリ使用量:



▶ 理解度チェック: データベース最適化

7.13.9 Prismaのエラーコード一覧

Prismaを使っていると、特定のエラーコードに遭遇することがあります。よく出会うエラーコードとその意味、対処法をまとめます。

Prismaの主要エラーコード:

コード	意味	対処法
P2000	値がカラムに格納可能な範囲を超えている (例: <code>VarChar(50)</code> に51文字以上)	バリデーションで入力値の長さを事前チェック
P2001	WHERE条件で指定したレコードが存在しない (例: <code>findUnique</code> で存在しないID)	<code>findUniqueOrThrow</code> の代わりに <code>findUnique</code> を使い、 <code>null</code> チェックを行う
P2002	ユニーク制約違反 (重複データの挿入)。既に存在するメールアドレスでユーザーを作成した場合、既にいいね済みの投稿に再度いいねした場合	<code>try-catch</code> でキャッチし、適切なエラーメッセージを返す
P2003	外部キー制約違反 (例: 存在しない <code>userId</code> を指定して投稿を作成)	リレーション先のレコードが存在するか事前確認
P2025	操作に必要なレコードが見つからない (例: <code>update/delete</code> で存在しないID)	<code>findUnique</code> で存在確認してから <code>update/delete</code> 、または <code>updateMany/deleteMany</code> を使う (0件でもエラーなし)
P1001	データベースサーバーに接続できない (PostgreSQLが起動していない)	<code>docker compose up -d postgres</code> を実行
P1008	クエリのタイムアウト (非常に重いクエリを実行した場合)	クエリを最適化する (インデックス追加、データ量制限)
P1017	サーバーが接続を閉じた (DBサーバーの再起動や接続タイムアウト)	リトライロジックを実装する

エラーハンドリングの実装パターン:

```

// lib/utils/prisma-error.ts

import { Prisma } from '@prisma/client'

// Prismaのエラーを判別して適切なメッセージを返す関数
export function handlePrismaError(error: unknown): {
  code: string
  message: string
} {
  // PrismaClientKnownRequestError: Prismaが認識するエラー
  if (error instanceof Prisma.PrismaClientKnownRequestError) {
    switch (error.code) {
      case 'P2002': {
        // ユニーク制約違反
        // error.meta.target に違反したフィールド名が入る
        const field = (error.meta?.target as string[])?.join(', ') || '不明'
        return {
          code: 'DUPLICATE',
          message: `${field} は既に使用されています`,
        }
      }
      case 'P2003':
        return {
          code: 'FOREIGN_KEY',
          message: '関連するデータが見つかりません',
        }
      case 'P2025':
        return {
          code: 'NOT_FOUND',
          message: '対象のデータが見つかりません',
        }
      default:
        return {
          code: 'UNKNOWN',
          message: 'データベースエラーが発生しました',
        }
    }
  }

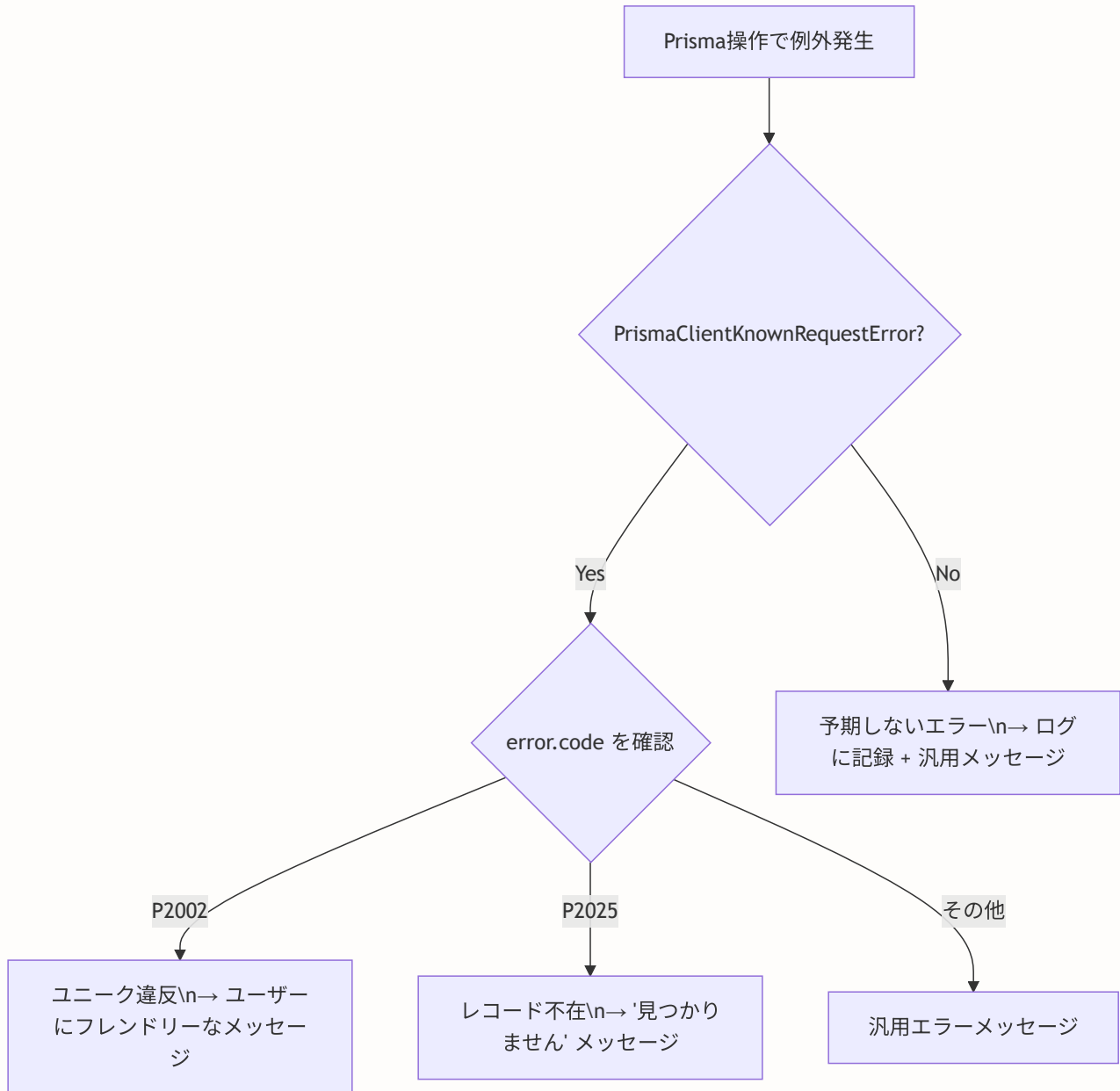
  // その他のエラー
  return {
    code: 'INTERNAL',
    message: '予期しないエラーが発生しました',
  }
}

// Server Actionsでの使用例:
// try {
//   await prisma.like.create({ data: { userId, postId } })
//   return { success: true }
// } catch (error) {
//   const { code, message } = handlePrismaError(error)
//   return { error: message }
//   // 実行結果（既にいいね済みの場合）:
//   // { code: 'DUPLICATE', message: 'userId, postId は既に使用されています' }
// }

```

```
// // 実行結果（存在しない投稿にいいねした場合）：  
// // { code: 'FOREIGN_KEY', message: '関連するデータが見つかりません' }  
// }
```

エラーハンドリングの判断フロー:



重要: ユーザーに技術的な詳細（SQLエラー等）を見せてはいけない

- セキュリティリスク（テーブル構造の漏洩）
- ユーザー体験の低下（意味不明なエラー文）

7.14 シードデータ

このセクションで学ぶこと

- シードデータとは何か、なぜ必要か
- seed.tsの構造と実装パターン
- テストデータ設計のベストプラクティス
- シードの実行方法

7.14.1 シードデータとは

シードデータ（Seed Data）は、アプリケーションの動作に必要な初期データのことです。種（Seed）を蒔くように、データベースに最初のデータを投入する作業を「シーディング」と呼びます。

シードデータが必要なケース：

1. マスタデータ（変更頻度の低い基礎データ）
 - ジャンル一覧（松柏類、雑木類、道具、施設 ...）
 - 都道府県一覧
 - 設定の初期値
2. テスト用データ
 - E2Eテスト用のユーザーアカウント
 - 開発時の動作確認用データ
3. デモデータ
 - 新規開発者がすぐに動作確認できるサンプルデータ

7.14.2 seed.tsの解説

BON-LOGの `prisma/seed.ts` を見ていきましょう。


```
// prisma/seed.ts

import { PrismaClient } from '@prisma/client'
import { PrismaPg } from '@prisma/adapters-pg'
import { Pool } from 'pg'
import bcrypt from 'bcryptjs'

// Prismaクライアントの初期化（シード専用）
const pool = new Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })
```

ジャンルのマスターデータ定義:

```
// 投稿用ジャンル（盆栽の樹種カテゴリ）
const postGenres = [
  // 松柏類
  { name: '黒松', category: '松柏類', type: 'post', sortOrder: 1 },
  { name: '赤松', category: '松柏類', type: 'post', sortOrder: 2 },
  { name: '五葉松', category: '松柏類', type: 'post', sortOrder: 3 },
  { name: '真柏', category: '松柏類', type: 'post', sortOrder: 4 },
  // ... 松柏類は17種、雑木類は38種、その他も含め全62種 ...

  // 雑木類
  { name: '紅葉', category: '雑木類', type: 'post', sortOrder: 1 },
  { name: '楓', category: '雑木類', type: 'post', sortOrder: 2 },
  // ...

  // 草もの
  { name: '山野草', category: '草もの', type: 'post', sortOrder: 1 },
  { name: '苔', category: '草もの', type: 'post', sortOrder: 2 },

  // 用品・道具
  { name: '道具', category: '用品・道具', type: 'post', sortOrder: 1 },
  { name: '鉢', category: '用品・道具', type: 'post', sortOrder: 3 },
  // ...

  // 施設・イベント、その他
  { name: '盆栽園', category: '施設・イベント', type: 'post', sortOrder: 1 },
  { name: '管理方法', category: 'その他', type: 'post', sortOrder: 1 },
]

// 盆栽園用ジャンル（サイズ・用品分類）
const shopGenres = [
  { name: 'ミニ盆栽', category: 'サイズ', type: 'shop', sortOrder: 1 },
  { name: '小品盆栽', category: 'サイズ', type: 'shop', sortOrder: 2 },
  { name: '大品盆栽', category: 'サイズ', type: 'shop', sortOrder: 4 },
  { name: '道具', category: '用品', type: 'shop', sortOrder: 1 },
  { name: '鉢', category: '用品', type: 'shop', sortOrder: 2 },
  // ...
]
```

シードの実行関数:

```

async function main() {
  console.log('Seeding genres ... ')

  // 投稿用ジャンルの投入
  for (const genre of postGenres) {
    await prisma.genre.upsert({
      where: {
        // IDを「カテゴリ-ジャンル名」の形式で生成
        id: `${genre.category}-${genre.name}`.replace(/[/]/g, '-'),
      },
      update: genre, // 既にあれば最新データで更新
      create: {
        id: `${genre.category}-${genre.name}`.replace(/[/]/g, '-'),
        ... genre, // なければ新規作成
      },
    })
  }

  // 盆栽園用ジャンルの投入
  for (const genre of shopGenres) {
    await prisma.genre.upsert({
      where: {
        id: `shop-${genre.category}-${genre.name}`.replace(/[/]/g, '-'),
      },
      update: genre,
      create: {
        id: `shop-${genre.category}-${genre.name}`.replace(/[/]/g, '-'),
        ... genre,
      },
    })
  }

  console.log(`Seeded ${postGenres.length} post genres and ${shopGenres.length} shop genres`)

  // E2Eテスト用ユーザーの作成
  const e2eTestPassword = await bcrypt.hash('TestPassword123!', 10)
  await prisma.user.upsert({
    where: { email: 'e2e-test@example.com' },
    update: {}, // 既にあれば何もしない
    create: {
      email: 'e2e-test@example.com',
      password: e2eTestPassword,
      nickname: 'E2Eテストユーザー',
    },
  })
  console.log('Seeded E2E test user')
}

// 実行とエラーハンドリング
main()
  .catch((e) => {
    console.error(e)
    process.exit(1) // エラー時は終了コード1で終了
  })
  .finally(async () => {

```

```
await prisma.$disconnect() // 必ずDB接続を切断
})
```

seed.tsの設計パターン:

1. upsertを使う（冪等性の確保）\n→ 何回実行しても安全\n→ createだと2回目でユニーク制約エラー



2. IDを決定的に生成\n→ カテゴリ + 名前 からIDを生成\n→ 毎回同じIDが生成される\n→ cuid()だと毎回ランダムIDになる



3. \$disconnectを必ず呼ぶ\n→ .finally() で確実に実行\n→ DB接続が残るとプロセスが終了しない

7.14.3 シードの実行方法

```
# シードデータの投入 (方法1: npx prisma db seed)
npx prisma db seed
# → package.json の "prisma.seed" に指定されたスクリプトを実行

# シードデータの投入 (方法2: 直接実行)
npx tsx prisma/seed.ts

# マイグレーション時に自動実行 (--seed オプション)
npx prisma migrate reset
# → データベースをリセットして、マイグレーションを再適用し、シードを実行
```

実行結果の例:

```
$ npx prisma db seed
Running seed command `tsx prisma/seed.ts` ...
Seeding genres ...
Seeded 62 post genres and 12 shop genres
Seeded E2E test user
```

 The seed command has been executed.

実行結果の確認方法 シード実行後に `npx prisma studio` を開くと、投入されたデータを確認できます。genres テーブルに62件の投稿用ジャンルと12件の盆栽園用ジャンルが表示されます。users テーブルにE2Eテストユーザーが1件表示されます。

```
// package.json のシード設定
{
  "prisma": {
    "seed": "tsx prisma/seed.ts"
  }
}
```

7.14.4 テストデータ設計のベストプラクティス

テストデータ設計のポイント:

1. 本番データとテストデータを明確に区別する
 - ✓ テスト用メール: `e2e-test@example.com` (@example.comドメイン)
 - ✗ 本番に紛れる: `test@gmail.com`
2. パスワードは十分に強固なものを使う
 - ✓ `'TestPassword123!'` (テストでも安全なパスワード)
 - ✗ `'password'` (本番に残った場合のリスク)
3. シードは冪等 (ベキとう) に設計する
 - ✓ `upsert` を使う (何回実行しても同じ結果)
 - ✗ `create` を使う (2回目エラー)
4. マスタデータとテストデータを分離する
 - ✓ ジャンル: 本番でも使うマスタデータ → 常に投入
 - テストユーザー: テスト環境のみ → 条件分岐で投入
5. データ量は必要最小限にする
 - ✓ 動作確認に必要な最小限のデータ
 - ✗ 大量のダミーデータ (シード実行が遅くなる)

7.14.5 開発用ダミーデータの生成パターン

実際の開発では、UIの動作確認やパフォーマンステストのために、ある程度リアルなデータが必要になることがあります。ここでは、`seed.ts`を拡張して開発用データを生成するパターンを紹介します。

```

// prisma/seed-dev.ts (開発環境専用のシードスクリプト)
// ※ 本番のseed.tsと同様にPrismaPgアダプターが必要 (Prisma 6)
import { PrismaClient } from '@prisma/client'
import { PrismaPg } from '@prisma/adapters-pg'
import { Pool } from 'pg'
import bcrypt from 'bcryptjs'

const pool = new Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })

// — ダミーデータ生成用のユーティリティ関数 —

// ランダムな要素を配列から選択する関数
function randomPick<T>(array: T[]): T {
  return array[Math.floor(Math.random() * array.length)]
}

// 指定範囲のランダムな整数を生成する関数
function randomInt(min: number, max: number): number {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

// 盆栽に関連するダミーの投稿テキスト
const sampleContents = [
  '五葉松の植え替えを行いました。根張りが良くなってきています。',
  '黒松の芽切りの季節です。今年は例年より早めに行いました。',
  '展示会で見つけた素晴らしい真柏です。樹齢は推定80年とのこと。',
  '楓の紅葉が見事です。秋の盆栽は本当に美しいですね。',
  '新しく入手した樺の素材です。これからじっくり作り込んでいきます。',
  '皐月が満開になりました。ピンクと白のグラデーションが美しい。',
  '梅の花が咲き始めました。春の訪れを感じます。',
  '赤松の針金かけが完了しました。3年後の姿が楽しみです。',
  '盆栽園で購入した鉢と松の植え替えを行いました。',
  '初めての盆栽に挑戦！まずはミニ盆栽から始めます。',
]

// — メイン処理 —

async function seedDevData() {
  console.log('開発用データの投入を開始 ... ')

  const password = await bcrypt.hash('DevPassword123!', 10)

  // ① 開発用ユーザーを10人作成
  const users = []
  for (let i = 1; i ≤ 10; i++) {
    const user = await prisma.user.upsert({
      where: { email: `dev-user-${i}@example.com` },
      update: {},
      create: {
        email: `dev-user-${i}@example.com`,
        password,
        nickname: `開発ユーザー${i}`,
        bio: `テスト用のユーザー${i}です。盆栽歴${randomInt(1, 30)}年。`,
        isPublic: i ≤ 8, // 8人は公開、2人は非公開
      }
    })
    users.push(user)
  }
}

```

```

    },
  })
  users.push(user)
}
console.log(`${users.length}人の開発用ユーザーを作成`)

// ② ジャンルIDを取得（シードで投入済みのジャンル）
const genres = await prisma.genre.findMany({
  where: { type: 'post' },
  select: { id: true },
})
const genreIds = genres.map((g) => g.id)

// ③ 各ユーザーに投稿を作成
let postCount = 0
for (const user of users) {
  const numPosts = randomInt(3, 10) // 3~10件の投稿
  for (let j = 0; j < numPosts; j++) {
    await prisma.post.create({
      data: {
        userId: user.id,
        content: randomPick(sampleContents),
        // ジャンルをランダムに1~2個紐付け
        genres: {
          create: Array.from(
            { length: randomInt(1, 2) },
            () => ({
              genreId: randomPick(genreIds),
            })
          ),
        },
      },
    })
    postCount++
  }
}
console.log(`${postCount}件の投稿を作成`)

// ④ フォロー関係を作成
let followCount = 0
for (const user of users) {
  // 各ユーザーがランダムに2~5人をフォロー
  const numFollows = randomInt(2, 5)
  const otherUsers = users.filter((u) => u.id !== user.id)

  for (let k = 0; k < Math.min(numFollows, otherUsers.length); k++) {
    const target = otherUsers[k]
    try {
      await prisma.follow.create({
        data: {
          followerId: user.id,
          followingId: target.id,
        },
      })
      followCount++
    } catch {
      // 重複フォローは無視 (@@id制約で弾かれる)
    }
  }
}

```



```

    }
  }
}
console.log(`${followCount}件のフォロー関係を作成`)

// ㊟ いいねを作成
const posts = await prisma.post.findMany({ select: { id: true } })
let likeCount = 0
for (const user of users) {
  // 各ユーザーがランダムに5~15件にいいね
  const numLikes = randomInt(5, 15)
  for (let l = 0; l < Math.min(numLikes, posts.length); l++) {
    const targetPost = randomPick(posts)
    try {
      await prisma.like.create({
        data: {
          userId: user.id,
          postId: targetPost.id,
        },
      })
      likeCount++
    } catch {
      // 重複いいねは無視
    }
  }
}
console.log(`${likeCount}件のいいねを作成`)

console.log('開発用データの投入完了!')
}

seedDevData()
  .catch(console.error)
  .finally(() => prisma.$disconnect())

```

開発用シードの実行:

```

# 基本シード (マスタデータ + テストユーザー)
$ npx prisma db seed

# 開発用データ (上記に加えてダミーの投稿・フォロー・いいね)
$ npx tsx prisma/seed-dev.ts

```

生成されるデータ量:

```

| データ種類 | 件数 |
├───┬───┤
| ユーザー | 10人 |
| 投稿 | 30~100件 |
| フォロー関係 | 20~50件 |
| いいね | 50~150件 |
→ タイムラインやプロフィール画面の動作確認に十分なデータ量

```

7.14.6 環境別のシード制御

開発環境と本番環境で投入するシードデータを切り替えるパターンです。

```
// prisma/seed.ts

async function main() {
  // 全環境共通: マスタデータの投入
  await seedGenres()           // ジャンルマスタ
  await seedSystemSettings()   // システム設定の初期値

  // テスト環境のみ: テストユーザーの投入
  if (process.env.NODE_ENV !== 'production') {
    await seedTestUsers()
  }

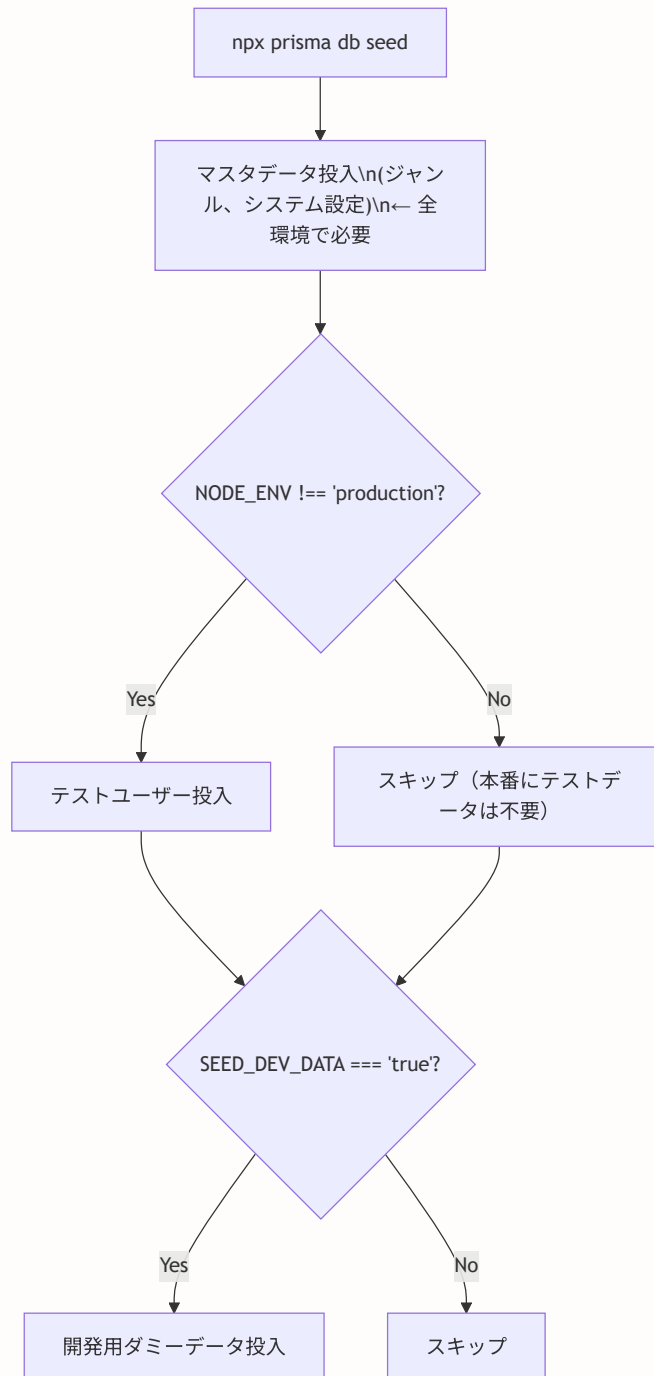
  // 開発環境のみ: ダミーデータの投入
  if (process.env.SEED_DEV_DATA === 'true') {
    await seedDevData()
  }
}

async function seedSystemSettings() {
  // メンテナンスモードのデフォルト値
  await prisma.systemSetting.upsert({
    where: { key: 'maintenance_mode' },
    update: {},
    create: {
      key: 'maintenance_mode',
      value: { enabled: false, message: '' },
    },
  })

  // 投稿制限のデフォルト値
  await prisma.systemSetting.upsert({
    where: { key: 'post_limits' },
    update: {},
    create: {
      key: 'post_limits',
      value: { maxPostsPerDay: 20, maxCommentsPerDay: 100 },
    },
  })

  console.log('システム設定の初期値を投入')
}
```

環境別シード投入の判断:



実行例:

```
# 本番: マスタデータのみ
$ NODE_ENV=production npx prisma db seed

# 開発: マスタ + テストユーザー
$ npx prisma db seed

# 開発フル: マスタ + テストユーザー + ダミーデータ
$ SEED_DEV_DATA=true npx prisma db seed
```

本番環境のシード: 本番環境でシードを実行するタイミングは、初回デプロイ時とマスタデータに変更があったときのみです。 `upsert` を使っているため何回実行しても安全ですが、不要な実行は避けましょう。

▶ 理解度チェック: シードデータ

7.15 本章の重要用語まとめ（復習用）

本章で登場した専門用語を、実際のBON-LOGでの使われ方と合わせて整理します。

データベースの基本用語と、BON-LOGでの実例:

用語	BON-LOGでの実例
リレーショナルデータベース (RDB)	PostgreSQLを使い、テーブル間の関係でユーザー・投稿・いいね等のデータを管理
テーブル	users, posts, likes, follows 等の各テーブル
カラム (列)	id, email, nickname, createdAt 等のフィールド
レコード (行)	「Aliceのユーザー情報1件」「投稿1件」等
主キー (Primary Key)	各テーブルのid (cuid()で自動生成) 例: "clx1abc2d0000xyz..."
外部キー (Foreign Key)	PostのuserId → UserのidでCascade参照。「この投稿はどのユーザーのものか」を示す
インデックス	<code>@@index([userId, createdAt])</code> タイムライン取得クエリ的高速化に使用
スキーマ	prisma/schema.prismaファイルに定義されたテーブル構造全体の設計図
ORM	Prismaが TypeScript <-> SQL の変換を担当。 <code>prisma.user.findMany()</code> → <code>SELECT * FROM users</code>
CRUD	Create(投稿作成), Read(タイムライン表示), Update(プロフィール編集), Delete(投稿削除)
トランザクション	いいね作成 + 通知作成 をセットで実行。 <code>\$transaction([...])</code> で原子性を保証
マイグレーション	<code>prisma migrate dev</code> でスキーマ変更を記録。prisma/migrations/ にSQLファイルが生成される
N+1問題	ループ内でDB問い合わせ → includeで1回に集約。投稿一覧 + 投稿者情報 を1クエリで取得

本章で学んだPrisma操作の早見表:

操作カテゴリ	メソッド	用途
作成 (Create)		
単一作成	<code>.create()</code>	投稿1件を作成
一括作成	<code>.createMany()</code>	複数レコードを一括挿入
読取 (Read)		
ID検索	<code>.findUnique()</code>	投稿IDで1件取得
条件で1件	<code>.findFirst()</code>	最新の1件を取得
複数件取得	<code>.findMany()</code>	タイムライン20件取得
件数取得	<code>.count()</code>	投稿数をカウント
集計	<code>.aggregate()</code>	統計情報を取得
グループ集計	<code>.groupBy()</code>	ジャンル別投稿数を集計
更新 (Update)		
単一更新	<code>.update()</code>	プロフィール編集
一括更新	<code>.updateMany()</code>	期限切れユーザー一括更新
あれば更新/なければ作成	<code>.upsert()</code>	アナリティクス記録
削除 (Delete)		
単一削除	<code>.delete()</code>	投稿1件を削除
一括削除	<code>.deleteMany()</code>	条件に合うレコードを全削除
リレーション		
関連データも取得	<code>include: { ... }</code>	投稿+投稿者情報
必要なカラムだけ	<code>select: { ... }</code>	ID+ニックネームのみ
カウント取得	<code>_count: { ... }</code>	いいね数・コメント数
トランザクション		
配列方式	<code>.\$transaction([])</code>	いいね+通知をセット実行
関数方式	<code>.\$transaction(fn)</code>	条件分岐を含む複数操作
生SQL		
結果を返すSQL	<code>.\$queryRaw` ``</code>	全文検索 (pg_trgm)

結果なしSQL

`.$executeRaw` ``

インデックス作成

7.16 よくある質問（FAQ）

データベース設計やPrisma操作でつまづきやすいポイントを、Q&A形式でまとめました。開発中に困ったときに参照してください。

7.16.1 Prisma全般

Q: `npx prisma generate` と `npx prisma db push` の違いは何ですか？

A: 2つは全く異なるコマンドです。

`npx prisma generate`:

`schema.prisma`ファイルを読み込んで、TypeScriptの型定義（Prisma Client）を生成する。

- `node_modules/.prisma/client/` に型定義が作られる
- データベースには一切触らない
- エディタの補完機能を最新にするために使う

`npx prisma db push`:

`schema.prisma`ファイルの内容を実際のデータベースに反映する。

- テーブルの作成・変更・削除が実行される
- マイグレーションファイルは作成されない
- 開発中の素早い試行錯誤に使う

よくある手順:

1. `schema.prisma` を編集
2. `npx prisma generate` ← 型定義を更新
3. `npx prisma db push` ← DBに反映

※ `db push` は内部的に `generate` も実行するので、実際には 1 → 3 だけでもOKです

Q: `prisma migrate dev` と `prisma db push` はどう使い分けますか？

A: 開発段階によって使い分けます。

開発初期・個人開発:

- `prisma db push` を使う
- マイグレーションファイルを気にせず素早く変更
- やり直しも簡単

チーム開発・本番準備段階:

- `prisma migrate dev` を使う
- マイグレーションファイル（変更履歴）が残る
- チームメンバーが同じ変更を再現できる
- 本番デプロイ時は `prisma migrate deploy` で適用

BON-LOGの場合:

フェーズ	使うコマンド
開発中	<code>prisma db push</code>
CI/CD	<code>prisma migrate deploy</code>
本番デプロイ	<code>prisma migrate deploy</code>

Q: Prismaを更新したら型エラーが出るようになりました。どうすればよいですか？

A: 以下の手順で解決することがほとんどです。

```
Step 1: node_modules を再インストール
$ rm -rf node_modules
$ npm install
```

```
Step 2: Prisma Client を再生成
$ npx prisma generate
```

```
Step 3: TypeScriptサーバーを再起動
VSCode: Ctrl+Shift+P → "TypeScript: Restart TS Server"
```

それでも解決しない場合:

```
$ npx prisma db push --force-reset ⚠ データが消えるので注意！
$ npx prisma generate
```

7.16.2 スキーマ設計

Q: テーブル名やカラム名の命名規則はどうすべきですか？

A: BON-LOGでは以下の規則を採用しています。

Prismaモデル (TypeScript側) :

- モデル名: PascalCase (例: BonsaiShop, PostGenre)
- フィールド名: camelCase (例: userId, createdAt)

データベーステーブル (PostgreSQL側) :

- テーブル名: snake_case + 複数形 (例: bonsai_shops, post_genres)
- カラム名: snake_case (例: user_id, created_at)

@mapで変換:

```
model BonsaiShop {
  // ...
  userId String @map("user_id") // カラム名をsnake_caseに変換
  // ...
  @map("bonsai_shops") // テーブル名をsnake_caseに変換
}
```

なぜ変換するのか:

- TypeScript: camelCaseが慣例 (JavaScript文化)
- PostgreSQL: snake_caseが慣例 (SQL文化)
- @mapで両方の慣例に従える

Q: `onDelete: Cascade` と `onDelete: SetNull` の使い分けは？

A: 親レコードが削除されたとき、子レコードをどうするかを決めます。

****Cascade (連鎖削除) :****

- ユーザーが退会 → そのユーザーの投稿も全て削除
- 投稿が削除 → その投稿のいいねも全て削除
- 使うケース: 親がなくなったら子も不要なデータ (いいね、コメント、フォロー、通知、メディア)

****SetNull (NULLに設定) :****

- 元投稿が削除 → 引用投稿のquotePostIdがnullに
- 盆栽が削除 → 投稿のbonsaiIdがnullに
- 使うケース: 親がなくなっても子は残したいデータ (引用投稿、リポスト、盆栽と投稿の関連)

BON-LOGの実例:

```
user → post: Cascade    (退会したら投稿も削除)
post → like: Cascade    (投稿削除でいいねも削除)
post → quotePost: SetNull (引用元が消えても引用投稿は残る)
bonsai → post: SetNull  (盆栽を消しても投稿は残る)
```

Q: `@@unique` と `@@id` の違いは何ですか？

A: どちらも一意性を保証しますが、用途が異なります。

`@@id` (主キー) :

- レコードを一意に識別する「本家のID」
- NULLは不可
- テーブルに1つだけ
- 自動的にインデックスが作成される
- 例: `@@id([followerId, followingId])`

`@@unique` (ユニーク制約) :

- 値の重複を防ぐ「追加の一意性チェック」
- NULLは許可される (NULLは重複とみなされない)
- テーブルに複数設定可能
- 自動的にインデックスが作成される
- 例: `@@unique([userId, postId])`

使い分け:

テーブル	<code>@@id</code>	<code>@@unique</code>
Follow	[followerId, followingId]	なし (<code>@@id</code> で一意性を保証)
Like	id (cuid)	[userId, postId], [userId, commentId]
UserAnalytics	id (cuid)	[userId, date]

FollowはIDフィールドを持たず複合主キーを使う (中間テーブルパターン)

LikeはIDを持つが、`userId+postId`の重複を防ぐために`unique`制約も設定

7.16.3 パフォーマンスとトラブルシューティング

Q: 「Too many connections」エラーが出ます。どうすればよいですか？

A: データベース接続数の上限を超えています。以下の原因と対策を確認してください。

原因1: 開発環境でホットリロードのたびに新しい接続が作成されている

対策: `lib/db.ts`のシングルトンパターンが正しく設定されているか確認

→ `globalForPrisma.prisma` に保存されているか

原因2: 本番環境でコネクションプーリングが設定されていない

対策: Supabaseのプーリング用URLを使用

→ `DATABASE_URL` にプーリング用URL (ポート6543) を設定

→ `DIRECT_URL` にダイレクト接続URL (ポート5432) を設定

原因3: `$disconnect`を呼び忘れている (スクリプト実行時)

対策: `seed.ts`などのスクリプトでは必ず`.finally()`で切断

確認コマンド:

```
$ docker compose exec postgres psql -U bonsai -d bonsai_sns \
  -c "SELECT count(*) FROM pg_stat_activity;"
```

→ 現在の接続数を確認できる

Q: データベースをリセットして最初からやり直したいです。

A: 以下のコマンドを状況に応じて使い分けます。

方法1: Dockerのデータボリュームを削除

```
$ docker compose down -v  ← データも含めて完全リセット
$ docker compose up -d postgres
$ npx prisma db push
$ npx prisma db seed
```

方法2: Prismaのリセットコマンド

```
$ npx prisma migrate reset
→ データベースを削除して再作成
→ マイグレーションを再適用
→ シードを自動実行
```

方法3: テーブルデータのみ削除（テーブル構造は維持）

```
$ npx prisma db execute --stdin <<< "TRUNCATE TABLE posts CASCADE;"
→ CASCADE: 関連テーブルのデータも連鎖削除
```

⚠ 本番環境では絶対に実行しないこと！

Q: 「P2025: An operation failed because it depends on one or more records that were required but not found.」エラーの意味は？

A: 操作対象のレコードが見つからないというエラーです。

よくある原因:

1. update/deleteで指定したIDのレコードが存在しない
→ findUniqueで存在確認してからupdate/delete
2. connectで指定したリレーション先が存在しない
→ connectOrCreateを使う
3. 他のリクエストが先にレコードを削除した（競合状態）
→ try-catchでエラーをハンドリング

対策例:

```
// ❌ レコードがない場合にP2025エラーになる
await prisma.post.delete({ where: { id: postId } })

// ✅ 存在チェックしてから削除
const post = await prisma.post.findUnique({ where: { id: postId } })
if (!post) return { error: '投稿が見つかりません' }
await prisma.post.delete({ where: { id: postId } })

// ✅ またはdeleteの結果を無視
await prisma.post.deleteMany({ where: { id: postId } })
// deleteManyは0件でもエラーにならない
```

Q: 開発中にスキーマを変更したら、既存のデータが消えてしまいました。

A: `prisma db push` でスキーマを変更する際、破壊的変更（カラム削除、型変更、必須カラム追加等）があるとデータが失われることがあります。

安全な変更:

- ✓ 新しいカラムの追加（Optionalまたはdefault付き）
- ✓ 新しいテーブルの追加
- ✓ インデックスの追加・削除
- ✓ @@mapの変更（テーブル名・カラム名の変更）

危険な変更:

- △ カラムの削除 → そのカラムのデータが消える
- △ カラムの型変更 → データの変換に失敗する可能性がある
- △ 必須カラムの追加（@default なし） → 既存データに値がないためエラー

ベストプラクティス:

1. 重要なデータがある場合は先にバックアップ
2. 必須カラム追加時は@default()を付ける
→ 例: `isPublic Boolean @default(true)`
3. 本番環境では必ず `prisma migrate dev` を使う
→ 変更内容がSQLファイルとして残る
→ レビュー可能

7.16.4 設計判断

Q: 同じような情報を複数テーブルに持つのは冗長ではないですか？（例: `DraftPost` と `Post`）

A: 一見冗長に見えますが、テーブルを分ける明確な理由があります。

`DraftPost`と`Post`を分離する理由:

1. **データの品質管理**: `posts`テーブルは「公開済みの完成したデータ」、`draft_posts`は「未完成のデータ」。分離することでデータの管理が容易になる。
2. **クエリの効率**: タイムラインは`posts`テーブルのみ参照。下書きが混ざるとWHERE句が必要 → 遅くなる
3. **ライフサイクルの違い**: 下書きは頻繁に更新される（自動保存）、投稿は基本的に更新されない（immutable）。テーブルの設計に反映させる。

一方、統合するのが適切なケース:

- `PostMedia`と`DraftPostMedia`のように構造が完全に同じ場合でも、Cascadeの挙動を独立に管理するために分離している
- 将来的に`DraftPost`に下書き固有のフィールド（自動保存日時等）を追加しやすい

Q: JSON型（`Json`）とリレーションテーブル、どちらを使うべきですか？

A: それぞれの特性を理解して使い分けます。

JSON型が適しているケース:

- データ構造が固定されていない (将来変わる可能性がある)
- データを直接検索する必要がない
- 例: `SystemSetting`の`value`, `User`の`notification_preferences`

リレーションテーブルが適しているケース:

- データを検索・集計する必要がある
- データの整合性が重要
- 例: `PostGenre` (ジャンル別検索が必要)

比較:

観点	JSON型	リレーション
スキーマ柔軟性	自由	マイグレ必要
検索性能	低い	インデックス可
データ整合性	アプリ側依存	DB制約で保証
TypeScript型	手動で定義	自動生成
集計クエリ	困難	groupBy等可

▶ 理解度チェック: FAQ

7.17 学習ロードマップ

ここまでの内容を学んだ後、データベースとPrismaのスキルをさらに伸ばすための学習指針を示します。

データベース学習のステップ:

Level 1: 基礎（この章で学んだこと） \nRDBの基本概念（テーブル、主キー、外部キー） \nPrismaのセットアップと基本操作（CRUD） \nschema.prismaの書き方（モデル定義、リレーション） \ninclude/selectでのリレーション取得 \nカーソルベースのページネーション \nトランザクション

Level 2: 実践（次章以降で使うスキル） \nServer Actions内でのPrisma操作 \n認証と連携したデータアクセス制御 \nZodバリデーション + Prismaの組み合わせ \nReact Queryとの連携（クライアント側キャッシュ） \nエラーハンドリングのパターン

Level 3: 最適化（アプリが成長したら必要） \nインデックス設計と実行計画の読み方 \nN+1問題の検出と解決 \nコネクションプーリングの設定 \nバッチ処理とバルクオペレーション \nクエリログの分析

Level 4: 発展（本番運用で必要になる知識） \nマイグレーション戦略（ダウンタイムゼロ移行） \nデータベースのバックアップと復旧 \n監視とアラート設定 \nリードレプリカとスケーリング \nSQLの基礎知識（デバッグ・最適化時に必要）

7.17.1 次に学ぶべき公式ドキュメント

おすすめの学習リソース:

Prisma公式:

1. **Prisma Docs** (prisma.io/docs) -- まず「Concepts」セクションを一読 → 「Guides」で実践的なパターンを学ぶ
2. **Prisma Client API Reference** -- 各メソッドの引数と戻り値を確認するときに使う。findMany, create, update等の詳細オプション

3. **Prisma Schema Reference** -- 属性 (@id, @unique, @default 等) の全一覧。データ型の対応表

PostgreSQL公式:

4. **PostgreSQL Documentation** -- Chapter 11: Indexes (インデックスの詳細)、Chapter 13: Concurrency Control (同時アクセス制御)
5. **PostgreSQL Wiki - Performance Optimization** -- 本番運用でパフォーマンスチューニングが必要になったときに参照

Next.js + Prisma:

6. **Vercel + Prisma Deployment Guide** -- Vercelへのデプロイ時のDB設定、コネクションプーリングの設定
7. **Next.js Data Fetching Docs** -- Server ComponentsでのDB操作パターン、キャッシュとrevalidateの戦略

7.17.2 この章の知識を活かす練習課題

以下の練習課題に取り組むことで、この章で学んだ知識をより深く定着させることができます。

練習課題一覧:

課題1 (Level 1: 基礎) : ジャンルのCRUD操作 -- 目安時間: 30分

- Prisma Studioでジャンル一覧を確認する
- 新しいジャンルをcreateで追加する
- ジャンルをupdateで名前変更する
- 不要なジャンルをdeleteで削除する

課題2 (Level 1: 基礎) : リレーションを使ったデータ取得 -- 目安時間: 1時間

- ユーザーの投稿一覧を include 付きで取得する
- 投稿にいいね数とコメント数を _count で含める
- 特定ジャンルの投稿を some フィルタで取得する

課題3 (Level 2: 実践) : カーソルページネーションの実装 -- 目安時間: 1時間

- 投稿一覧を20件ずつ取得する関数を作成
- cursor パラメータで次のページを取得できる
- hasMore フラグで「次のページがあるか」を返す

課題4 (Level 2: 実践) : トランザクションの実装 -- 目安時間: 1時間30分

- いいね追加時に通知も同時に作成する
- \$transactionを使って原子性を保証する
- エラーハンドリングを適切に実装する

課題5 (Level 3: 最適化) : クエリパフォーマンスの改善 -- 目安時間: 2時間

- タイムライン取得クエリをログで確認する
- N+1問題が発生していないかチェックする
- select を使って転送データ量を最小化する
- Promise.all で並列実行に書き換える

7.17.3 よくある失敗パターンと回避方法

開発初心者がデータベース操作でよく犯す失敗を事前に知っておくことで、同じ轍を踏まずに済みます。

失敗パターン集:

失敗1: パスワードをselectで返してしまう

```
// Bad: passwordフィールドが含まれる
const user = await prisma.user.findUnique({ where: { id } })
res.json(user) // passwordフィールドが含まれる！

// Good: selectで必要なフィールドのみ取得
const user = await prisma.user.findUnique({
  where: { id },
  select: {
    id: true,
    nickname: true,
    avatarUrl: true,
    // passwordは含めない
  },
})
```

失敗2: 認可チェックを忘れて他人のデータを操作してしまう

```
// Bad: 誰の投稿でも削除可能
await prisma.post.delete({
  where: { id: postId },
})

// Good: 自分の投稿のみ削除可能
await prisma.post.delete({
  where: {
    id: postId,
    userId: session.user.id, // 自分の投稿のみ
  },
})
```

失敗3: awaitを忘れてPromiseが解決する前に結果を使う


```
// Bad: Promiseオブジェクトが返る
const user = prisma.user.findUnique(...)
console.log(user.nickname) // Promiseオブジェクト

// Good: awaitで結果を待つ
const user = await prisma.user.findUnique(...)
console.log(user?.nickname) // 正しい値
```

失敗4: findUniqueの結果がnullかチェックしない

```
// Bad: postがnullだとクラッシュ
const post = await prisma.post.findUnique(...)
return post.content

// Good: nullチェックを行う
const post = await prisma.post.findUnique(...)
if (!post) {
  return { error: '投稿が見つかりません' }
}
return post.content // 安全
```

失敗5: 無限ループで全レコードを取得してサーバーダウン

```
// Bad: 100万件あったらメモリ不足
const allPosts = await prisma.post.findMany()

// Good: 必ず件数制限をかける
const posts = await prisma.post.findMany({
  take: 20,
})
```

初心者へのアドバイス: 上記の失敗パターンは、経験豊富な開発者でもうっかり犯すことがあります。コードレビュー時にチェックリストとして活用し、チーム全体で品質を維持しましょう。

▶ 理解度チェック: 学習ロードマップ

7.18 まとめ

この章では以下を学びました:

1. **リレーショナルデータベースの基本**: テーブル、主キー、外部キー、リレーション (Excelの超強力版としての理解)
2. **技術選定**: PostgreSQL / Prisma / Supabase / cuid を選んだ理由と他の選択肢との比較
3. **Prismaの概要**: ORM (通訳) の役割、型安全、直感的なAPI、SQLとの対比
4. **schema.prismaの書き方**: データ型、属性、リレーション定義
5. **BON-LOGのスキーマ設計 (全モデル)**: User, Post, Follow, Like, Comment, Bookmark, Block, Mute, FollowRequest, Notification, BonsaiShop/ShopReview, Event, Report, AdminUser/AdminLog, Conversation/Message, Payment, Bonsai/BonsaiRecord等の設計と実装コード
6. **CRUD操作**: create, findUnique, findMany, update, delete
7. **includeとselect**: リレーション先データの取得と最適化
8. **ページネーション**: カーソルベースの無限スクロール対応
9. **トランザクション**: 複数操作の原子性保証
10. **マイグレーション**: スキーマ変更の管理 (db push vs migrate dev)
11. **追加モデル詳細**: Poll, Hashtag, UserAnalytics, SystemSetting, ブラックリスト, 下書き・予約投稿, NextAuth認証モデル, CommentMedia, ShopChangeRequest, ContactInquiry, UserDevice, CommentThreadMute等
12. **高度なPrismaパターン**: groupBy, \$queryRaw, updateMany, createMany, upsert, connectOrCreate, aggregate, distinct, ネスト書き込み, バッチ処理
13. **データベース最適化**: N+1問題, インデックス設計, コネクションプーリング, select/includeの使い分け, 複合インデックス, クエリ分析, メモリ効率
14. **シードデータ**: 初期データの投入と冪等な設計
15. **FAQ**: 開発現場でよくある疑問と解決策
16. **学習ロードマップ**: 次のステップと練習課題

この章の知識が活きる場面:

次章以降で、ここで学んだDB操作を実際に使います:

第8章 (認証) → User / Account テーブルの操作

第9章 (API) → Server Actionsで prisma.post.create() 等を使う

第10章 (フィード) → include + カーソルページネーション

第11章 (検索) → \$queryRaw + pg_trgm による全文検索

第12章 (通知) → トランザクションで通知を一括作成

データベースはアプリケーションの「土台」です。

この章の理解が深いほど、以降の章がスムーズに進みます。

次章では、NextAuth.js (Auth.js v5) を使った認証システムの構築を学びます。